

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/365415242>

Physics-Informed Machine Learning: A Survey on Problems, Methods and Applications

Preprint · November 2022

DOI: 10.48550/arXiv.2211.08064

CITATIONS

7

READS

994

7 authors, including:



Yichi Zhang

Tsinghua University

13 PUBLICATIONS 46 CITATIONS

SEE PROFILE



Chengyang Ying

Tsinghua University

21 PUBLICATIONS 47 CITATIONS

SEE PROFILE



Hang Su

Peking University

176 PUBLICATIONS 6,906 CITATIONS

SEE PROFILE

Physics-Informed Machine Learning: A Survey on Problems, Methods and Applications

Zhongkai Hao, Songming Liu, Yichi Zhang, Chengyang Ying, Yao Feng,
Hang Su, Jun Zhu

Abstract—Recent advances of data-driven machine learning have revolutionized fields like computer vision, reinforcement learning, and many scientific and engineering domains. In many real-world and scientific problems, systems that generate data are governed by physical laws. Recent work shows that it provides potential benefits for machine learning models by incorporating the physical prior and collected data, which makes the intersection of machine learning and physics become a prevailing paradigm. By integrating the data and mathematical physics models seamlessly, it can guide the machine learning model towards solutions that are physically plausible, improving accuracy and efficiency even in uncertain and high-dimensional contexts. In this survey, we present this learning paradigm called Physics-Informed Machine Learning (PIML) which is to build a model that leverages empirical data and available physical prior knowledge to improve performance on a set of tasks that involve a physical mechanism. We systematically review the recent development of physics-informed machine learning from three perspectives of machine learning tasks, representation of physical prior, and methods for incorporating physical prior. We also propose several important open research problems based on the current trends in the field. We argue that encoding different forms of physical prior into model architectures, optimizers, inference algorithms, and significant domain-specific applications like inverse engineering design and robotic control is far from being fully explored in the field of physics-informed machine learning. We believe that the interdisciplinary research of physics-informed machine learning will significantly propel research progress, foster the creation of more effective machine learning models, and also offer invaluable assistance in addressing long-standing problems in related disciplines.

Index Terms—Physics-Informed Machine Learning, AI for Science, PDE/ODE, Symmetry, Intuitive Physics

1 INTRODUCTION

The paradigm of scientific research in recent decades has undergone a revolutionary change with the development of computer technology. Traditionally, researchers used theoretical derivation combined with experimental verification to study natural phenomena. With the development of computational methods, a large number of methods based on computer numerical simulation have been developed to understand complex real systems. Nowadays, with the automation and batching of scientific experiments, scientists have accumulated a large amount of observational data. *The paradigm of (data-driven) machine learning is to understand and build models that leverage empirical data to improve performance on some set of tasks [1].* It is an important research area to promote the development of modern science and engineering technology with the aid of learning from observational data since we could extract a lot of information from data.

As part of the remarkable progress of machine learning in recent years, deep neural networks [2] have achieved milestone breakthroughs in the fields of computer vision [3], natural language processing [4], speech processing [5], and reinforcement learning [6]. Their flexibility and scalability allow neural networks to be easily applied to many different domains, as long as there is a sufficient amount

of data. The powerful abstraction ability of deep neural networks also motivates researchers to apply them on scientific problems in modeling physical systems. For example, AlphaFold 2 [7] has revolutionized the paradigm of protein structure prediction. Similarly, FourCastNet [8] has built an ultra-large learning-based weather forecasting system that surpasses traditional numerical forecasting systems. Deep Potential [9] proposed neural models for learning large-scale molecular potential satisfying symmetry. The integration of prior knowledge of physics, which represents a high-level abstraction of natural phenomena or human behaviors, with data-driven machine learning models is becoming a new paradigm since it has the potential to facilitate novel discoveries and solutions to challenges across a diverse range of domains.

Moreover, despite the impressive advancements of machine learning based models, there remain significant limitations when deploying purely data-driven models in real-world applications. In particular, data-driven machine learning models can suffer from several limitations such as a lack of robustness, interpretability, and adherence to physical constraints or commonsense reasoning. In computer vision, recognizing and understanding the geometry, shape, texture, and dynamics from images or videos can pose a significant challenge for deep neural networks, which can lead to limitations in their ability to extrapolate beyond their training data. Additionally, such models have demonstrated suboptimal performance outside of their training distribution [10] and are susceptible to adversarial attacks via human-imperceptible noise [11]. In deep reinforcement learning, an agent may learn to take actions that result in

- Zhongkai Hao, Songming Liu, Yichi Zhang, Chengyang Ying, Yao Feng, Hang Su, Jun Zhu are with Dept. of Comp. Sci. & Techn., Institute for AI, BNRist Center, Tsinghua-Bosch Joint ML Center, Tsinghua University, Email: {hzj21, liusm18, zyc22, ycy21, y-feng20}@mails.tsinghua.edu.cn, {suhangss, dcszj}@mail.tsinghua.edu.cn,
- Jun Zhu is also with RealAI.

higher rewards through trial and error, but it may not understand the underlying physical mechanisms. These issues are particularly pertinent in scientific problems where the laws of physics and scientific principles govern the behavior of the system under study. For example, data obtained from scientific and engineering experiments often tends to be sparse and noisy due to the high cost and the presence of environmental and device-related noise, which can result in significant generalization errors in common machine learning models. One possible explanation for the generalization errors observed in common statistical learning models is their sole reliance on empirical data without incorporating any understanding of the internal physical mechanisms that generate the data. By contrast, humans have the capacity to extract concise physical laws from data, which allows them to interact with the world more efficiently and robustly [12], [13]. *The integration of physical laws or constraints into machine learning models, therefore, presents new opportunities for traditional scientific research, substantially advancing the discovery of new knowledge, and facilitating the research in persistent issues of machine learning, such as robustness, interpretability, and generalization* [12], [14].

Numerous methods have been proposed by researchers to integrate physical knowledge with machine learning, which are tailored to the specific context of the problem and the representation of physical constraints. While the existing literature on this topic is extensive and multifaceted, *we propose to establish a concise and formalized concept in the form of Physics-Informed Machine Learning (PIML), which is a paradigm that seeks to construct models that make use of both empirical data and prior physical knowledge to enhance performance on tasks that involve a physical mechanism.* In this survey, we propose a concise theoretical framework for machine learning problems with physical constraints, based on probabilistic graphical models using latent variables to represent the real state of a system that satisfies physical prior constraints. Our framework provides a unified view of such problems and is flexible in handling physical systems with various constraints, including high-dimensional observational data. It can be combined with methods like autoencoders and dynamic mode decomposition. Moreover, we introduce a physical bottleneck network that can learn low-dimensional, physics-aware representations from high-dimensional, noisy data based on the choice of physical priors.

As an attractive research area, several surveys have been recently published. Karniadakis [12] provides a comprehensive overview of the historical development of PIML. Cuomo et al. [15] focus on algorithms and applications of PINNs. Beck et al. [16] review the theoretical results obtained using NNs for solving PDEs. Other studies have focused on subdomains or applications of PIML, such as fluid mechanics [17], uncertainty quantification [18], domain decomposition [19], and dynamic systems [20]. Zubov et al. [21], Cheung et al. [22], Blechschmidt et al. [23], Pratama et al. [24], and Das et al. [25] provide further examples and tutorials with software. Additionally, Rai et al. [26], Meng et al. [27], Willard et al. [28], and Frank et al. [29] focus on other hybrid modeling paradigms that integrate machine learning with physical knowledge. In this survey, we summarize the developments in PIML from the perspective of machine

learning researchers, providing a comprehensive review of algorithms, theory, and applications, and proposing future challenges for PIML that will advance interdisciplinary research in this area.

In this review paper, we begin by presenting mathematical preliminaries and background. We then discuss the development of physics-informed machine learning methods for both scientific problems and traditional machine learning tasks, such as computer vision and reinforcement learning. For scientific problems, we focus on representative methods like PINNs and DeepONet, as well as current improvements, theories, applications, and unsolved challenges. We also summarize the methods that incorporate physical prior knowledge into computer vision and reinforcement learning, respectively. Finally, we describe some representative and challenging tasks for the machine learning community.

2 PROBLEM FORMULATION

In this section, we introduce the concept and commence by examining fundamental problems in physics-informed machine learning (PIML). We elaborate on the representation methodology for physical knowledge, the approach for integrating physical knowledge into machine learning models, and the practical problems that PIML resolves, as is illustrated in Figure 1.

2.1 Representation of Physics Prior

Physical prior knowledge refers to the understanding of the fundamental laws and principles of physics that describe the behavior of the physical world. This knowledge can be categorized into various types, ranging from strong to weak inductive biases, such as partial differential equations (PDEs), symmetry constraints, and intuitive physical constraints. PDEs, ODEs, and SDEs are prevalent in scientific and engineering domains and can be easily integrated into machine learning models, as they have analytical mathematical expressions. For example, PINNs [30] use PDEs and ODEs as regularization terms in the loss function, while NeuralODE [31] construct a neural architecture that obeys ODEs.

Symmetry and intuitive physical constraints are weaker inductive biases than PDEs/ODEs, which can be represented in various ways, such as designing network architectures that respect these constraints or incorporating them as regularization terms in the loss function. Symmetry constraints include translation, rotation, and permutation invariance or equivariance, which are widely used when designing novel network architectures, e.g., PointNet [32] and Graph Convolutional Networks (GCN) [33]. Intuitive physics, also known as naive physics, is the interpretable physical commonsense about the dynamics and constraints of objects in the physical world. Although intuitive physical constraints are essential and straightforward, mathematically and systematically representing them remains a challenging task. We will elaborate on the different types of physical priors in the following.

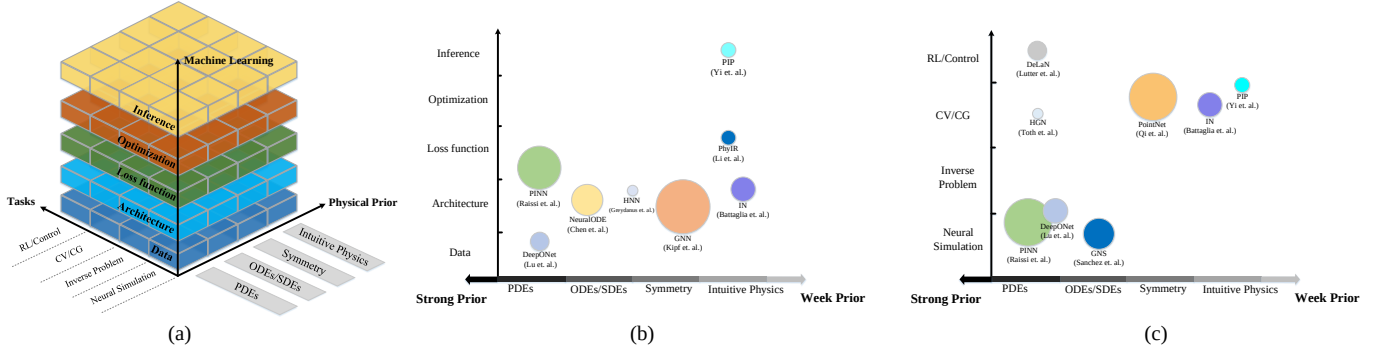


Fig. 1: An overview of physics-informed machine learning. We review various methods of incorporating physical prior knowledge into machine learning models, ranging from strong to weak forms, such as PDEs/ODEs/SDEs, symmetry, and intuitive physics. These physical priors can be incorporated into different aspects of machine learning models, such as data, model architecture, loss function, optimizer, and inference algorithm. We also highlight different applications of physics-informed machine learning in tasks such as neural simulation, inverse problems, CV/NLP, and RL/control. Finally, we identify some significant areas for exploration in the PIML field, such as physics-informed optimizers and physics-informed inference methods.

2.1.1 Differential Equations

Differential equations represent precise physical laws that can effectively describe various scientific phenomena. In this paper, we consider a physical system that exists on a spatial or spatial-temporal domain $\Omega \subseteq \mathbb{R}^d$, where $u(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^m$ denotes a vector of state variables, which are the physical quantities of interest, and also functions of the spatial or spatial-temporal coordinates $\mathbf{x}$. The physical laws governing this system are characterized by partial differential equations (PDEs), ordinary differential equations (ODEs), or stochastic differential equations (SDEs). These equations are known as the *governing equations*, with the unknowns being the state variables u . The system is either parameterized or controlled by $\theta \in \Theta$, where θ could be either a vector or a function incorporated in the governing equations. Unless otherwise stated, Table 1 provides a detailed list of the notations used in the following sections.

In many domains of science and engineering, real-world physical systems can be modeled using differential equations that are based on different domain-specific assumptions and simplifications. These models can be used to approximate the behavior of these systems. In this paper, we introduce the basic concepts of differential equations. Formally, we consider a system with state variables $u(\mathbf{x}) \in \mathbb{R}^m$, where $\mathbf{x} \in \Omega$ is the domain of definition. For simplicity, we use $\mathbf{x}$ to denote the spatial-temporal coordinates, i.e., $\mathbf{x} = (x_1, \dots, x_d) \in \Omega$ for time-independent systems and $\mathbf{x} = (x_1, \dots, x_{d-1}, t) \in \Omega$ for time-dependent systems. The behavior of the system can be represented by ordinary or partial differential equations (ODEs/PDEs) as follows:

$$\text{Partial differential equation: } \mathcal{F}(u; \theta)(\mathbf{x}) = 0, \quad (1)$$

$$\text{Initial Conditions: } \mathcal{I}(u; \theta)(\mathbf{x}, t_0) = 0, \quad (2)$$

$$\text{Boundary Conditions: } \mathcal{B}(u; \theta)(\mathbf{x}, t) = 0. \quad (3)$$

Without confusion of notation, we rewrite equivalent forms of Eq. (1) as:

$$\mathcal{F}(u; \theta)(\mathbf{x}) \equiv \mathcal{F}(u, \mathbf{x}; \theta) = 0, \mathbf{x} \in \Omega. \quad (4)$$

Notations	Description
u	state variables of the physical system
$\mathbf{x}$	spatial or spatial-temporal coordinates
x	spatial coordinates
t	temporal coordinates
θ	parameters for a physical system
w	weights of neural networks
$\frac{\partial}{\partial x_i}$	partial derivatives operator
$\mathcal{D}_i^k$	$\frac{\partial^k}{\partial x_i^k}$, k -order derivatives for variable x_i
∇	nabla operator (gradient)
Δ	Laplace operator
$\int$	integral operator
$\mathcal{F}$	differential operator representing the PDEs/ODEs
$\mathcal{I}$	initial conditions (operator)
$\mathcal{B}$	boundary conditions (operator)
Ω	spatial or spatial-temporal domain of the system
Θ	space of the parameters θ
$\mathcal{W}$	space of weights of neural networks
$\mathcal{L}$	loss functions
$\mathcal{L}_r$	residual loss
$\mathcal{L}_b$	boundary condition loss
$\mathcal{L}_i$	initial condition loss
l_k	residual (error) terms
$\ \cdot\ $	norm of a vector or a function

TABLE 1: A table of mathematical notations.

For time-dependent cases (i.e., dynamic systems), we need to pose the initial conditions for state variables and sometimes their derivatives at a certain time t_0 that can be described as $\mathcal{I}(u; \theta)(\mathbf{x}, t_0) = 0, \mathbf{x} \in \Omega_0$. For systems characterized by PDEs, we also need constraints for state variables on the boundary of the spatial domain $\partial\Omega$ to make the system well-posed. For boundary points $\mathbf{x} \in \partial\Omega$, we have the boundary conditions as $\mathcal{B}(u; \theta)(\mathbf{x}, t) = 0, \mathbf{x} \in \partial\Omega$. If there are no corresponding constraints of initial conditions and boundary conditions, we can define $\mathcal{I}(u; \theta) \triangleq 0$ and $\mathcal{B}(u; \theta) \triangleq 0$.

2.1.2 Symmetry Constraints

Symmetry constraints are considered a weaker inductive bias compared to partial differential equations (PDEs) or ordinary differential equations (ODEs). Symmetry constraints

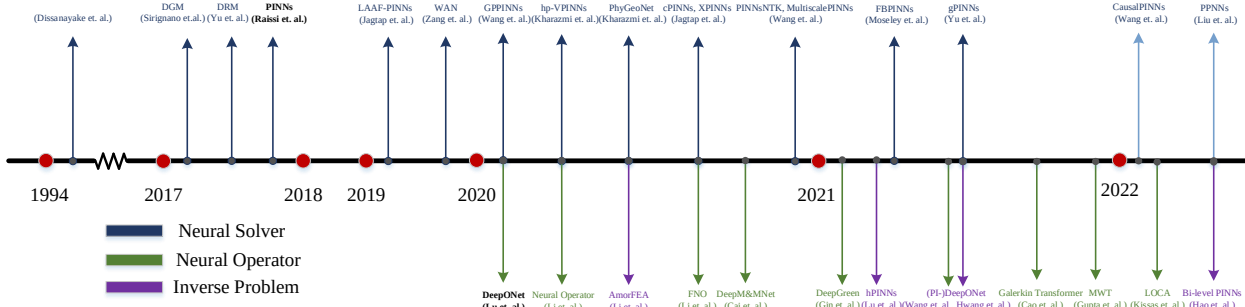


Fig. 2: A chronological overview of important methods for neural simulation (neural solver and neural operator) and inverse problems (inverse design) of physics-informed machine learning. The earliest work could be traced back to [34].

refer to a collection of transformations that can be applied to objects, where the abstract set of symmetries is capable of transforming diverse objects. Examples of symmetry constraints are translation, rotation, and permutation invariance or equivariance. In mathematics, symmetries are represented as invertible transformations that can be composed which can be formulated as the concept of groups [35].

Symmetries or invariants can be incorporated into machine learning to improve the performance of algorithms, depending on the type of data and problem being addressed. There are several types of symmetries, such as translation, rotation, reflection, scale, permutation, and topological invariance, which can be useful in different scenarios [36]. For example, translation invariance is important for data that is shift-invariant, like images or time-series data. Similarly, rotational symmetry is essential for data that is invariant to rotations [32], like images or point clouds, and reflection symmetry is critical for data that is invariant to reflections, such as images or shapes. Scale invariance is useful for data that is invariant to changes in scale, such as images or graphs, while permutation invariance is significant for data that is invariant to permutations of its elements, such as sets or graphs [33]. Finally, topological invariance is important for data that is invariant to topological transformations, such as shape or connectivity changes.

The symmetry constraint is that for data $x \in \mathcal{D}$, there exist an operation $s : \mathcal{D} \rightarrow \mathcal{D}$, such that the property function $\varphi(\cdot) : \mathcal{D} \rightarrow \mathbb{R}^k$ is the same under the symmetric operation, i.e.

$$\varphi(x) = \varphi(s(x)). \quad (5)$$

Incorporating symmetries or invariants can provide numerous advantages for machine learning models. These benefits include improved generalization performance, reduced data redundancy, increased interpretability, and better handling of complex data structures. Symmetries or invariants can aid in improving generalization by providing prior knowledge about the data and by training the model on a representative subset of the data, reducing redundancy. By incorporating symmetries or invariants, we can also gain insights into the underlying structure of the data, making the models more interpretable, especially in scientific or engineering applications. Finally, incorporating symmetries or invariants can be useful for handling complex data structures such as graphs or manifolds, which may not have a simple Euclidean structure. By respecting the underlying

geometry of the data, we can design algorithms that can handle these complex symmetries or invariants.

2.1.3 Intuitive Physics

Intuitive physics refers to the common-sense knowledge about the physical world that humans possess that they use to reason about and make predictions, such as the understanding that objects fall to the ground when dropped. Integrating intuitive physics into machine learning involves incorporating this prior knowledge into the design of machine learning algorithms to improve their performance [37], [38]. There are several commonly used intuitive physics principles that can be incorporated into machine learning models such as [39]

- Object permanence: The understanding that objects continue to exist even when they are no longer visible;
- Gravity: The understanding that objects are attracted to each other with a force proportional to their mass and inversely proportional to the square of their distance;
- Newton's laws of motion: The principles that describe the relationship between an object's motion and the forces acting upon it;
- Conservation laws: The principles that describe the conservation of energy, momentum, and mass in physical systems.

These principles can be used as physical priors or constraints in machine learning models to improve their accuracy, robustness, and interpretability including computer vision, robotics, and natural language processing. For example, object permanence can be used to improve object tracking algorithms by predicting the future location of an object based on its previous motion. Gravity can be used to simulate the behavior of objects in a physical environment, such as in physics-based games or simulations. Therefore, intuitive physics can help us to develop machine learning models that can reason about and predict the behavior of objects in the physical world.

However, intuitive physics is a challenging concept to formalize using traditional mathematical models and equations, hindering its integration into machine learning algorithms. In general, intuitive physics can be incorporated as constraints or regularizers to enhance machine learning

models [30]. For instance, by including the conservation of energy or momentum as constraints, we can design models to predict the behavior of physical systems. Additionally, physical simulations can generate training data for machine learning models, improving their understanding of physical phenomena and validating their performance [40], [41]. Finally, hybrid models that combine machine learning and physics can leverage the strengths of both approaches [42]. For example, a physics-based model can generate initial conditions for a machine learning model, which can refine those predictions using observed data.

2.2 Possible Ways towards PIML

A fundamental issue for PIML is how physical prior knowledge is integrated into machine learning models. As is illustrated in Figure 1, the training of a machine learning model involves several fundamental components including data, model architecture, loss functions, optimization algorithms, and inference. The incorporation of physical prior knowledge can be achieved through modifications to one or more of these components.

Formally, let $\mathcal{D} = \{(x_i, y_i)\}$ denote a given training dataset. Machine learning tasks can be generally put as searching for a model f from a hypothesis space $\mathcal{H}$. The performance of a particular model on dataset $\mathcal{D}$ is often characterized by a loss function $\mathcal{L}(f; \mathcal{D})$. Then the problem is cast as solving an optimization objective as

$$\min_{f \in \mathcal{H}} \mathcal{L}(f; \mathcal{D}) + \Omega(f), \quad (6)$$

where $\Omega(f)$ is a regularization term that introduces some inductive bias for better generalization. Then, we solve problem (6) using an optimizer $OPT(\cdot)$ that outputs a model f from some initial guess f_0 , i.e., $f = OPT(\mathcal{H}, f_0)$.

Physics-informed machine learning is a direction of ML that aims to leverage physical prior knowledge and empirical data to improve performance on a set of tasks that involve a physical mechanism. Training a machine learning model consisting of several basic components, i.e. data, model architecture, loss functions, optimization algorithms, and inference. In general, there are various approaches to incorporating physical prior into different components of machine learning:

- *Data*: we could augment or process the dataset utilizing available physical prior like symmetry. Mathematically we have $\mathcal{D}_p = P(\mathcal{D})$ where $P(\cdot)$ denotes a preprocessing or augmentation operation using physical prior.
- *Model*: we could embed physical prior into the model design (e.g., network architecture). We usually achieve this by introducing inductive biases guided by the physical prior into the hypothesis space, i.e., $f \in \mathcal{H}_p \subseteq \mathcal{H}$.
- *Objective*: we could design better loss functions or regularization terms using given physical priors like ODE/PDE/SDEs, i.e. replace $\mathcal{L}(f; \mathcal{D})$ or $\Omega(f)$ with $\mathcal{L}_p(f; \mathcal{D})$ or $\Omega_p(f)$.
- *Optimizer*: we could design better optimization methods that are more stable or converge faster. We use OPT_p to denote the optimizer that incorporates the physical prior.

- *Inference*: we could enforce the physical constraints by using modifying the inference algorithms. For example, we could design a post-processing function g_p , we use $g_p(x, f(x))$ instead of $f(x)$ when inferring.

First, data could be augmented or synthesized for problems with symmetry constraints or known PDEs/ODEs. Models could learn from these generated data. Second, the architecture of the model may need to be redesigned and evaluated. Physical laws such as PDEs/ODEs, symmetry, conservation laws, and the possible periodicity of data may require us to redesign the structure of the current neural network to meet the needs of practical problems. Third, loss functions and optimization methods for general deep neural networks may not be optimal for training models that incorporate physical constraints. For example, when physical constraints are used as regular term losses, the weight adjustment of each loss function is very important, and commonly used first-order optimizers such as Adam [43] are not necessarily suitable for the training of such models. Finally, for pre-trained machine learning models, we might also design different inference algorithms to enforce physical prior or enhance interpretability.

First, physical prior knowledge can be integrated into the data by augmenting or synthesizing it for problems with symmetry constraints or known partial differential equations (PDEs) or ordinary differential equations (ODEs). By training models on such generated data, they can learn to account for the physical laws that govern the problem. Second, the model architecture may need to be redesigned and evaluated to accommodate physical constraints. Physical laws such as PDEs/ODEs, symmetry, conservation laws, and periodicity of data may necessitate a rethinking of the structure of the neural network. Third, standard loss functions and optimization algorithms for deep neural networks may not be optimal for models that incorporate physical constraints. For instance, when physical constraints are used as regular term losses, the weight adjustment of each loss function is crucial, and commonly used first-order optimizers such as Adam are not necessarily suitable for training such models. Finally, for pre-trained machine learning models, different inference algorithms can be designed to enforce physical prior knowledge or improve interpretability. By incorporating physical prior knowledge into one or more of these components, machine learning models can achieve improved performance and better align with practical problems that adhere to the laws of physics.

2.3 Tasks of PIML

Physics-Informed Machine Learning (PIML) can be applied to various problem settings of statistical machine learning such as supervised learning, unsupervised learning, semi-supervised learning, reinforcement learning, etc. However, PIML requires real-world physical processes, and we must have some knowledge about them; otherwise, it would turn into pure statistical learning. The existing works on PIML can be categorized into two classes: using PIML to solve scientific problems and incorporating physical priors to solve machine learning problems.

The field of physics-informed machine learning (PIML) has witnessed significant progress in addressing scientific problems that rely on accurate physical laws, often formulated by differential equations. PIML can be classified into two main categories, namely, “neural simulation” and “inverse problems” related to physical systems [12]. The neural simulation focuses on predicting or forecasting the states of physical systems using physical knowledge and available data. Examples of forward problems include solving PDE systems, predicting molecular properties, and forecasting future weather patterns. In contrast, inverse problems aim to identify a physical system that satisfies the given data or constraints. Examples of inverse problems include scientific discovery of PDEs from data and optimal control of PDE systems. The remarkable advancements in PIML have enabled the development of accurate models and efficient algorithms that combine physical knowledge and machine learning. This integration has opened up new opportunities for interdisciplinary research, enabling insights into complex problems across various fields such as computational biology, geophysics and environmental science [44], etc. PIML has the potential to revolutionize scientific discovery and technological innovation. Figure 2 shows a chronological summary of recent work proposed in this area. The ongoing research in this field continues to push the boundaries of what is possible.

Incorporating physical knowledge into machine learning models can significantly enhance their effectiveness, simplicity, and robustness. For instance, PIML can improve the efficiency and robustness of robots’ design [45]. In computer vision, PIML can improve object detection and recognition and increase models’ robustness to environmental changes [37]. PIML can also improve natural language processing models’ ability to generate and comprehend text in numerous disciplines, and it can enhance the accuracy and efficiency of reinforcement learning models by integrating physical knowledge [46]. By incorporating physical knowledge, PIML can overcome the limitations of traditional machine learning algorithms, which typically require large amounts of data to learn. Nevertheless, representing physical knowledge as physical priors in various domains, where symmetry and intuitive physical constraints prevail, can be more challenging than representing them as partial differential equations. Despite these challenges, the integration of PIML in AI has significant potential to enhance the performance and robustness of AI systems in various fields.

3 NEURAL SIMULATION

Using neural network based methods for simulating physical systems governed by PDEs/ODEs/SDEs (named *neural simulation*) is a fruitful and active research domain in physics-informed machine learning. In this section, we first list notations and background knowledge used in the paper. Neural simulation mainly consists of two parts, i.e. solving a single PDEs/ODEs using neural networks (named *neural solver*) and learning solution maps of parametric PDEs/ODEs (named *neural operator*). Then we will summarize problems, methods, theory and challenges for *neural solver* and *neural operator* in detail.

3.1 Challenges of Traditional ODEs/PDEs Solvers

Numerical methods are the main traditional solvers for ODEs/PDEs. These methods convert *continuous* differential equations (original ODEs/PDEs or their equivalents) into *discrete* systems of linear equations. Then, the equations are solved on (regular or irregular) meshes. For ODEs, the finite difference methods (FDM) [47] are the most important ones, of which the Runge–Kutta method [48] is most representative. The FDM replaces the derivatives in the equations with numerical differences which are evaluated on meshes. For PDEs, in addition to FDM (usually only applicable to geometrically regular PDEs), the finite volume methods (FVM) [49] and the finite element methods (FEM) [50] are also commonly used mesh-based methods. Such methods consider the integral form equivalent to the original PDEs, and follow the idea of numerical integration to transform the original equations into a system of linear equations. In addition, in recent years, meshless methods (such as spectral methods [51], which are based on the series expansion) have been developed and become powerful solvers for PDEs.

Traditional solvers for ODEs/PDEs are relatively mature, and are of high precision and good stability with complete theoretical foundations. However, we have to point out some of the bottlenecks that severely limit their application. First, traditional solvers suffer from the “curse of dimensionality”. Supposing that the number of grid nodes is n . A crude estimate of the time complexity is given by $\mathcal{O}(dn^r)$ for most traditional solvers [52], where $d \geq 1$ is the constant and r generally satisfies that $r \approx 3$. Computational cost increases dramatically when the dimensionality of the problem becomes very high, making the computation time of the problem unacceptable. What is more, for nonlinear and geometrically complex PDEs, d is far larger than 1 and the cost is even worse (for many practical geometrically complex problems, although the dimension is only 3 or 4, the computation time can take weeks or even months). Second, traditional solvers have difficulty in incorporating data from experiments and cannot handle situations where the governing equations are (partially) unknown (such as inverse design, described in Section 4). This is because the theoretical basis of the traditional solvers requires the PDEs to be known; otherwise, no meaningful solution will be obtained. Further, these methods are usually not learning-based and cannot incorporate data, which makes it difficult to generalize them to new scenarios.

Although traditional solvers are still the most widely used at present, they face serious challenges. This provides an opportunity for neural network-based methods. First, neural networks have the potential to resist the “curse of dimensionality”. In many application scenarios, the high-dimensional data can be well approximated by a much lower-dimensional manifold. *With the help of generalizability, we believe they have the potential to learn such a lower-dimensional mapping and handle high-dimensional problems efficiently; we take the success of neural networks in computer vision [53] as an example. Second, it is easy to incorporate data for neural networks, implicitly enabling knowledge extraction to enhance prediction results. A simple way is to include the supervised data losses into the loss function and directly train the neural network with some gradient descent algorithm like SGD and Adam [43].*

3.2 Neural Solver

3.2.1 Problem Formulation

This problem aims to solve a single physical system using (partially) known physical laws and available data. Assume the system is governed by the ODEs/PDEs in Eq. (3). We also might have a dataset containing state variables collected by sensors at some given points $\mathcal{D} = \{u(\mathbf{x}_i)\}_{i=1,\dots,n}$. Our goal is to solve and represent the state variables of the system $u(\mathbf{x})$. If we use neural networks with weights $w \in W$ to parameterize the state variables, then

$$\min_{w \in W} \|u_w(\mathbf{x}) - \tilde{u}(\mathbf{x})\|, \quad (7)$$

where $\tilde{u}(\mathbf{x})$ is the ground truth state variable.

The problem is to use neural networks to represent and solve the state of the physical system if the physical laws are completely known, to replace traditional methods like FEMs and FVMs. We call the methods for solving this problem "neural solvers." There are two potential advantages to use neural methods which might revolutionize numerical simulation in the future. First, the ability and flexibility of neural networks to integrate data and knowledge provide a scalable framework for handling problems with imperfect knowledge or limited data. Second, neural networks, as a novel function representation tool, are shown to be more effective for representing high-dimensional functions, which offers a promising direction for solving high-dimensional PDEs. However, there are still many drawbacks involving computational efficiency, accuracy, and convergence problems for existing neural solvers compared with numerical solvers such as FEM, which has been studied for decades. Thus, how to develop a scalable, efficient, and accurate neural solver is a fundamental challenge in the field of physics-informed machine learning.

In this section, we introduce methods based on neural networks that are able to incorporate (partially) known physical knowledge (PDEs) for simulating and solving a physical system. The most representative approach along these lines is Physics-Informed Neural Networks (PINNs) [30]. First, we introduce the basic ideas and framework of PINNs. Then, we present different variants of PINNs that improve PINNs from different viewpoints such as architectures, loss functions, speed and memory cost, etc. Finally, we propose several unresolved challenges in the field of neural solvers. [32]

3.2.2 Framework of Physics-Informed Neural Networks

PINNs are proposed by [30], which is the first work that incorporates physical knowledge (PDEs) into the architecture of neural networks to solve forward and inverse problems of PDEs. It is a flexible neural network method that can incorporate PDE constraints into the data-driven learning paradigm. Suppose there is a system that obeys the PDEs of Equation (3) and a dataset $\{u(\mathbf{x}_i)\}_{i=1,\dots,N}$. Then, it is possible to construct a neural network $u_w(\mathbf{x})$ and train it

with the following loss functions as

$$\begin{aligned} \mathcal{L} = & \frac{\lambda_r}{|\Omega|} \int_{\Omega} \|\mathcal{F}(u_w; \theta)(\mathbf{x})\|^2 d\mathbf{x} + \frac{\lambda_i}{|\Omega_0|} \int_{\Omega_0} \|\mathcal{I}(u_w; \theta)(\mathbf{x})\|^2 d\mathbf{x} \\ & + \frac{\lambda_b}{|\partial\Omega|} \int_{\partial\Omega} \|\mathcal{B}(u_w; \theta)(\mathbf{x})\|^2 d\mathbf{x} + \frac{\lambda_d}{N} \sum_{i=1}^N \|u_w(\mathbf{x}_i) - u(\mathbf{x}_i)\|^2, \end{aligned} \quad (8)$$

in which the term $\int_{\Omega} \|\mathcal{F}(u_w; \theta)(\mathbf{x})\|^2 d\mathbf{x}$ is the (PDE) residual loss that forces the PINNs u_w to satisfy the PDE constraints; $\int_{\Omega_0} \|\mathcal{I}(u_w; \theta)(\mathbf{x})\|^2 d\mathbf{x}$ and $\int_{\partial\Omega} \|\mathcal{B}(u_w; \theta)(\mathbf{x})\|^2 d\mathbf{x}$ are respectively the initial condition loss and boundary condition loss that force the PINNs to satisfy the initial condition and boundary condition; $\frac{1}{N} \sum_{i=1}^N \|u_w(\mathbf{x}_i) - u(\mathbf{x}_i)\|^2$ is the regular data loss in data-driven machine learning that tries to fit the dataset.

For simplicity of notation, we denote,

$$l_r \triangleq \|\mathcal{F}(u_w; \theta)(\mathbf{x})\|^2, \quad (9)$$

$$l_i \triangleq \|\mathcal{I}(u_w; \theta)(\mathbf{x})\|^2, \quad (10)$$

$$l_b \triangleq \|\mathcal{B}(u_w; \theta)(\mathbf{x})\|^2, \quad (11)$$

$$l_d \triangleq \|u_w(\mathbf{x}_i) - u(\mathbf{x}_i)\|^2, \quad (12)$$

and $\Omega_r \triangleq \Omega$, $\Omega_i \triangleq \Omega_0$, $\Omega_b \triangleq \partial\Omega$, $\Omega_d \triangleq \mathcal{D}_d$ so that these losses can be written in a unified form as

$$\mathcal{L}_k = \frac{\lambda_k}{|\Omega_k|} \int_{\Omega_k} l_k d\mathbf{x}, k \in \{r, i, b, d\}. \quad (13)$$

Because the losses of PINNs are flexible and scalable, we can simply omit the corresponding loss terms if there are no available data or initial/boundary constraints. The learning rate or the weights of these losses can be tuned by setting the hyperparameters λ_r , λ_i , λ_b and λ_d . In order to compute Equation (8), we need to evaluate several integral terms that involve the high-order derivatives computation of $u_w(\mathbf{x})$. PINNs uses the automatic differentiation of the computation graph to calculate these derivative terms. Then, it uses Monte-Carlo sampling to approximate the integral using a set of collocation points. We use $\mathcal{D}_r$, $\mathcal{D}_i$, $\mathcal{D}_b$ and $\mathcal{D}_d$ to represent the dataset of collocation points. We denote N_r , N_i , N_b and N_d as the amount of data. Then, the loss function can be approximated by

$$\begin{aligned} \mathcal{L} = & \frac{\lambda_r}{N_r} \sum_{i=1}^{N_r} \|\mathcal{F}(u_w; \theta)(\mathbf{x}_i)\|^2 + \frac{\lambda_i}{N_i} \sum_{i=1}^{N_i} \|\mathcal{I}(u_w; \theta)(\mathbf{x}_i)\|^2 \\ & + \frac{\lambda_b}{N_b} \sum_{i=1}^{N_b} \|\mathcal{B}(u_w; \theta)(\mathbf{x}_i)\|^2 + \frac{\lambda_d}{N_d} \sum_{i=1}^N \|u_w(\mathbf{x}_i) - u(\mathbf{x}_i)\|^2. \end{aligned} \quad (14)$$

Because of the use of automatic differentiation, Equation (14) is tractable and can be efficiently trained using first-order methods like SGD and second-order optimizers like L-BFGS.

3.2.3 PINN Variants

Although PINNs is a concise and flexible framework for solving forward and inverse problems of PDEs, there are many limitations and much room for improvement. Roughly speaking, all variants of PINNs focus on developing better optimization targets and neural architectures to improve the performance of PINNs. Here, we briefly

Neural Solver	Method	Description	Representatives
	Loss Reweighting	Grad Norm	GradientPathologiesPINNs [54]
		NTK Reweighting	PINNsNTK [55]
	Novel Optimization Targets	Variance Reweighting	Inverse-Dirichlet PINNs [56]
		Numerical Differentiation	DGM [57], CAN-PINN [58], cvPINNs [59]
		Variational Formulation	vPINN [60], hp-PINN [61], VarNet [62], WAN [63]
	Novel Architectures	Regularization	gPINNs [64], Sobolev Training [65]
		Adaptive Activation	LAAF-PINNs [66], [67], SReLU [68]
Feature Preprocessing		Fourier Embedding [69], Prior Dictionary Embedding [70]	
Boundary Encoding		TFC-based [71], CENN [72], PFNN [73], HCNet [74]	
Other Learning Paradigms	Sequential Architecture	PhyCRNet [75], PhyLSTM [76] AR-DenseED [77], HNN [78], HGN [79]	
	Convolutional Architecture	PhyGeoNet [80], PhyCRNet [75], PPNN [81]	
	Domain Decomposition	XPINNs [82], cPINNs [83], FBPINNs [84], Shukla et al. [85]	
Other Learning Paradigms	Transfer Learning	Desai et al. [86], MF-PIDNN [87]	
	Meta-Learning	Psaros et al. [88], NRPINNs [89]	

TABLE 2: An overview of variants of PINNs. Variants of PINNs include loss reweighting, novel optimization targets, novel architectures and other techniques such as meta-learning.

summarize the limitations that are addressed by the current variants of PINNs.

- Different loss terms in PINNs might have very different convergence speeds; more seriously, these losses might conflict with each other. To resolve this problem, many variants of PINNs have proposed different learning rate annealing methods from different perspectives. Some studies have borrowed ideas from traditional multi-task learning [90]. There are also studies that invent new reweighting schemes, inspired by theoretical analysis or empirical discoveries of PINNs [54], [55].
- PINNs directly penalizes a simple weighted average the PDE residual losses and initial/boundary condition losses, which might be sub-optimal [60], [64] for optimization and training on complex PDEs. Some work has attempted to adopt different loss functions for optimization that have better convergence and generalization ability [58], [60], [63], [91]. Other work has proposed adding more regularization terms for training PINNs [64], [65]. There is another line of papers that combines the variational formulation with residual loss of PINNs [60], [62], [91].
- Many physical systems exhibit extremely complicated multi-scale and chaotic behaviors, such as shock waves, phase transition, and turbulence. For these complex phenomena, it is difficult or inefficient to represent the system using a single MLP architecture. To resolve this challenge, many studies have proposed specific neural architectures for solving some PDEs. Some work [75], [76] has proposed incorporating LSTMs/RNNs, which are more suitable for processing sequential data into PINNs, to solve time-dependent problems involved in reducing errors accumulated over a long period of time. Other work [75], [80], [81] has proposed mesh-based representation and has used the architecture of CNNs. To further deal with the complexity brought about by complex geometric shapes in many practical applications, some work has designed neural networks using hard constraints for encoding initial/boundary conditions. Domain decomposition [84], [92] and feature preprocessing techniques [69] are proposed to handle multi-scale and large-scale problems. An-

other line of work has proposed to utilize other learning paradigms such as transfer learning [86], [87] and meta-learning [88], [89] to improve the performance of PINNs.

In summary, we use Table (2) to give the big picture of these variants of PINNs.

3.2.4 Loss Re-Weighting and Data Re-Sampling

A physical system dominated by PDEs usually simultaneously satisfies multiple constraints, such as PDEs and initial and boundary conditions. If we directly optimize it by adding these losses together, as shown in Equation (8), there arises a problem. The scale and convergence speed of different losses might be completely different, so that the optimization process might be dominated by some losses, which might converge slowly or converge to wrong solutions [90]. Existing methods for resolving this problem can be categorized into two classes. One is to re-weight different losses to balance the training process and accelerate the convergence speed. The other is to re-sample data (collocation points) to boost the optimization.

Loss re-weighting. Many different studies have proposed loss re-weighting or adaptive learning rate annealing methods by analyzing the training dynamics of PINNs from different perspectives or using different assumptions. [54] is the most famous work that shows that the gradients when training PINNs might be pathological, i.e., the loss of PDE residual is much larger than the boundary condition loss for high frequency functions. The training process is then dominated by the PDE loss, making it difficult to converge to a solution that satisfies boundary conditions. This study also introduced a simple method to mitigate the loss imbalance by re-weighting the learning rates. Let $\mathcal{L}_r(w)$ and $\mathcal{L}_i(w)$ respectively be the loss of PDE residual and other loss terms, i.e., initial/boundary conditions. It computes the update $\hat{\lambda}_i$ using the following equations at the n -th iteration as

$$\hat{\lambda}_i = \frac{\max\{\nabla_w \mathcal{L}_r(w_n)\}}{|\nabla_w \mathcal{L}_i(w_n)|}. \quad (15)$$

Then, the learning rate λ_i is updated by

$$\lambda_i \leftarrow (1 - \alpha)\lambda_i + \alpha\hat{\lambda}_i, \quad (16)$$

where α is a momentum hyperparameter controlling the update of the learning rates. Further, [55] rigorously ana-

lyzes the training of PINNs on a Poisson equation using the theory of Neural Tangent Kernel [93]. It proves that PINNs has a spectral bias, in that it prefers learning low-frequency functions, and therefore high frequency components are hard to converge. Based on this observation, it designs a learning rate annealing method based on NTKs.

For PDEs with Dirichlet conditions, they sample a dataset of collocation points from Ω and $\partial\Omega$, i.e. $\{\mathbf{x}_i\} \subset \Omega$ and $\{\mathbf{x}_i^b\} \subset \partial\Omega$. The neural tangent kernel of PINNs is defined by

$$\mathbf{K} = \begin{pmatrix} \mathbf{K}_{bb} & \mathbf{K}_{br} \\ \mathbf{K}_{br} & \mathbf{K}_{rr} \end{pmatrix}, \quad (17)$$

where $\mathbf{K}_{uu}$, $\mathbf{K}_{ur}$ and $\mathbf{K}_{rr}$ are submatrices of the NTK defined by,

$$(\mathbf{K}_{bb})_{i,j} = \left\langle \frac{du_w(\mathbf{x}_i^b)}{dw}, \frac{du_w(\mathbf{x}_j^b)}{dw} \right\rangle, \quad (18)$$

$$(\mathbf{K}_{br})_{i,j} = \left\langle \frac{du_w(\mathbf{x}_i^b)}{dw}, \frac{d\mathcal{F}(u_w)(\mathbf{x}_j)}{dw} \right\rangle, \quad (19)$$

$$(\mathbf{K}_{rr})_{i,j} = \left\langle \frac{d\mathcal{F}(u_w)(\mathbf{x}_i)}{dw}, \frac{d\mathcal{F}(u_w)(\mathbf{x}_j)}{dw} \right\rangle. \quad (20)$$

The convergence speed of PINNs is decided by the eigenvalues of $\mathbf{K}$. To balance the optimization of PDE residual and boundary condition losses, we use the trace of the neural tangent kernel to tune the learning rates λ_b and λ_r for $\mathcal{L}_b$ and $\mathcal{L}_r$,

$$\lambda_b = \frac{\text{Tr}(\mathbf{K})}{\text{Tr}(\mathbf{K}_{bb})}, \quad (21)$$

$$\lambda_r = \frac{\text{Tr}(\mathbf{K})}{\text{Tr}(\mathbf{K}_{rr})}. \quad (22)$$

This learning rate annealing scheme is shown to be efficient solving systems containing multiple frequencies like wave equations.

Another study, [56], proposed to use gradient variance to balance the training of PINNs,

$$\hat{\lambda}_i = \frac{\max_k \{\text{Var}[\nabla_w \mathcal{L}_k(w)]\}}{\text{Var}[\nabla_w \mathcal{L}_i(w)]}. \quad (23)$$

It also uses the momentum update with parameter α ,

$$\lambda_i \leftarrow (1 - \alpha)\lambda_i + \alpha\hat{\lambda}_i. \quad (24)$$

This approach is called Inverse-Dirichlet Weighting. Experiments show that it alleviates gradients vanishing and catastrophic forgetting in multi-scale modeling.

[94], [95] propose to use the characteristic quantities M_i defined as follows,

$$M_i[u] \approx \frac{\|u_i\|_2^2}{|\Omega|} \quad (25)$$

Then, the learning rate for each loss is determined by

$$\lambda_i = \left(\frac{\sum_k M_k[u]}{M_i[u]} \right)^{-1}. \quad (26)$$

The idea of this method is to approximate the optimal loss weighting under the assumption that error could be uniformly bounded. [94] also uses a soft penalty method to incorporate learning from data of different levels of fidelity.

To ensure causality, [96] propose to set the learning rates for training PINNs on time-dependent problems to decay with time. Let $\mathcal{L}(t_k, w)$ be the losses at time t_k . Then, the total loss is,

$$\mathcal{L}(w) = \sum_k \lambda_k \mathcal{L}(t_k, w) \quad (27)$$

And the weights λ_k are

$$\lambda_i = \exp \left(-\varepsilon \sum_k \mathcal{L}(t_k, w) \right). \quad (28)$$

Besides heuristic methods, [88] attempts to learn optimal weights from data using meta-learning. [97] investigate the properties of the Pareto front between data losses and physical regularization. [98], [99] model the tuning of loss weights as a problem of finding saddle points in a min-max formulation. [98] solves the min-max problem using the Dual-Dimer method. [99] shows connections between the min-max problem and a PDE Constrained Optimization (PDECO) using a penalty method. Though there are many methods for tuning weights of loss functions, there is no fair and comprehensive benchmark to compare these methods.

Data Re-Sampling. Another set of methods to handle the imbalance learning process is to re-sample collocation points adaptively. One simple strategy is to sample quasi-random points or a low-discrepancy sequence of points from the geometric domain [25]. This sampling strategy is model-agnostic and only depends on the geometric shape. Representative sampling methods include Sobel sequence [100], Latin hypercube sampling [101], Halton sequence [102], Hammersley sampling [103], Faure sampling [104] and so on [105], [106].

Besides these model-agnostic sampling strategies, another intuitive idea is to sample collocation points from areas with higher error. Thus, we could put more effort into optimizing losses in these areas. Some approaches have designed adaptive sampling strategies based on this idea. Along these lines, PDE residual loss of a vanilla PINNs could viewed as an expectation over a probability distribution as

$$\mathcal{L}_r = \mathbb{E}_{\mathbf{x} \sim p} [l_r] = \mathbb{E}_{\mathbf{x} \sim p} [\|\mathcal{F}(u)(\mathbf{x})\|^2], \quad (29)$$

and the initial/boundary losses could be described in the same manner. Here, p is a uniform distribution defined on Ω . In [107], the author proposed to sample collocation points with importance sampling,

$$\mathcal{L}_r = \mathbb{E}_{\mathbf{x} \sim q} \left[\frac{p(\mathbf{x})}{q(\mathbf{x})} \|\mathcal{F}(u)(\mathbf{x})\|^2 \right].$$

Choosing a better probability distribution might accelerate training of PINNs, because it uses the following distribution to sample a mini-batch of M collocation points from a dataset of N points uniformly selected ($M < N$),

$$q(\mathbf{x}_i) = \frac{\|\nabla_w l_r(w, \mathbf{x}_i)\|}{\sum_j \|\nabla_w l_r(w, \mathbf{x}_j)\|} \approx \frac{l_r(w, \mathbf{x}_i)}{\sum_j l_r(w, \mathbf{x}_j)}, 1 \leq i \leq N. \quad (30)$$

However, this requires the evaluation of residuals in a large dataset, which is inefficient. Therefore, the study proposes using a piece-wise constant approximation to the loss function to accelerate sampling from the distribution. Note that

Equation (30) approximates the norm of gradients, with the loss itself similar to [108].

Further, [109] view the losses as a probability distribution and use a generative model to sample from this distribution. Thus, areas with higher residuals contain more collocation points for optimization. Specifically, the distribution of residuals is

$$q_r(\mathbf{x}) = \frac{1}{Z} l_r(\mathbf{x}) = \frac{\|\mathcal{F}(u)(\mathbf{x})\|^2}{\int_{\Omega} \|\mathcal{F}(u)(\mathbf{x})\|^2 d\mathbf{x}}. \quad (31)$$

Sampling from this distribution is not trivial and the authors propose to use a flow-based generative model [110] to sample from the distribution. Similarly, [111] uses self-paced learning that gradually modifies the sampling strategy from uniform sampling to residual-based sampling.

3.2.5 Novel Optimization Objectives

In this section, we describe variants of PINNs that adopt different optimization objectives. Although various loss re-weighting and data re-sampling methods accelerate convergence of PINNs for some problems, these methods only serve as a trick, since they only allocate different weights for losses but do not modify the losses themselves. There is another strand of research that has proposed to train PINNs with novel objective functions rather than weighted summation of residuals. Some studies combine numerical differentiation into PINNs' training process. Some propose to adopt or incorporate variational (or weak) formulation inspired by Finite Element Methods (FEM) instead of PDE residuals. Other approaches propose adding more regularization terms to accelerate training of PINNs.

Incorporating Numerical Differentiation. Vanilla PINNs use automatic differentiation to calculate higher-order derivatives of a neural network with respect to input variables (spatial and temporal coordinates). This method is accurate because we can analytically calculate the derivatives with respect to each layer using backpropagation. DGM [57] points out that computing higher-order derivatives is computationally expensive for high-dimensional problems such as high-dimensional Hamilton-Jacobian-Bellman (HJB) equations [112], which are widely used in control theory and reinforcement learning. This approach proposes to use Monte-Carlo methods to approximate second-order derivatives. Suppose the sum of the second-order derivatives in $\mathcal{L}_r$ is $\frac{1}{2} \sum_{i,j} \rho_{i,j} \sigma_i(x) \sigma_j(x) \frac{\partial^2 f}{\partial x_i \partial x_j}(t, x; w)$. Assume $(\rho_{i,j})_{i,j=1}^d$ is a positive definite matrix, and define $\sigma(x) = (\sigma_1(x), \dots, \sigma_d(x))$. There are many PDEs corresponding to this case, such as the HJB equation and Fokker-Planck equation. We have the following equation,

$$\sum_{i,j} \rho_{i,j} \sigma_i(x) \sigma_j(x) \frac{\partial^2 f}{\partial x_i \partial x_j}(t, x; \theta) = \lim_{\Delta \rightarrow 0^+} \mathbb{E} \left[\sum_i \frac{\sigma_i(x) W_{\Delta}^i}{\Delta} \left(\frac{\partial f}{\partial x_i}(t, x + \sigma(x) W_{\Delta}; w) - \frac{\partial f}{\partial x_i}(t, x; w) \right) \right], \quad (32)$$

where $W_t \in \mathbb{R}^d$ is a Brownian motion and we choose $\Delta > 0$ as the step size. This reduces the computational complexity from $O(d^2 N)$ to $O(dN)$.

CAN-PINN [58] shows that PINNs using automatic differentiation might need a large number of collocation points for training. CAN-PINN uses carefully designed numerical differentiation schemes to replace some terms in automatic differentiation. Specifically, upwind schemes and central schemes [113] are adopted in convection terms, replacing automatic differentiation.

Control volume PINNs (cvPINNs) [59] borrow the idea of traditional finite volume methods to solve hyperbolic PDEs. This approach partitions the domain into several cells and the PDE losses of hyperbolic conservation laws are transformed into an integral over these cells. Non-local PINNs [114] uses a Peridynamic Differential Operator, which is a numerical method incorporating long-range interactions, and removes spatial derivatives in the governing equations.

Variational formulation. In traditional FEM solvers, variational (or weak) formulation is an essential tool that reduces the smoothness requirements for choosing basis functions. In variational formulation, the PDEs are multiplied by a set of test functions and transformed into an equivalent form using integrals by parts, as introduced before. The derivative order of this equivalent form is lower than the original PDEs. Although PINNs with smooth activation functions are infinitely differentiable, many studies have shown that there might be potential benefits from adopting the variational (or weak) formulation. In the theory of FEM analysis, solving a PDEs in variational form is equivalent to minimizing an energy function. While this functional form is different from the optimization target of vanilla PINNs, the optimal solution is exactly the same.

For example, consider a system satisfying the following Poisson's equation with natural boundary conditions over the boundary:

$$\Delta u = f(\mathbf{x}), \mathbf{x} \in \Omega, \quad (33)$$

$$\frac{\partial u}{\partial n} = 0, \mathbf{x} \in \partial\Omega. \quad (34)$$

If we use PINNs to solve this problem, we use a neural network u_w to represent the solution and minimize the following objective:

$$\mathcal{L}(w) = \frac{\lambda_r}{|\Omega|} \int_{\Omega} \|\Delta u_w - f(\mathbf{x})\|^2 d\mathbf{x} + \frac{\lambda_b}{|\partial\Omega|} \int_{\partial\Omega} \left\| \frac{\partial u_w}{\partial n} \right\|^2 d\mathbf{x}. \quad (35)$$

The Deep Ritz Method (DRM) [91] proposes incorporating the variational formulation into training the neural networks. Specifically, the objective function using the variational formulation for this problem is

$$\mathcal{J}(w) = \int_{\Omega} \left(\frac{1}{2} |\nabla u_w(\mathbf{x})|^2 - f(\mathbf{x}) u_w(\mathbf{x}) \right) d\mathbf{x}. \quad (36)$$

Note that this objective function only involves the first-order derivatives of $u(\mathbf{x})$; thus, we do not need to calculate high-order derivatives. Additionally, the variational formulation naturally absorbs the natural boundary conditions, so we do not need to add more penalty terms. This objective function could also be minimized using gradient descent on weights w similar to PINNs. If the system satisfies Dirichlet boundary conditions, i.e., if

$$u(\mathbf{x}) = g(\mathbf{x}), \mathbf{x} \in \partial\Omega, \quad (37)$$

we would still need to add a constraint to enforce this type of boundary conditions:

$$\mathcal{J}(w) = \int_{\Omega} \left(\frac{1}{2} |\nabla u_w(\mathbf{x})|^2 - f(\mathbf{x})u_w(\mathbf{x}) \right) d\mathbf{x} + \lambda_b \int_{\partial\Omega} (u(\mathbf{x}) - g(\mathbf{x}))^2 d\mathbf{x}. \quad (38)$$

In fact, the DRM method was proposed even before PINNs. However, DRM is only available for self-adjoint differential operators, thus limiting its applications. What's more, [115] shows that the fast rate generalization bound of DRM is suboptimal on elliptic PDEs.

Further, in VPINNs [60], the authors propose to develop a Petrov-Galerkin formulation [116] for training PINNs on more general PDEs. VPINNs consider a broader type of PDEs,

$$\mathcal{F}(u)(\mathbf{x}) = 0, \mathbf{x} \in \Omega, \quad (39)$$

$$u(\mathbf{x}) = g(\mathbf{x}), \mathbf{x} \in \partial\Omega. \quad (40)$$

It first chooses a (finite) set of test functions $v(\mathbf{x}) \in V_K$, N_b points from the boundary, and constructs the following loss functions,

$$\mathcal{J}(w) = \frac{1}{K} \sum_{k=1}^K |\langle \mathcal{F}(u_w), v \rangle_{\Omega}|^2 + \lambda_b \frac{1}{N_b} \sum_{i=1}^{N_b} |u_w(\mathbf{x}_i) - g(\mathbf{x}_i)|^2. \quad (41)$$

The interior product denotes an integral over the geometric domain,

$$\langle \mathcal{F}(u_w), v \rangle_{\Omega} = \int_{\Omega} \langle \mathcal{F}(u_w)(\mathbf{x}), v(\mathbf{x}) \rangle d\mathbf{x}. \quad (42)$$

The key of VPINNs is to properly choose test functions according to different problems. In applications, sine and polynomial functions are good candidates for test functions. As a special case, if we use a delta function $v(\mathbf{x}, \mathbf{x}_0) = \delta(\mathbf{x} - \mathbf{x}_0)$ as the test function and $\mathbf{x}_0$ as the collocation points, VPINNs are the same with vanilla PINNs.

In subsequent work, VarNet [62] has proposed to take piecewise linear functions as test functions, so that it is more parallelizable and easier to compute the inner products between test functions and neural networks. hp-VPINNs [61] proposes to partition the domain into several subdomains and then solve PDEs in these subdomains using variational formulation. The partitioning technique is also called domain decomposition, which will be introduced in detail in subsection 3.2.6. Similar work, such as CENN [72] and D3M [117], also adopt domain decomposition-based variational formulation as loss functions, but they employ other tricks like multi-scale features [69] or multi-scale neural networks. PFNN [73] constructs two neural networks and use one of them to enforce essential boundary conditions. Then they use the second neural network to learn from the variational formulation, similar to DGM. By encoding the boundary condition first, it avoids the penalty terms and does not need to tune the weights λ_b for them. The boundary encoding technique will be introduced in subsection 3.2.6.

The selection of test functions is crucial for variational formulation-based PINNs. The studies mentioned above chose test functions from a specific function class such as sine or polynomial functions, using priors about the problem. Besides these heuristically chosen test functions, there

is another work called Weak Adversarial Networks (WAN) [63] that models the training using variational formulation as a min-max problem. Specifically, if the PDEs are strictly satisfied, then for any test function $v \in V$,

$$\langle \mathcal{F}(u), v \rangle_{\Omega} = 0. \quad (43)$$

Instead of selecting many test functions from a predefined set, WAN chooses the worst case test function to measure the mismatch of current solution u_w . We define the norm of a test function v as $\|v\|_{\Omega} = \sqrt{\langle v, v \rangle_{\Omega}}$. For problems with natural boundary conditions, we define an operator norm of $\mathcal{F}$ as follows,

$$\|\mathcal{F}(u)\|_{\text{op}} := \max \left\{ \frac{\langle \mathcal{F}(u), v \rangle_{\Omega}}{\|v\|_{\Omega}} : v \in H_0^1, v \neq 0 \right\}. \quad (44)$$

If u is the solution of variational formulation in Equation (43), the operator norm should be 0. From this perspective, minimizing the operator norm equals solving the variational formulation of PDEs. Then training PINNs is to minimize the following objective,

$$\min_{u \in H^1} \|\mathcal{F}(u)\|_{\text{op}}^2. \quad (45)$$

In fact, this is a min-max problem like Generative Adversarial Network [118]. If we represent solutions and test functions with neural networks parameterized with w and θ , we have,

$$\min_w \max_{\theta} \frac{|\langle \mathcal{F}(u_w), v_{\theta} \rangle_{\Omega}|^2}{\|v_{\theta}\|_{\Omega}^2}. \quad (46)$$

This is exactly a loss function for optimizing a GAN and we can use existing techniques for training GANs to optimize it. For problems with other boundary conditions like Dirichlet/Robin boundary conditions, regularization terms including boundary condition losses should be included when defining the operator norm. [63], [119] discuss training details and other applications of Weak Adversarial Networks.

Variational (or weak) formulation of PDEs is widely used in Finite Element Method. Such formulation is also shown to be effective for training PINNs in many situations. Many studies have paid attention to selecting appropriate test functions and loss formulations, as introduced before. However, variational form is not the only equivalent form of PDEs. There are other works adopting different formulations of PDEs. For example, BINet [120] combines boundary integral equation methods with neural networks for solving PDEs. In summary, combining other equivalent formulations of PDEs inspired by traditional numerical PDE solvers and PINNs' training is an important topic. Empirical or theoretical analysis on which formulation benefits the training of PINNs has still been largely unexplored.

Regularization terms.

Regularization is an important and simple trick that can boost the training or the generalization ability of machine learning models in many practical applications. In computer vision and machine learning, many regularization terms are proposed according to their effect on the neural networks. For example, L_2 regularization [121] can mitigate the overfitting of the model. L_1 regularization [122] is used to extract sparse features. There are other regularization approaches such as label smoothing [123] and knowledge

distillation [124]. These methods are called explicit regularization because they add new loss terms that directly modify the gradients computation. Despite existing regularization methods that might also be useful to PINNs, there are novel regularization terms specifically designed for PINNs.

A representative example of these new regularization methods is called gradient-enhanced training [64], or Soblev training [56], [65]. The motivation of gradient-enhanced training is to incorporate higher order derivatives for PDEs as regularization terms. Since PDEs are a set of identical relations, we can calculate any order of derivatives of it. Denote $\mathcal{D}_i^k = \frac{\partial^k}{\partial x_i^k}$ to be the operator of k -th order derivatives for variable x_i . Then for all k, i , we have

$$\mathcal{D}_i^k \mathcal{F}(u)(\mathbf{x}) = \frac{\partial^k}{\partial x_i^k} \mathcal{F}(u)(\mathbf{x}) = 0. \quad (47)$$

Gradient enhanced training (or Soblev training) adds regularization terms based on the derivatives of PDE residuals. Suppose we choose a set of indexes $K = \{k_t\}_{t=1\dots m}$ and $I = \{i_t\}_{t=1\dots m}$, where $k_t, i_t \in \mathbb{N}_+$ and $1 \leq i_t \leq d$. Then, the gradient enhanced regularization is

$$\mathcal{L}_{\text{reg}} = \sum_{k, i \in K, I} \sum_{\mathbf{x}_j \in \mathcal{D}_r} \lambda_{k, i} \|\mathcal{D}_i^k \mathcal{F}(u)(\mathbf{x}_j)\|^2. \quad (48)$$

Here, $\mathcal{D}_r = \{\mathbf{x}_j\}$ are the collocation points to evaluate these regularization terms based on higher order derivatives of PDE residuals. Experiments show that in some situations these regularization terms enable the PINNs to train more quickly and accurately. However, choosing the index sets K and I as the heuristic decision for gradient enhanced PINNs.

3.2.6 Novel Neural Architectures

In this subsection, we introduce variants of PINNs with novel neural architectures for specific problems. Developing proper architectures of neural networks with strong generalization ability is a crucial challenge in machine learning. Although the multi-layer perceptron (MLPs) is a general architecture with the capacity to fit any function, its ability to generalize to many domain-specific problems is sub-optimal since it lacks appropriate inductive biases [125]. To incorporate the priors about the data into the model structure, different neural architectures are proposed. For example, for image or grid data, convolutional neural networks (CNNs) [126] are proposed because they can extract information from local structures of these types of data. RNN [127], LSTM [128] and transformers [129], which have strong ability to model temporal dependency, are proposed to recognize and generate sequential data such as text, audio and time series. Graph neural networks [33] are proposed to extract local node features and global graph features on irregular graph data.

In vanilla PINNs, multi-layer perceptron (MLPs) has been adopted for solving general PDEs, and has achieved remarkable success. However, the architecture of MLP has many drawbacks when solving some domain-specific and complex PDE systems. Therefore, many variants of PINNs have been developed to improve on architectures for the purpose of adapting to these domain-specific problems. These studies can roughly be divided into several classes. First, the selection of activation functions is noteworthy and

many studies have proposed adaptive activation functions for PINNs to deal with the multi-scale structure of physical systems. Second, some work has investigated to embed input spatial-temporal coordinates. These studies propose different feature preprocessing layers, such as Fourier features, to enable learning of different frequencies. Third, architectures like CNNs and LSTMs can be used in PINNs for specific problems. For example, PINNs using convolutional architecture is able to output the whole solution field in one pass rather than the value on a single point. In addition, sequential neural architectures like LSTM can be used to accelerate solving time-dependent PDEs. Fourth, hard boundary constraints can be enforced for some problems with the help of an additional neural network that trains only on boundary condition losses. These methods separate the training of PDEs and initial/boundary conditions in order to avoid the loss imbalance issue. Finally, domain decomposition is proposed to solve large-scale problems. Its purpose is to partition the geometric domain into several subdomains and train a PINNs on each domain to reduce the training difficulty.

Activation Functions.

The nonlinear activation functions play an important role in the expressive power of neural networks. For deep neural networks, ReLU [130], Sigmoid, Tanh, and Sine [131], [132] are the most commonly used. We often need to calculate higher-order derivatives. Therefore, only smooth activation function can be used in PINNs. The Swish activation function is used as a smoothed approximation of ReLU. It is defined as $\text{Swish}(x) = x \cdot \text{Sigmoid}(\beta x)$, where β is a hyperparameter. Despite using existing activation functions, some studies [66], [67] have proposed adaptive activation functions for PINNs to deal with multi-scale physical systems and the gradient vanishing problem. In [66], the authors propose to use $\sigma(na \cdot x)$, where $\sigma(\cdot)$ is an activation function, a is a learnable weight and n is a positive integer hyperparameter to scale the inputs. [67] further extends the adaptive activation in [66] to two types: layer-wise adaptive activation and neuron-wise adaptive activation. The layer-wise adaptive activation learns one a for each layer. The neuron-wise adaptive activation learns a for each output neuron. It also proposes a special regularization term called the slope recovery term to increase the slope of activation functions. Suppose $\{a_k : 1 \leq k \leq K\}$ is the set of parameters of adaptive activation functions. The slope recovery regularization term is

$$\mathcal{J} = \lambda_{\text{reg}} \frac{1}{\frac{1}{K} \sum_{k=1}^K \exp(a_k)}, \quad (49)$$

where λ_{reg} is a hyperparameter.

[68] proposes two novel activation functions with compact support, defined as

$$\text{SReLU}(x) = x_+(1 - x)_+, \quad (50)$$

$$\phi(x) = x_+^2 - 3(x - 1)_+^2 + 3(x - 2)_+^2 - (x - 3)_+^2, \quad (51)$$

where $x_+ = \max\{x, 0\} = \text{ReLU}(x)$. These two activation functions look like the RBF kernel but they are compactly supported.

Feature Preprocessing (Embedding). Feature preprocessing is a basic tool before we feed data into neural networks. Data whitening is an essential preprocessing method widely used in image preprocessing. It normalizes the data to zero means and unit variance. A good feature preprocessing or embedding method might accelerate the training of neural networks. For many practical multi-scale physical systems, we face the challenge that the scale and magnitude are completely different for different parts of the system. For example, for a wave propagation problem in two mediums, the wave length is about 10^3 times shorter in solid material than in air. It will not make any difference if we directly apply simple normalization to the input coordinates. For these problems with a sharp variation in space or time, the solution usually contains multiple distinct frequencies. [133] provides a feature embedding method called Fourier features, which was first used in scene representation [134]. Suppose $\mathbf{x} \in \mathbb{R}^d$ is the input coordinates, and $\mathbf{b}_i \in \mathbb{R}^d$ are scale parameters. Then the Fourier feature embeds the input coordinates using the following equation:

$$\gamma(\mathbf{x}) = (\sin(2\pi\mathbf{b}_1^T \cdot \mathbf{x}), \cos(2\pi\mathbf{b}_1^T \cdot \mathbf{x}), \dots, \sin(2\pi\mathbf{b}_m^T \cdot \mathbf{x}), \cos(2\pi\mathbf{b}_m^T \cdot \mathbf{x})). \quad (52)$$

It embeds the low-dimensional coordinates into high dimensions. The selection of scale parameters $\mathbf{b}_i$ plays a crucial role in Fourier feature embedding. For instance, in NERF [134], it uses a geometric series and maps each spatial coordinate separately:

$$\gamma(x_i) = (\sin(2^0\pi x_i), \cos(2^0\pi x_i), \dots, \sin(2^{L-1}\pi x_i), \cos(2^{L-1}\pi x_i)). \quad (53)$$

We see that, for large L , $\sin(2^{L-1}\pi x_i)$ changed dramatically even if x_i varies only a little. This naturally has the effect of scaling the input. More detailed analysis [133] based on the theory of the Neural Tangent Kernel (NTK) shows that Fourier features make it easier for the neural networks to learn high-frequency functions, which mitigates spectral biases. [69] further extends this analysis to the training of PINNs. This approach proposes to sample the scale parameter $\mathbf{b}_i$ from a Gaussian distribution, i.e.,

$$\mathbf{b}_i \sim \mathcal{N}(0, \sigma^2), \quad (54)$$

where σ is a hyperparameter. It also uses two independent Fourier features networks to embed the spatial coordinates and temporal coordinates, respectively. [131], [132] propose to use sine as the activation functions for neural networks and the corresponding initialization scheme for weights $\mathbf{b}_i$. [68] proposes multi-scale feature embedding based on the SReLU and $\phi(\cdot)$ activation functions introduced in Equation 50 and 51. We simply use σ to denote one of them; it embeds the coordinates using the following equation:

$$\gamma(x_i) = \sigma(n x_i), n = 1, \dots, L. \quad (55)$$

This formulation can be viewed as both an adaptive activation function and multi-scale feature embedding. [70] generalizes the functions used for feature preprocessing as a prior dictionary. The dictionary includes trigonometric functions, locally supported functions or learnable functions. The prior dictionary is flexible; it is chosen based on prior knowledge about the problem.

We have introduced different activation functions and feature preprocessing layers in PINNs. In the next several subsections, we will introduce variants of PINNs that adopt different network architectures.

Multiple NNs and Boundary Encoding.

The vanilla PINNs inputs the spatial-temporal coordinates and outputs the state variable, which is usually a vector for high-dimensional PDEs. These high-dimensional problems are multi-task learning problems; therefore, vanilla PINNs can be viewed as parameter sharing for all tasks. This might lead to suboptimal performance due to the capacity limit of a single MLP. To achieve better accuracy, some studies [135], [136], [137] propose using multiple MLPs without sharing parameters to separately output each component of the state variable. As a simple approach to avoiding parameter sharing, multiple NNs are also used in decomposing the problem into several easier subproblems. First, multiple NNs are used to output intermediate variables and reduce the order of PDEs/ODEs. [138] proposes using variable substitution and outputs several intermediate variables using different branches. This method can reduce the order of the PDEs and provides many advantages in practice. Second, multiple NNs with postprocessing layers are used to encode boundary and initial conditions with hard constraints, which will be described in the next several paragraphs. Third, multiple NNs are used in domain decomposition, which will be presented in a later subsection, since it is an independent and comprehensive technique to improve performance of PINNs and is associated with a rich body of literature.

An important technique for many variants of PINNs is to adopt (multiple) NNs with post-processing layers for encoding boundary/initial conditions with hard constraints. In the previous subsection 3.2.4, we see that balancing losses between PDE residuals and boundary/initial conditions is critical for PINNs. In addition to using adaptive schemes for loss reweighting, another approach is to hard constraint the NNs to satisfy one of them and learn the other one. However, encoding PDEs with hard constraints is only feasible for several simple PDEs with a general solution. For example, for the following one-dimensional wave equation,

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0, \quad (56)$$

it has a general solution,

$$u = f(x - ct) + g(x + ct). \quad (57)$$

For this equation, we could encode PDE with hard constraints by using NNs to represent f_w and g_w . Then, we could train the NNs by fitting f_w and g_w on boundary/initial conditions. However, most PDEs do not have an analytical general solution. For this reason, most studies focus on encoding boundary/initial conditions with hard constraints rather than PDEs. These studies can be traced back more than two decades [139], [140]. For simple boundary conditions like $\Omega = [0, L]$ and $u(0) = u(L) = 0$, we can simply construct a hypothesis space satisfying the constraints and do not need to use multiple NNs with carefully designed architectures. Specifically, we can add

a simple post-processing layer [139], [140] after the neural networks u_w

$$u'_w = u_w \cdot x(L - x). \quad (58)$$

This simple method can be extended to Dirichlet boundary conditions on rectangle domains. If the boundary condition is periodic, we can construct a neural network [141] that outputs a vector $u_w = (u_1, \dots, u_n, v_1, \dots, v_n)$ and represent the solution on the basis of Fourier,

$$u_w = \sum_{k=1}^n \left(u_k(x) \sin \frac{2\pi kx}{L} + v_k(x) \cos \frac{2\pi kx}{L} \right). \quad (59)$$

These methods are further generalized and unified in the Theory of Functional Connections (TFC) [71], [142].

For a simple geometric problem, it is possible to design handcrafted post-processing layers for hard constraining boundary/initial conditions. However, they fail to handle problems on a general, irregular domain. Encoding general boundary conditions, including boundary conditions of arbitrary forms, on irregular domains is still an unresolved problem, despite successful attempts [143], [144] for Dirichlet boundary conditions. It is worth noting that recent work [74] proposes a unified framework for encoding the three most widely used boundary conditions, i.e., Dirichlet, Neumann, and Robin boundary conditions, on geometrically complex domains, which significantly improves the applicability of hard-constraint methods.

Specifically, as a abstract framework of hard-constraint methods, we decompose the problem into several parts, i.e., a PDE losses part and a boundary/initial condition part. Then, multiple NNs are used to solve them separately. The resulting solution is a combination of them. Suppose the system satisfies the Dirichlet boundary conditions:

$$\mathcal{F}(u)(x) = 0, x \in \Omega, \quad (60)$$

$$u(x) = g(x), x \in \partial\Omega. \quad (61)$$

We decompose the solution of the problem into two parts if such decomposition exists:

$$u(x) = v(x) + D(x)y(x). \quad (62)$$

Here we first train a neural network $v_w(x)$ to satisfy boundary conditions only:

$$\mathcal{L}_b = \int_{\partial\Omega} |v(x) - g(x)|^2 dx. \quad (63)$$

Then, the function $D(x)$ is the key to separating the training of PDE residuals and boundary/initial conditions. It should be smooth and vanishes on the boundary, i.e.,

$$D(x) = 0, x \in \partial\Omega. \quad (64)$$

However, it is usually difficult to choose a $D(x)$ that is smooth everywhere for a general domain Ω . Since we train a neural network only on a set of collocation points, we fit a smooth $D_{w'}(x)$ with a neural network to approximate the following distance function [145]:

$$d(x) = \min_{x_b \in \partial\Omega} \|x - x_b\|_2. \quad (65)$$

We can also train $D_{w'}(x)$ on collocation points. Then, the final step is to train $y(x)$ in domain Ω with only PDE residuals because the boundary condition is naturally satisfied

by using this decomposition. This method can be extended to initial conditions as well [143].

Many studies that followed the original work have proposed better variants of the distance function $D(x)$. For instance, CENN [72] constructs an (approximation of) $D(x)$ through a linear combination of a radical basis function,

$$D(x) = \sum_{i=1}^n w_i \phi(-\|x - x_i\|), \quad (66)$$

where $\phi(x) = \exp(-\gamma r^2)$ is a radical basis function with hyperparameter γ . $\{x_i : x_i \in \Omega\}$ is a dataset of collocation points. $y_i = \min_{x \in \partial\Omega} \|x_i - x\|$ is the distance of these collocation points to the boundary, which can be precomputed. We can solve w_i using the following linear equation:

$$\begin{pmatrix} \phi(\|x_1 - x_1\|) & \dots & \phi(\|x_1 - x_n\|) \\ \vdots & \ddots & \vdots \\ \phi(\|x_n - x_1\|) & \dots & \phi(\|x_n - x_n\|) \end{pmatrix} \cdot \begin{pmatrix} w_1 \\ \vdots \\ w_n \end{pmatrix} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}. \quad (67)$$

The meaning of $D(x)$ is to interpolate a distance function with a dataset of collocation points with radical basis functions.

For PFNN, [73], proposes a novel distance function $D(x)$ for multiple complex boundaries. It divides the boundary $\partial\Omega$ into several segments $\{\gamma_i : 1 \leq i \leq K\}$ and constructs a $D(x)$ based on these segments. For a given γ_k and a non-neighbor segment γ_{k_0} , it defines a spline function l_k to satisfy the following property:

$$\begin{cases} l_k(x) = 0, & x \in \gamma_k \\ l_k(x) = 1, & x \in \gamma_{k_0} \\ 0 \leq l_k(x) \leq 1, & \text{otherwise} \end{cases} \quad (68)$$

It defines a type of indicator function that vanishes only on a certain segment of the boundary. Then, we define the overall $l(x)$,

$$l(x) = \prod_{k=1}^K 1 - (1 - l_k(x))^\mu, \quad (69)$$

where $\mu \geq 1$ is a hyperparameter. Finally, we define $D(x)$ as follows:

$$D(x) = \frac{l(x)}{\max_{x \in \Omega} l(x)}. \quad (70)$$

In practice, $l_k(x)$ is constructed by a combination of a radical basis function and a linear function; because this is complicated, we omit the details here. We see that the $D(x)$ is then smooth and vanishes on all boundary segments.

Sequential Neural Architecture. A large amount of work in machine learning is about specific network architectures to process sequential data such as text, audio, and time series. By now, there are many famous architectures for sequential data recognition, including Recurrent Neural Networks (RNN) [127], Long-Short Term Memory network (LSTM) [128], Gated Recurrent Unit (GRU) [146], Transformer [129] and so on. In the field of physics, many real physical systems are time-dependent; therefore, future states rely on the past states of systems. These systems can be naturally modeled as sequential data. Along this line, many studies [75], [76], [77] propose to combine these

neural architectures to train PINNs. A typical example is the following time-dependent PDEs:

$$\frac{\partial u}{\partial t} + F\left(u, \frac{\partial u}{\partial x_1}, \dots, \frac{\partial u}{\partial x_d}, \dots; \theta\right) = 0. \quad (71)$$

Vanilla PINNs builds a neural network that inputs $u(x_1, \dots, x_d, t)$ and updates the model using the PDE residual. If we adopt a sequential neural architecture to solve the problem, we first discretize $t \in [0, T]$ into several n time slots $\{t_i : t_i = i\Delta t, 1 \leq i \leq n\}$. Then, we use numerical differentiation to approximate the derivatives $\frac{\partial u}{\partial t}$. The loss is then constructed by:

$$\mathcal{L}_{\text{reg}} = \left\| \frac{u_{i+1} - u_i}{\Delta t} - F\left(u_i, \frac{\partial u_i}{\partial x_1}, \dots, \frac{\partial u_i}{\partial x_d}, \dots, \theta\right) \right\|^2. \quad (72)$$

Here, u_i is the output of the neural networks at time t_i . In many studies [76], [77], LSTMs are used to represent the solution u_i . We see that, by using sequential architecture, we can transform the problem into a set of time-independent PDEs.

As well as work using LSTMs to solve general time-dependent problems, another line of work proposes a sequential architecture combining numerical differentiation to solve a specific class of systems governed by Newton's laws. In physics, solving or identifying dynamic systems governed by Newton's laws (or Hamiltonian, Lagrangian equations) is a fundamental issue. It has a wide range of applications in physics, robotics, mechanical engineering and molecular dynamics. There is a great deal of work designing specific neural architectures that naturally obey Hamiltonian equations and Lagrangian equations [78], [79], [147], [148].

Hamiltonian equations are a class of basic and concise first-order equations to describe temporal evolution of physical systems. For Hamiltonian systems, states are $(\mathbf{q}, \mathbf{p})$, where $\mathbf{q}$ represents the coordinates and $\mathbf{p}$ represents the momentum of the system. Hamiltonian neural networks (HNN) [78], [149] represent the Hamiltonian with a neural network $\mathcal{H}_w(\mathbf{q}, \mathbf{p})$. The evolution of the system is determined by

$$\frac{d\mathbf{q}}{dt} \approx \frac{\mathbf{q}(t + \Delta t) - \mathbf{q}(t)}{\Delta t} = \frac{\partial \mathcal{H}_w}{\partial \mathbf{p}}, \quad (73)$$

$$\frac{d\mathbf{p}}{dt} \approx \frac{\mathbf{p}(t + \Delta t) - \mathbf{p}(t)}{\Delta t} = -\frac{\partial \mathcal{H}_w}{\partial \mathbf{q}}. \quad (74)$$

By using numerical differentiation, Hamiltonian systems naturally evolve. We can learn the Hamiltonian from data using the following residual:

$$\mathcal{L}_r = \left\| \frac{d\mathbf{q}}{dt} - \frac{\partial \mathcal{H}_w}{\partial \mathbf{p}} \right\|_2^2 + \left\| \frac{d\mathbf{p}}{dt} + \frac{\partial \mathcal{H}_w}{\partial \mathbf{q}} \right\|_2^2. \quad (75)$$

Some work proposes advanced integrators or improved architecture [150], [151], [152], [153] for more accurate prediction. HGN [148] combines generative models such as variational auto-encoders (VAE) [154] and Hamiltonian neural networks to model time-dependent systems with uncertainty.

Convolutional Architectures. Convolutional neural networks are widely used in image processing and computer vision. Convolution utilizes the local dependency of pixels

on image data to extract semantic information. In the field of numerical computing, certain convolutional kernels can be viewed as a numerical approximation of differential operators. Many studies [75], [76], [80], [155] exploit this connection between convolutional kernels and (spatial) differential operators to develop convolutional neural architectures for physics-informed machine learning. Specifically, a one-dimensional Laplace operator can be approximated (discretized) by

$$D_1 \approx \frac{1}{h^2} \begin{pmatrix} 1 & -2 & 1 \end{pmatrix}. \quad (76)$$

Similarly, a two-dimensional Laplace operator can be approximated by

$$D_2 \approx \frac{1}{h^2} \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix} \approx \frac{1}{4h^2} \begin{pmatrix} 1 & 2 & 1 \\ 2 & -12 & 2 \\ 1 & 2 & 1 \end{pmatrix}. \quad (77)$$

The former convolutional kernel is called a five-point stencil; the latter is called a nine-point stencil and has a higher approximation order. Similarly, we can define convolutional kernels to approximate a Laplace operator in higher dimensions. By discretizing state variables u and applying these discretized convolutional operators to them, we can approximate the function of Laplace operators. Suppose we discretize a two-dimensional state variable $u(x, y)$ based on a mesh (grid) $U = (u_{ij})_{1 \leq i, j \leq n}$. Then we have

$$\Delta u(x, y) \approx D_2 * U, \quad (78)$$

where $*$ denotes the (discretized) convolution operation. We also can numerically represent other differential operators by using different convolutional kernels. By discretizing states and differential operators in spatial dimensions, we can naturally use convolutional neural architectures to solve PDEs or learn from data. Another advantage of discretization is that the Dirichlet boundary condition and initial condition can be easily satisfied by assigning boundary/initial points to given values. These convolutional neural architectures are usually jointly used with recurrent architecture like LSTMs as a Conv-LSTM network for learning spatial-temporal dynamic systems [75], [76]. [81] proposes a novel Conv-ResNet based architecture with a PDE preserving part and a learnable part for solving forward and inverse problems. It also introduces a U-Net architecture [156] with an encoder and a decoder to extract multiple-resolution features.

However, a limitation of vanilla CNN architecture is that it can only be used in a regular grid. For problems with complex irregular geometric domains, new methods need to be developed. [80] proposes a parameterized coordinate transformation from the irregular physical domain to a regular reference domain,

$$\mathbf{x} = \mathcal{G}(\boldsymbol{\xi}), \boldsymbol{\xi} = \mathcal{G}^{-1}(\mathbf{x}), \quad (79)$$

where $\mathbf{x} \in \Omega_p$ is the irregular physical domain and $\boldsymbol{\xi} \in \Omega_r$ is the regular reference domain. This map needs to be a bijection to ensure its reversibility. Then, we need to transform the PDEs into the reference domain by using a theorem of variable substitution,

$$\frac{\partial}{\partial x_i} = \sum_j \frac{\partial}{\partial \xi_j} \frac{\partial \xi_j}{\partial x_i}. \quad (80)$$

For a higher-order differentiable operator, we can simply apply the discretized version of Equation (80) to avoid a complicated theoretical derivation. Finding an analytical mapping of $\mathcal{G}$ is impossible as a practical matter; a feasible solution is to calculate and store the mapping and its inverse numerically [157]. Besides using a coordinate transformation from an irregular domain to a regular reference domain, there are some studies that use graph networks [40], [144], [158] to learn (next-timestep) simulation results from data with PDE as inductive biases. [159] improves the performance of the graph network based architecture by introducing attention layers over the temporal dimension. [160], [161] also adopts graph neural networks to improve operator learning.

Domain Decomposition. Domain decomposition is a basic and effective framework to improve the performance of PINNs on large-scale problems or multi-scale problems. It partitions a domain into many subdomains and solves an easier problem in each subdomain using smaller subnetworks. To ensure the consistency and continuity of the whole solution, additional loss terms are imposed at the interfaces between these subdomains. This is a general framework for improving PINNs and many techniques introduced in previous sections can also be combined with domain decomposition.

Suppose we have a domain Ω and it is decomposed into many subdomains $\Omega = \bigcup_{k=1}^K \Omega^k$. The PDEs in each subdomain are

$$\mathcal{F}^k(u; \theta^k)(\mathbf{x}) = 0, \mathbf{x} \in \Omega^k, \quad (81)$$

$$\mathcal{B}^k(u; \theta^k)(\mathbf{x}) = 0, \mathbf{x} \in \partial\Omega^k, \quad (82)$$

$$\mathcal{I}^k(u; \theta^k)(\mathbf{x}) = 0, \mathbf{x} \in \Omega_0^k. \quad (83)$$

In each domain, we sample datasets of collocation points $\mathcal{D}_r^k = \{\mathbf{x}_i^k : \mathbf{x}_i \in \Omega^k\}$ and boundary/initial collocation points $\mathcal{D}_b^k = \{\mathbf{x}_i^k : \mathbf{x}_i \in \partial\Omega^k\}$, $\mathcal{D}_i^k = \{\mathbf{x}_i^k : \mathbf{x}_i \in \Omega_0^k\}$. Note that mathematically subdomains could be disjoint. However, in practice these subdomains need to have overlapping area to allow the subnetworks training on these subdomains to communicate with each other. This is necessary to ensure consistency of the solution. We call these overlapping areas the interface and we denote $\{I^m : I^m \subset \Omega, 1 \leq m \leq M\}$ to be the set of these interfaces. We can then sample collocation points from interfaces $\mathcal{D}_I^m = \{\mathbf{x}_i^m : \mathbf{x}_i^m \in I^m\}$. For simplicity of notation, we use u^+ and u^- to denote two subnetworks' learning on two adjacent subdomains for any interface. In each subdomain Ω^k , we parameterize the state variables u with a (small) neural network $u_k(\mathbf{x})$ parameterized by w_k . Denote all weights to be $w = (w_1, \dots, w_K)$. The general training losses for PINNs with domain decomposition are

$$\mathcal{L} = \sum_{k=1}^K (\lambda_r^k \mathcal{L}_r^k + \lambda_b^k \mathcal{L}_b^k + \lambda_i^k \mathcal{L}_i^k) + \sum_{m=1}^M \lambda_I^m \mathcal{L}_I^m. \quad (84)$$

Here, $\mathcal{L}_r^k, \mathcal{L}_b^k, \mathcal{L}_i^k$ are subdomain losses for each subdomain Ω_k . $\mathcal{L}_I^m$ is called the interface loss. Subdomain losses can be optimized independently for each subnetwork. However, because interface losses are communication losses between these subnetworks, they should be optimized together as a message passing from one subnetwork to another. There are different approaches for the decision choice of interface

losses and subdomain losses. Here, we will introduce several representative studies adopting domain decomposition techniques.

cPINNs [83] considers systems obeying conservation laws:

$$\frac{\partial u}{\partial t} + \nabla \cdot f(u, u_x, u_{xx}, \dots) = 0. \quad (85)$$

It decomposes the whole domain into the subdomains mentioned above and designs the following interface loss:

$$\mathcal{L}_I^m = \frac{1}{N_m} \sum_{i=1}^{N_m} (\lambda_{\text{Avg}} |u^-(\mathbf{x}_i^m) - u^+(\mathbf{x}_i^m)|^2 + \lambda_{\text{flux}} |f(u^-(\mathbf{x}_i^m)) \cdot \mathbf{n} - f(u^+(\mathbf{x}_i^m)) \cdot \mathbf{n}|^2). \quad (86)$$

The first loss term penalizes the difference of two subnetworks on the interface and the second loss term enforces the flux through the interface to be the same. For inverse problems, learned parameters for different subdomains should also be penalized,

$$\mathcal{L}_{\text{reg}} = \frac{1}{N_m} \sum_{i=1}^{N_m} (\theta^+(\mathbf{x}_i^m) - \theta^-(\mathbf{x}_i^m))^2. \quad (87)$$

However, penalizing the flux between interfaces is only feasible for systems satisfying the conservative laws. To resolve the limitation, XPINNs [82], [92] further proposes a generalized version of interface conditions. XPINNs proposes the following interface conditions:

$$\mathcal{L}_I^m = \frac{1}{N_m} \sum_{i=1}^{N_m} (\lambda_{\text{Avg}} \|u^-(\mathbf{x}_i^m) - u^+(\mathbf{x}_i^m)\|^2 + \lambda_F \|\mathcal{F}^-(u^-; \theta^-)(\mathbf{x}_i^m) - \mathcal{F}^+(u^+; \theta^+)(\mathbf{x}_i^m)\|^2). \quad (88)$$

The second term is different from cPINNs since it enforces the continuity of general PDE residuals rather than flux continuity. It also is more flexible since it allows the same code for both forward and inverse problems. It also makes it possible to add more domain-specific penalty terms in practical usage.

cPINNs and XPINNs are two basic methods using the domain decomposition framework. There are many methods that enhance domain decomposition with other techniques to improve performance. For FBPINNs, [84] proposes to apply different weighted normalization layers to different subdomains to handle multi-scale problems. [85] scales cPINNs and XPINNs to problems with a larger scale with both optimized hardware and software implementation based on the MPI framework. This approach is able to train PINNs efficiently with multiple GPUs. [162] combines domain decomposition in temporal dimensions with a traditional ODE solver to boost accuracy. [163] introduces adaptive weight balancing for interfaces based on the intersection of union (IoU). [61], [117] combine variational formulation with domain decomposition. [164] proposes to use an architecture-gated mixture of experts (MoE) [165], which can be viewed as a soft version of domain decomposition, since it does not explicitly divide subdomains and subnetworks.

3.2.7 Open Challenges and Future Work.

Though many attempts have been made to improve the convergence speed and accuracy of PINNs, there is still much room for improvement, which is left for future work. Here, we present several important topics that are far from being fully explored.

- **Optimization Process.** The optimization method of PINNs can be improved in many respects. The training process of PINNs is significantly different from ordinary neural networks. Current optimizers and loss functions might not be optimal for PINNs. However, existing attempts are not ready, either theoretically or experimentally, to build a stable and effective neural solver.
- **Model Architecture** The neural network architecture, like other fields of deep learning, still needs more study. In the frontiers of deep learning, there are many novel architectures, such as normalization layers, and transformer architectures have been proposed which are shown to be superior in multiple domains. However, the application of these architectures in the field of physics-informed machine learning is far from completely explored.
- **Solving High Dimensional Problems.** High dimensional PDEs like HJB equations and Schrödinger equations play a key role in science and engineering but they are notoriously difficult to solve due to the curse of dimensionality. The efficiency of neural networks representing high dimensional functions provides a promising advantage of neural solvers using physics-informed machine learning [91], [166]. This is an open challenge for solving high dimensional PDEs with a wide range of applications such as quantum mechanics, molecular dynamics and control theory using neural networks.

3.3 Neural Operator

In this section, we will first formally give the goal of neural operator, followed by revisiting several important methods (and their variants) in this field. These methods can be broadly classified into four categories, including the direct methods represented by the DeepONet [167], Green's function learning, grid-based operator learning (which is similar to approximating image-to-image mappings), and graph-based operator learning. A brief summary is provided in Table 3. Finally, in Section 3.3.6, we mark the open challenges and future work in this field.

3.3.1 Problem Formulation

The goal of a neural operator is to approximate a latent operator, that is, a mapping between the (vector of) parameters and the state variables (with neural networks). A neural operator solves a class of differential equations that map given parameters or control functions $\theta \in \Theta$ to its solutions (i.e., state variables). The physical laws in Equation (3) are (partially) known and we might have a dataset of $\mathcal{D} = \{\tilde{G}(\theta_i)(\mathbf{x}_j)\}_{1 \leq i \leq N_1, 1 \leq j \leq N_2}$ of different parameter

θ and collocation points $\mathbf{x}$, where $\tilde{G}$ is the latent operator. Mathematically, the goal can be formalized as

$$\min_{w \in W} \|G_w(\theta)(\mathbf{x}) - \tilde{G}(\theta)(\mathbf{x})\|, \quad (89)$$

where $G : \Theta \times \Omega \rightarrow \mathbb{R}^m$ is the neural operator (a neural network with weights w) and $\tilde{G}$ is the ground truth.

For any $\theta \in \Theta$, $\tilde{G}(\theta)(\cdot)$ is the solution to the governing equations in Equation (3). The difference between a neural solver and a neural operator is that the goal of the neural operator is to learn a surrogate model representing ODEs/PDEs for all $\theta \in \Theta$ rather than just solving an instance of the physical system. The appearance of neural operators might revolutionize the surrogate modeling that is widely used in science and engineering. The key advantage of a neural operator is the strong generalization ability and large model capacity. Similar to pre-trained models in CV and NLP, a large pre-trained neural operator could be employed as a surrogate for the expensive traditional ODEs/PDEs' solver or digital twins for a real physical system. From the perspective of algorithms, designing specialized model architectures and pre-training methods for neural operators are important open problems for physics-informed machine learning. From the perspective of applications, finding important application scenarios or downstream tasks for neural operators will be a challenge that requires interdisciplinary collaboration.

3.3.2 Direct Methods

Based on the Universal Approximation Theorem of Operators [188], the direct methods parameterize the mapping by a neural network which takes both the parameters θ and the coordinates $\mathbf{x}$ as its inputs. DeepONet [167] is one of the most famous representatives. In the following text, we will briefly introduce this method and its relevant variants.

DeepONets. We now present the architecture of DeepONets as follows,

$$G_w(\theta)(\mathbf{x}) = b_0 + \sum_{k=1}^p b_k(\theta) t_k(\mathbf{x}), \quad (90)$$

where G is the neural operator instantiated as DeepONet with learnable parameters w , $b_0 \in \mathbb{R}$ is a learnable bias, and the branch network $(b_1, \dots, b_p)$ as well as the trunk network $(t_1, \dots, t_p)$ are two neural networks. The networks take the parameters θ and the coordinates $\mathbf{x}$ as the inputs, and output a vector of width p . We note that DeepONet does not specify the architectures of the branch and trunk networks, which can be FNNs, ResNets, or other architectures. Besides, if θ is an infinite-dimensional vector (e.g., a function in an abstract Hilbert space), we may need to represent it with another finite-dimensional vector since the width of the neural layer cannot be infinite. For instance, if θ is a function $f(x)$, $x \in \mathbb{R}$, we may represent it in terms of the first n Fourier coefficients of f or the values at a set of given points $[f(x_1), \dots, f(x_n)]^\top$.

Given the dataset $\mathcal{D} = \{\tilde{G}(\theta_i)(\mathbf{x}_j)\}_{1 \leq i \leq N_1, 1 \leq j \leq N_2}$, we can train the DeepONet with the following supervised loss function:

$$\mathcal{L} = \frac{1}{N_1 N_2} \sum_{i=1}^{N_1} \sum_{j=1}^{N_2} \|G_w(\theta_i)(\mathbf{x}_j) - \tilde{G}(\theta_i)(\mathbf{x}_j)\|^2. \quad (91)$$

Category & Formulation	Representative	Description
Direct Methods $G_w(\theta)(\mathbf{x}) = b_0 + \sum_{k=1}^P b_k(\theta)t_k(\mathbf{x})$	DeepONet [167]	Parameterize b_k and t_k with neural networks, which are trained with supervised data.
	Physics-informed DeepONet [168]	Train DeepONet with a combination of data and physics-informed losses.
	Improved Architectures for DeepONet [169], [170]	Including modified network structures (see Eq. (95)), input transformation ($\mathbf{x} \mapsto (\mathbf{x}, \sin(\mathbf{x}), \cos(\mathbf{x}), \dots)$), POD-DeepONet (see Eq. (97)), and output transformation (see Eq. (98) and Eq. (99)).
	Multiple-input DeepONet [171]	A variant of DeepONet taking multiple various parameters as input, i.e., $\tilde{G}: \Theta_1 \times \Theta_2 \times \dots \times \Theta_n \rightarrow Y$.
	Pre-trained DeepONet for Multi-physics [172], [173]	Model a multi-physics system with several pre-trained DeepONets serving as building blocks.
	Other Variants	Including Bayesian DeepONet [174], multi-fidelity DeepONet [175], and MultiAuto-DeepONet [176].
Green's Function Learning $G_w(\theta)(\mathbf{x}) = \int_{\Omega} \mathcal{G}(\mathbf{x}, \mathbf{y})\theta(\mathbf{y})d\mathbf{y} + u_{\text{hom}}(\mathbf{x})$, where θ is a function $\theta = v(\mathbf{x})$	Methods for Linear Operators [177], [178]	Parameterize $\mathcal{G}$ and u_{hom} with neural networks, which are trained with supervised data (and possibly physics-informed losses).
	Methods for Nonlinear Operators [179]	Discretize the PDEs and use trainable mappings to linearize the target operator, where Green's function formula is subsequently applied to construct the approximation.
Grid-based Operator Learning $G_w(\theta) = \{u(\mathbf{x}_i)\}_{i=1}^N$, where $\{u(\mathbf{x}_i)\}_{i=1}^N$ and $\theta = \{v(\mathbf{x}_i)\}_{i=1}^N$ are discretizations of input and output functions in some grids	Convolutional Neural Network [80], [180]	A convolutional neural network is utilized to approximate such an image-to-image mapping, where the loss function is based on supervised data (and possibly physics-informed losses).
	Fourier Neural Operator [181]	Several Fourier convolutional kernels are incorporated into the network structure, to better learn the features in the frequency domain.
	Neural Operator with Attention Mechanism [182], [183], [184]	The attention mechanism is introduced to the design of the network structure, to improve the abstraction ability of the model.
Graph-based Operator Learning $G_w(\theta) = \{u(\mathbf{x}_i)\}_{i=1}^N$, where $\{u(\mathbf{x}_i)\}_{i=1}^N$ and $\theta = \{v(\mathbf{x}_i)\}_{i=1}^N$ are discretizations of input and output functions in some graphs	Graph Kernel Network [185]	A graph kernel network is employed to learn such a graph-based mapping.
	Multipole Graph Neural Operator [186]	The graph kernel is decomposed into several multi-level sub-kernels, to capture multi-level neighboring interactions.
	Graph Neural Operator with Autogressive Methods [187]	Extend graph neural operators to time-dependent PDEs.

TABLE 3: A brief summary of the methods in the neural operator.

Hereinafter, we refer to Equation (91) as the operator loss function $\mathcal{L}_{\text{operator}}$. It is worth noting that, in Equation (90) the output of DeepONet is a scalar; however, the solution to the PDEs (i.e., $\tilde{G}(\theta)(\mathbf{x})$) can be a vector of dimension higher than 1. To bridge this gap, we can split the outputs of the branch and trunk networks and apply Equation (90) to each group separately to generate the components of the resulting vector. For example, supposing that $\tilde{G}(\theta)(\mathbf{x})$ is a 2-dimensional vector and $p = 100$; then, we can use the following ansatz,

$$G_w(\theta)(\mathbf{x}) = \mathbf{b}_0 + \left[\sum_{k=1}^{50} b_k(\theta)t_k(\mathbf{x}), \sum_{k=51}^{100} b_k(\theta)t_k(\mathbf{x}), \right]^\top, \quad (92)$$

where the bias $\mathbf{b}_0 \in \mathbb{R}^2$ also becomes a 2-dimensional vector. More discussion on such multiple-output DeepONets can be found in the paper [170].

DeepONet is a simple but effective neural operator. With the help of the generalizability of neural networks, DeepONet is able to learn latent operators from the data, not just the solution to a certain instance of the PDEs. In this way, for any instance in a class of parameterized PDEs, only a single forward pass of DeepONet is needed to obtain its solution. This is something that neural solvers like PINNs cannot do. Unlike numerical methods such as FEM, both the input and

output of DeepONet are mesh-independent and thus it is more flexible and less sensitive to the increase in dimensionality. However, DeepONet still has some limitations. For example, since DeepONet is purely data-driven, the requirement of the training data is usually relatively large, especially for some complex PDEs. The generation of these data (by numerical simulations or experiments) is often expensive, thus greatly limiting its application. Therefore, some variants of DeepONet have been proposed to solve these problems, which are described below.

Physics-informed DeepONets. As mentioned above, a severe problem with DeepONet is that training data acquisition is sometimes too expensive. A straightforward way to overcome this defect is to incorporate the idea of PINNs [30] and add the physics-informed loss $\mathcal{L}_{\text{physics}}$ into the loss function. Such a method [168] can reduce the data requirements of DeepONet (for some simple PDEs, the labeled data are not even needed) since the physics-informed loss does not require any labeled data, and only some points in the domain Ω is necessary (that is to say, we only need to sample $\{\theta_i\}$ and $\{\mathbf{x}_j\}$ and do not have to evaluate $\{\tilde{G}(\theta_i)(\mathbf{x}_j)\}$). The loss function of this method can be expressed as

$$\mathcal{L} = \mathcal{L}_{\text{operator}} + \mathcal{L}_{\text{physics}}, \quad (93)$$

where $\mathcal{L}_{\text{operator}}$ is defined in Equation (91) and $\mathcal{L}_{\text{physics}}$ is

given by,

$$\begin{aligned} \mathcal{L}_{\text{physics}} = & \frac{1}{N_1} \sum_{i=1}^{N_1} \left(\frac{\lambda_r}{N_r} \sum_{j=1}^{N_r} \|\mathcal{F}(u_w; \theta_i)(\mathbf{x}_j)\|^2 \right. \\ & \left. + \frac{\lambda_i}{N_i} \sum_{j=1}^{N_i} \|\mathcal{I}(u_w; \theta_i)(\mathbf{x}_j)\|^2 + \frac{\lambda_b}{N_b} \sum_{j=1}^{N_b} \|\mathcal{B}(u_w; \theta_i)(\mathbf{x}_j)\|^2 \right), \end{aligned} \quad (94)$$

where $u_w = G_w(\theta_i)$ is the approximation of the solution to PDEs under parameters θ_i , N_r , N_i , N_b are, respectively, the number of points sampled inside the domain Ω , at the initial time, and on the boundary, and λ_r , λ_i , λ_b are the corresponding weights of losses. Recalling the loss function of PINNs in Equation (14), we find that Equation (94) is similar, except for additional parameterization of θ_i and the absence of the fourth term (i.e., the regular data loss whose role has been fulfilled by $\mathcal{L}_{\text{operator}}$ here).

Physics-informed DeepONet directly combines the approaches of PINNs and DeepONet, which alleviates the problems of both DeepONet's large data demand and PINNs' poor approximation of the solution to complex PDEs. This idea has been applied to solve other parametric systems in addition to parametric PDEs, such as a specific class of eigenvalue problems [189]. In addition, many variants based on physics-informed DeepONet have been proposed, such as the variant for long-term simulation of dynamic systems [190]. While physics-informed DeepONet has the advantages of physics-informed and data-driven learning, it is much too simple to combine the two learning methods by merging loss functions, where the weights of the losses can be difficult to specify. Similar to PINNs, some loss re-weighting and data re-sampling algorithms have been proposed to address this difficulty [169], [191]. Future work includes finding a more efficient way to combine physical prior knowledge with available data.

Improved Architectures for DeepONets. The architecture plays an important role in deep learning, including operator learning. Recently, researchers have proposed several improved architectures for DeepONets. These can be divided into two types: novel network structures and pre-processing & post-processing techniques.

Wang *et al.* [169] propose a novel modified structure for DeepONets, which incorporates encoders. The forward pass of the modified structure can be formulated as:

$$\begin{aligned} U &= \phi(\mathbf{W}_\theta \theta + \mathbf{b}_\theta), V = \phi(\mathbf{W}_x \mathbf{x} + \mathbf{b}_x), \\ \mathbf{H}_\theta^{(1)} &= \phi(\mathbf{W}_\theta^{(1)} \theta + \mathbf{b}_\theta^{(1)}), \mathbf{H}_x^{(1)} = \phi(\mathbf{W}_x^{(1)} \mathbf{x} + \mathbf{b}_x^{(1)}), \\ \mathbf{Z}_\theta^{(l)} &= \phi(\mathbf{W}_\theta^{(l)} \mathbf{H}_\theta^{(l)} + \mathbf{b}_\theta^{(l)}), \mathbf{Z}_x^{(l)} = \phi(\mathbf{W}_x^{(l)} \mathbf{H}_x^{(l)} + \mathbf{b}_x^{(l)}), \\ \mathbf{H}_\theta^{(l+1)} &= (1 - \mathbf{Z}_\theta^{(l)}) \odot U + \mathbf{Z}_\theta^{(l)} \odot V, \\ \mathbf{H}_x^{(l+1)} &= (1 - \mathbf{Z}_x^{(l)}) \odot U + \mathbf{Z}_x^{(l)} \odot V, \\ \mathbf{H}_\theta^{(L)} &= \phi(\mathbf{W}_\theta^{(L)} \mathbf{H}_\theta^{(L-1)} + \mathbf{b}_\theta^{(L)}), \\ \mathbf{H}_x^{(L)} &= \phi(\mathbf{W}_x^{(L)} \mathbf{H}_x^{(L-1)} + \mathbf{b}_x^{(L)}), \\ G_w(\theta)(\mathbf{x}) &= \langle \mathbf{H}_\theta^{(L)}, \mathbf{H}_x^{(L)} \rangle, \end{aligned} \quad (95)$$

where $l = 1, \dots, L-1$, ϕ denotes a given activation function; $\odot$ denotes the element-wise multiplication; $\{\mathbf{W}_\theta^{(i)}, \mathbf{b}_\theta^{(i)}\}_{i=1}^{L+1}$ and $\{\mathbf{W}_x^{(i)}, \mathbf{b}_x^{(i)}\}_{i=1}^{L+1}$ are, respectively, the learnable weights

and biases of the branch and trunk networks; and $\langle \cdot \rangle$ denotes the inner product. The modified structure utilizes two encoders to embed the inputs θ and $\mathbf{x}$ into two feature vectors U and V , respectively, in high-dimensional latent spaces. U and V are then added element-wise and fed into the hidden layers $\mathbf{H}_\theta^{(l)}$ and $\mathbf{H}_x^{(l)}$. In contrast to vanilla DeepONet (see Equation (90)), the messages of θ and $\mathbf{x}$ are merged before going through each hidden layer instead of merging them just before output, which improves the ability to abstract nonlinear features. In addition to the structure introduced above, other studies have considered the Auto-Decoder structure [192].

Pre-processing refers to techniques that process the input before putting it into the neural network. Feature expansion [170] is one of the frequently used techniques. For example, for the PDEs with oscillating solutions, a harmonic feature expansion [193] can be applied to the input $\mathbf{x}$ before entering into the trunk network,

$$\mathbf{x} \mapsto (\mathbf{x}, \sin(\mathbf{x}), \cos(\mathbf{x}), \sin(2\mathbf{x}), \cos(2\mathbf{x}), \dots), \quad (96)$$

where we assume $\mathbf{x}$ is a 1-dimensional vector. With carefully designed feature expansion or feature mapping, we can pass more valuable information to the neural network, allowing the model to better approximate the underlying operator. Similarly, we can also pre-compute the proper orthogonal decomposition (POD) modes of the state variables $u(\mathbf{x})$ from the dataset (after zero-mean normalization) and replace the trunk network with them (POD-DeepONet [170]),

$$G_w(\theta)(\mathbf{x}) = \phi_0(\mathbf{x}) + \sum_{k=1}^p b_k(\theta) \phi_k(\mathbf{x}), \quad (97)$$

where $\phi_0(\mathbf{x})$ is the mean function of $u(\mathbf{x})$, i.e., $\phi_0(\mathbf{x}) = \mathbb{E}_\theta[\tilde{G}(\theta)(\mathbf{x})]$, $\{\phi_1(\mathbf{x}), \dots, \phi_p(\mathbf{x})\}$ are the POD modes of $u(\mathbf{x})$.

In contrast, post-processing refers to techniques that process the output of the neural network before generating the approximation. For example, to stabilize the training process, we may rescale the output of the DeepONet to achieve a unit variance,

$$G_w(\theta)(\mathbf{x}) = \frac{1}{\sqrt{\text{Var}[\sum_{k=1}^p b_k(\theta) t_k(\mathbf{x})]}} \left[b_0 + \sum_{k=1}^p b_k(\theta) t_k(\mathbf{x}) \right], \quad (98)$$

where the variance $\text{Var}[\sum_{k=1}^p b_k(\theta) t_k(\mathbf{x})]$ depends on the specific initialization methods of the neural network. We refer to the paper [170] for a detailed discussion. Another example is the hard-constraint boundary conditions which have drawn much attention in PINNs (see Section 3.2.6, **Multiple NNs and Boundary Encoding.**). We consider the following Dirichlet BC,

$$\mathcal{B}(u; \theta)(\mathbf{x}) \triangleq u(\mathbf{x}) = g(\mathbf{x}), \mathbf{x} \in \partial\Omega, \quad (99)$$

where we note again that $\mathbf{x} = (x, t)$, x and t are the spatial and temporal coordinates, respectively. To enforce the above BC, we can construct our ansatz as follows,

$$G_w(\theta)(\mathbf{x}) = g(\mathbf{x}) + l(\mathbf{x}) \mathcal{N}(\theta)(\mathbf{x}), \quad (100)$$

where $\mathcal{N}(\theta)(\mathbf{x})$ is the output of the original DeepONet (see Equation (90)), and $l(x)$ is a smooth distance function which satisfies:

$$\begin{cases} l(x) = 0 & \text{if } x \in \partial\Omega, \\ l(x) > 0 & \text{otherwise.} \end{cases} \quad (101)$$

We note that, if $g(\mathbf{x})$ is not defined in total Ω , we may have to extend its definition smoothly. As for other types of BCs such as Periodic BCs [194], preprocessing techniques may be also utilized to enforce the BCs.

[195] **Multiple-input DeepONets.** As we discussed above, the DeepONet is designed for the operator whose input space Θ is a single Banach space, that is, the input θ can only be a vector (or a function) but not multiple vectors (or a vector-value function) which are defined on different spaces. To extend DeepONet to learn a operator with the input of multiple vectors (i.e., a multiple-input operator), a new theory of universal approximation needs to be proven and a new network architecture needs to be designed. The paper [171] gives the answers to both of them. The authors first prove the theory of universal approximation of neural networks for a multiple-input operator, which can be described as,

$$\tilde{G}: \Theta_1 \times \Theta_2 \times \cdots \times \Theta_n \rightarrow Y, \quad (102)$$

where $\Theta_1, \dots, \Theta_n$ are n (different) input spaces, and Y is the target space. In the context of the neural operator (see Section 3.3.1), we have $Y = \Omega \rightarrow \mathbb{R}^m$ and Equation (102) can be rewritten as,

$$\tilde{G}: \Theta_1 \times \Theta_2 \times \cdots \times \Theta_n \times \Omega \rightarrow \mathbb{R}^m. \quad (103)$$

Motivated by the newly proven theory, the authors propose the extension of DeepONet, MIONet, for the multiple-input operator,

$$G_w(\boldsymbol{\theta})(\mathbf{x}) = b_0 + \sum_{k=1}^p b_k^1(\theta_1) \cdots b_k^n(\theta_n) t_k(\mathbf{x}), \quad (104)$$

where $\theta_i \in \Theta_i$, $i = 1, \dots, n$ are n input vectors, $\boldsymbol{\theta} = [\theta_1, \dots, \theta_n]^\top$, and $(b_1^i, \dots, b_p^i)$, $i = 1, \dots, n$ are n independent branch networks.

Multiple-input DeepONet (MIONet) is very useful when we want to learn a latent operator which takes more than two functions as its inputs. For example, we consider the following ODE system,

$$\frac{du_1}{dt} = u_2(t), \quad (105)$$

$$\frac{du_2}{dt} = -f_1(t) \sin(u_1(t)) + f_2(t), \quad (106)$$

where $t \in (0, 1]$ and the initial condition is $u_1(0) = u_2(0) = 0$. Our operator of interest is given by,

$$\tilde{G}: (f_1, f_2) \mapsto u_1. \quad (107)$$

Here, the solution u_1 depends on both f_1 and f_2 , conforming to the definition of the multiple-input operator. We can employ the MIONet to learn such operators.

Pre-trained DeepONets for multi-physics.

We note that DeepONets learn a mapping between functions, which can be used as pre-trained models for fast inference. As proposed in [172] and [173] (DeepM&Mnets),

multiple pre-trained DeepONets are used as building blocks for multi-physical systems where there are multiple state variables and PDEs. We consider the following example of electroconvection,

$$\frac{\partial \mathbf{u}}{\partial t} = -\nabla p + \nabla^2 \mathbf{u} + \mathbf{f}, \quad (108)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (109)$$

$$-2\epsilon^2 \nabla^2 \phi = c^+ - c^-, \quad (110)$$

$$\frac{\partial c^\pm}{\partial t} = -\nabla \cdot (c^\pm \mathbf{u} - \nabla c^\pm \mp c^\pm \nabla \phi), \quad (111)$$

where $\mathbf{u}(\mathbf{x})$ is the velocity, p is the pressure, $\phi(\mathbf{x})$ is the electric potential, $c^+(\mathbf{x})$ and $c^-(\mathbf{x})$ are, respectively, the cation and anion concentrations, $\mathbf{f}(\mathbf{x})$ is the electrostatic body force, and ϵ is the Debye length.

We first define two classes of operators. The first class is to map the electric potential to other state variables:

$$G_\diamond: \phi \mapsto \diamond, \diamond = \mathbf{u}, p, c^\pm. \quad (112)$$

The second class is to map the concentrations to the electric potential:

$$G_\phi: (c^+, c^-) \mapsto \phi. \quad (113)$$

Without loss of clarity, we confuse the notations of the ground truth operator and the corresponding neural operator (i.e., DeepONet). Then, we can pretrain DeepONets for those operators with proper datasets. For example, we can train $G_{\mathbf{u}}$ on the dataset $\{G_{\mathbf{u}}(\phi_i)(\mathbf{x}_j)\}_{1 \leq i \leq N_1, 1 \leq j \leq N_2}$, where $\{\phi_i\}_{i=1}^{N_1}$ are sampled in the Hilbert space according to a certain distribution. After pretraining all the DeepONets, we finally train our ansatz, a neural network $\mathcal{N}: \mathbf{x} \mapsto (\hat{\mathbf{u}}, \hat{p}, \hat{c}^\pm, \hat{\phi})$ with the dataset $\{(\mathbf{u}(\mathbf{x}_i), p(\mathbf{x}_i), c^\pm(\mathbf{x}_i), \phi(\mathbf{x}_i))\}_{i=1}^{N_d}$. The loss function is given by

$$\mathcal{L} = \lambda_{\text{data}} \mathcal{L}_{\text{data}} + \lambda_{\text{op}} (\mathcal{L}_{\text{op1}} + \mathcal{L}_{\text{op2}}), \quad (114)$$

$$\mathcal{L}_{\text{data}} = \sum_{\diamond \in \{\mathbf{u}, p, c^\pm, \phi\}} \frac{1}{N_d} \sum_{i=1}^{N_d} (\hat{\diamond}(\mathbf{x}_i) - \diamond(\mathbf{x}_i))^2, \quad (115)$$

$$\mathcal{L}_{\text{op1}} = \sum_{\diamond \in \{\mathbf{u}, p, c^\pm\}} \frac{1}{N_{\text{op}}} \sum_{i=1}^{N_{\text{op}}} (\hat{\diamond}(\mathbf{x}_i) - G_\diamond(\hat{\phi})(\mathbf{x}_i))^2, \quad (116)$$

$$\mathcal{L}_{\text{op2}} = \frac{1}{N_{\text{op}}} \sum_{i=1}^{N_{\text{op}}} (\hat{\phi}(\mathbf{x}_i) - G_\phi(\hat{c}^+, \hat{c}^-)(\mathbf{x}_i))^2, \quad (117)$$

where $\mathcal{L}_{\text{data}}$ measures the data mismatch and $\mathcal{L}_{\text{op1}}, \mathcal{L}_{\text{op2}}$ measure the deviation between the neural network and the pre-trained DeepONets. We note that the pre-trained DeepONets stay fixed during the training process. Moreover, if we need to estimate only some of the state variables, we can make the other state variables the hidden outputs. We refer to [172] and [173] for relevant details.

DeepM&Mnets are very suitable for performing "physically meaningful" interpolation on discrete observation data points, where the physical priors are embedded in pre-trained DeepONets. Since we have decoupled the training of DeepONets and that of the ansatz, incorporating the estimations of DeepONets into the loss function does not bring too much computational overhead to the training of the latter. However, the premise is that we need to train these DeepONets on another dataset in advance, where the

acquisition of the dataset and the training of DeepONets are often very time-consuming. This is also a fundamental drawback of applying DeepONets as building blocks to other neural models.

Other Variants. Besides the important variants of DeepONet introduced above, many other variants have been proposed to solve problems in different domains, such as Bayesian DeepONets for training with noise and uncertainty estimation [174], multi-fidelity DeepONets for training with multi-fidelity data [175], [196], and MultiAuto-DeepONets for high-dimensional stochastic problems and multi-resolution inputs [176].

3.3.3 Green's Function Learning

In this subsection, we first review the concept of Green's function and the goal of Green's function learning. Then we introduce the methods of Green's function learning for linear and nonlinear operators, respectively.

Green's function. The method of Green's function is a basic approach for manually solving a class of linear PDEs such as Poisson's equation. It can be described as

$$\mathcal{F}_L(u) = f, x \in \Omega, \quad (118)$$

$$\mathcal{B}_L(u) = g, x \in \partial\Omega, \quad (119)$$

where $\mathcal{F}_L$ and $\mathcal{B}_L$ are two linear operators, $f(x)$ is the forcing term, and $g(x)$ is the constraint function on the boundary $\partial\Omega$. Green's function $\mathcal{G}(x, y)$ corresponding to the above boundary value problem is implicitly defined as follows,

$$\mathcal{F}_L(\mathcal{G}(x, y)) = \delta(y - x), x, y \in \Omega, \quad (120)$$

$$\mathcal{B}_L(\mathcal{G}(x, y)) = 0, x \in \partial\Omega, \quad (121)$$

where the inputs of $\mathcal{F}_L$ and $\mathcal{B}_L$ are both the function $x \mapsto \mathcal{G}(x, y)$ for fixed y and $\delta(\cdot)$ is the Dirac delta function. From the superposition principle, we can construct the solution to the boundary value problem defined by Equation (118) and (119) as

$$u(x) = \int_{\Omega} \mathcal{G}(x, y) f(y) dy + u_{\text{homo}}(x), \quad (122)$$

where u_{homo} is the homogeneous solution which satisfies $\mathcal{F}_L(u_{\text{homo}}(x)) = 0, x \in \Omega$ and $\mathcal{B}_L(u_{\text{homo}}(x)) = g, x \in \partial\Omega$.

However, for complicated PDEs, the analytical expression of Green's function $\mathcal{G}(x, y)$ may be hard to solve. To tackle this challenge, we may approximate Green's function $\mathcal{G}(x, y)$ with neural networks, which is the original intention of Green's function learning. To be formal, Green's function learning hopes to learn an operator $\tilde{G}: f \mapsto u$ from the forcing term f to the solution u using the structure of the Green's function (see Equation (122)). Green's function learning can be considered as a subclass of the neural operators, where the parameter θ is restricted to be the forcing term f . Several methods have been proposed for Green's function learning, which will be presented in the following.

Green's function learning for linear operators.

As for linear $\mathcal{F}_L$ and $\mathcal{B}_L$, we can directly utilize the format given in Equation (122) to construct our ansatz as proposed in [177] and [178]. Specifically, we parameterize the Green's function $\mathcal{G}(x, y)$ and the homogeneous solution u_{homo} with two neural networks which are

trained in a supervised learning manner on a dataset $\mathcal{D} = \{\tilde{G}(f_i)(x_j)\}_{1 \leq i \leq N_1, 1 \leq j \leq N_2}$. In addition, physics-informed losses corresponding to the PDEs in Equation (118) and (119) can be incorporated in the loss function [177].

Compared with other neural operator methods, Green's function learning has the following advantages. First, the structure of Green's function (see Equation (122)) contains more priors about the physical system, which makes the training of neural networks more data-efficient. Second, it is mathematically easier to approximate Green's function than the latent operator $\tilde{G}$ [178]. Third, the structure of Green's function can be employed flexibly, since many physical or mathematical properties (such as the symmetry of Green's function) can be encoded into the network architecture to improve accuracy. However, Green's function learning also has some limitations. On the one hand, such methods are limited to a special class of PDEs as in Equation (118) and (119). On the other hand, the input dimension of Green's function is twice the spatial dimension, which makes it infeasible to apply many grid-based methods.

Green's function learning for nonlinear operators.

We recall that the format of Green's function given in Equation (122) is only available for linear operators which satisfy the superposition principle. Necessary processing techniques must be undertaken when handling nonlinear boundary value problems of the form

$$\mathcal{F}_N(u) = f, x \in \Omega, \quad (123)$$

where $\mathcal{F}_N$ is a nonlinear operator and a linear boundary condition is imposed as in Equation (119). For example, in DeepGreen [179] we first discretize the boundary value problem to obtain that

$$\mathbf{F}_N[\mathbf{u}] = \mathbf{f}, \quad (124)$$

where $\mathbf{F}_N, \mathbf{u}, \mathbf{f}$ are spatial discretizations of $\mathcal{F}_N, u, f$, respectively. Then we use two mappings, ψ and ϕ , which are parameterized by autoencoder networks, to transform $\mathbf{u}$ and $\mathbf{f}$,

$$\mathbf{v} = \psi(\mathbf{u}), \quad (125)$$

$$\mathbf{h} = \phi(\mathbf{f}), \quad (126)$$

where $\mathbf{v}$ and $\mathbf{h}$ satisfy the following:

$$\mathbf{F}'_L[\mathbf{v}] = \mathbf{h}, \quad (127)$$

for some linear operator $\mathbf{F}'_L$ in the latent space. Finally, we can apply the structure of Green's function to this linearized boundary value problem. It is noted that we learn the two mappings, ψ and ϕ , from a dataset $\mathcal{D} = \{(\mathbf{f}_i, \mathbf{u}_i)\}_{i=1}^N$ and do not specify $\mathbf{F}'_L$, whose linearity is enforced by a linear superposition loss.

DeepGreen has successfully extended Green's function learning to nonlinear boundary value problems. Nevertheless, we should point out that there is a no theoretically rigorous underpinning for the existence of ψ or for ϕ , and the linearity of $\mathbf{F}'_L$ does not strictly hold. Besides, DeepGreen is inherently a grid-based method that may be prohibitive for high-dimensional problems.

3.3.4 Grid-based Operator Learning

Besides the direct methods such as DeepONet, we can also formalize the latent operator as a grid-based mapping, that is,

$$\tilde{G}: \theta \mapsto \{u(\mathbf{x}_i)\}_{i=1}^N, \quad (128)$$

where u is the solution to PDEs under parameters θ , and $\{\mathbf{x}_i\}_{i=1}^N$ is a set of N query coordinates (i.e., a grid of points) which is usually predefined. We call the methods to learn the operator of such a formalization *grid-based operator learning* methods. If θ is (some representation of) a function and shares the same grid with u , Equation (128) can be equivalently rewritten as

$$\tilde{G}: \theta = \{v(\mathbf{x}_i)\}_{i=1}^N \mapsto \{u(\mathbf{x}_i)\}_{i=1}^N, \quad (129)$$

where v is the input function. Moreover, if the points are uniformly distributed (i.e., a regular grid), we can replace the notations of $\{v(\mathbf{x}_i)\}_{i=1}^N$ and $\{u(\mathbf{x}_i)\}_{i=1}^N$ with tensors $\mathbf{X}$ and $\mathbf{Y}$, respectively. Therefore, such operators are also called *image-to-image mappings*.

Convolutional neural networks. The convolutional neural network [197] is a well-known and powerful model to learn image-to-image mapping in the field of computer vision. Here, we can also utilize convolutional architecture, such as the U-Net architecture [156], to approximate an operator which can be formalized as image-to-image mapping:

$$\tilde{G}(\theta) \approx G_w(\theta) = \text{CNN}_w(\theta), \quad (130)$$

where the output size of the convolutional neural network CNN is the same as the size of the regular grid.

According to the connection between numerical differential operators and convolutional kernels (see Section 3.2.6, **Convolutional Architectures**), physics-informed learning methods already have been developed for convolutional neural networks [80], [180], which do not require any labeled data. Moreover, [198] applies a Bayesian framework on convolutional neural networks, facilitating pointwise uncertainty quantification.

With the help of the convolution operation, convolutional neural networks can effectively extract segmentation behind the training “images” and learn a low-rank representation of the physical laws. However, such architectures suffer from the “curse of dimensionality” due to the dependency of the grid and hardly utilize the frequency information of the input, which is sometimes very important for the input function (we note that the input is usually a very smooth function but not a real “image”). In addition, the architectures are only applicable for regular grids. Although there are methods which intend to map an irregular domain to a regular one (see Section 3.2.6, **Convolutional Architectures**), they are still very inflexible for geometrically complicated PDEs, which is exactly what graph-based methods are meant to solve (we discuss graph-based methods in Section 3.3.5)

Fourier neural operators. The Fourier neural operator (FNO) [181] is another architecture for learning image-to-image mappings, which considers the features of the input function in both spatial and frequency domains via the Fourier transformation. The architecture of FNOs is presented in Figure 3. In the FNO, we first apply a local

transformation $P: \mathbb{R} \rightarrow \mathbb{R}^{d_z}$ on the input function v (which is represented by the function values on a regular grid $\{\mathbf{x}_i\}_{i=1}^N$) as well as some extra features (if needed),

$$z_0(\mathbf{x}_i) = P(v(\mathbf{x}_i)) \in \mathbb{R}^{d_z}, i = 1, \dots, N, \quad (131)$$

where the transformation P is usually parameterized by a shallow fully-connected neural network. It is noted that the output $\{z_0(\mathbf{x}_i)\}_{i=1}^N$ lives on the same grid as v , which can be viewed as an image with d_z channels. In the next step, we iteratively apply L Fourier layers on z_0 :

$$z_0(\mathbf{x}_i) \mapsto z_1(\mathbf{x}_i) \mapsto \dots \mapsto z_L(\mathbf{x}_i), i = 1, \dots, N, \quad (132)$$

where $z_j \in \mathbb{R}^{d_z}, j = 1, \dots, L$. The Fourier layer is defined as

$$z_{l+1} = \sigma(\mathcal{F}^{-1}(\mathbf{R}_l \cdot \mathcal{F}(z_l)) + \mathbf{W}_l \cdot z_l + \mathbf{b}_l), \quad (133)$$

where $l = 0, \dots, L-1$, σ is an activation function; $\mathbf{b}$ is a bias; $\mathbf{R}_l$ and $\mathbf{W}_l$ are, respectively, the weight tensors in the frequency and spatial domain; $\mathcal{F}$ is the operator representing the Fast Fourier Transformation (FFT); and $\mathcal{F}^{-1}$ is its inverse. Finally, another local transformation $Q: \mathbb{R}^{d_z} \rightarrow \mathbb{R}$ is used to project z_L back to the domain of the output,

$$u(\mathbf{x}_i) = Q(z_L(\mathbf{x}_i)) \in \mathbb{R}, i = 1, \dots, N. \quad (134)$$

Here, we assume the input and output functions v and u are scalar-value functions. We also note that FNOs can be easily extended to the scenarios of vector-value functions with multi-channel versions of P and Q .

Using the Fourier transformation, FNOs can effectively abstract the features of the input function in the frequency domain, which makes their experimental performance significantly better than other architectures such as U-Net. Therefore, neural operators combined with the Fourier transformations have become a paradigm, and a lot of work is devoted to improving or applying this approach. For instance, [170] extends FNOs to geometrically complex cases and cases where the input and output functions are defined on different domains. [199] has designed a distributed version of FNOs for large-scale problems. Other relevant work includes introducing the wavelet transform [200] into operator learning [201], [202], improved FNO for irregular geometries [203], and so on [204], [205], [206]. In addition, FNOs recently have been applied for weather forecasting [8]. However, we must emphasize that FNOs are still grid-based and cannot overcome the “curse of dimensionality”.

Neural operators with the attention mechanism.

The attention mechanism is a famous and powerful tool for natural language processing, computer vision, and many machine learning tasks [207]. The vanilla attention mechanism can be formulated as

$$\mathbf{z}_i = \sum_{j=1}^n \alpha_{ij} \mathbf{v}_j, \alpha_{ij} = \frac{\exp(h(\mathbf{q}_i, \mathbf{k}_j))}{\sum_{s=1}^n \exp(h(\mathbf{q}_i, \mathbf{k}_s))}, \quad (135)$$

where $\{\mathbf{q}_j\}_{j=1}^n, \{\mathbf{k}_j\}_{j=1}^n, \{\mathbf{v}_j\}_{j=1}^n$ are the query vectors, key vectors, and value vectors respectively; $\mathbf{z}_i$ is the output corresponding to $\mathbf{q}_i$; and α_{ij} is the attention weight. The weight function $h(\cdot)$ is usually chosen as a scaled dot-product [129]. In this way, Equation (135) can be rewritten in matrix fashion:

$$\mathbf{Z} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right) \mathbf{V} \quad (136)$$

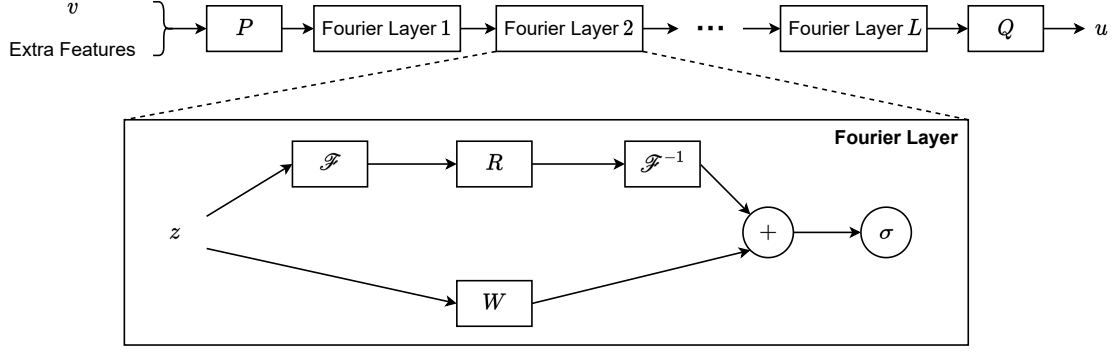


Fig. 3: The Architecture of FNOs.

where $q_i, k_i, v_i, z_i \in \mathbb{R}^d$ are the i -th row vectors of the matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{Z} \in \mathbb{R}^{n \times d}$ respectively.

In [182], Cao *et. al.* have introduced the attention mechanism into the architecture of neural operators. They employ CNN-like architectures to extract features from the input function (which is represented by its values on a regular grid, as a matrix $\mathbf{X}$) and incorporate the attention mechanism in hidden layers. Specifically, the matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are parameterized as follows:

$$\mathbf{Q} \triangleq \mathbf{y}_{\text{in}} \mathbf{W}_Q, \mathbf{K} \triangleq \mathbf{y}_{\text{in}} \mathbf{W}_K, \mathbf{V} \triangleq \mathbf{y}_{\text{in}} \mathbf{W}_V, \quad (137)$$

where $\mathbf{y}_{\text{in}} \in \mathbb{R}^{n \times d}$ is the input feature embedding, and $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d}$ are learnable weights. The attention head that maps a feature embedding $\mathbf{y}_{\text{in}} \in \mathbb{R}^{n \times d}$ to another feature embedding $\mathbf{y}_{\text{out}} \in \mathbb{R}^{n \times d}$ is given by

$$\mathbf{y}_{\text{out}} = \text{Ln}(\mathbf{y}_{\text{in}} + \text{Attn}(\mathbf{y}_{\text{in}}) + g(\text{Ln}(\mathbf{y}_{\text{in}} + \text{Attn}(\mathbf{y}_{\text{in}})))), \quad (138)$$

where $\text{Ln}(\cdot)$ is the layer normalization, $g(\cdot)$ is parameterized as a neural network, and Attn is the attention layer, which can be one of the followings,

$$(\text{Fourier-type attention}) \quad \text{Attn}(\mathbf{y}_{\text{in}}) \triangleq (\tilde{\mathbf{Q}} \tilde{\mathbf{K}}^\top) \mathbf{V} / n, \quad (139)$$

$$(\text{Galerkin-type attention}) \quad \text{Attn}(\mathbf{y}_{\text{in}}) \triangleq \mathbf{Q} (\tilde{\mathbf{K}}^\top \tilde{\mathbf{V}}) / n, \quad (140)$$

where $\tilde{\cdot}$ denotes learnable non-batch-based normalization. Here, we can find the softmax function does not show up, reducing much computational effort for high-dimensional inputs.

One of the limitations of the aforementioned attention mechanism is that the input and output feature embedding $\mathbf{y}_{\text{in}}$ and $\mathbf{y}_{\text{out}}$ literally live on the same grid. To address this limitation and allow for arbitrary query locations, [183] proposed the cross-attention mechanism, where the query matrix $\mathbf{Q}$ is encoded from the query locations instead of the input feature embedding $\mathbf{y}_{\text{in}}$. Another study [184] has developed the kernel-couple attention mechanism to better model the correlations between the query locations, which can be described as:

$$G_w(\theta)(\mathbf{x}) = \sum_{i=1}^n \text{softmax} \left(\int_{\Omega} \kappa(\mathbf{x}, \mathbf{x}') g(\mathbf{x}') d\mathbf{x}' \right)_i \odot \mathbf{e}_i(\theta), \quad (141)$$

where $\kappa: \Omega \times \Omega \rightarrow \mathbb{R}$ is a kernel function, g is a score function (which can be parameterized as a neural network), and $\mathbf{e}_i$ is the input feature encoder. We remark that the

above formalization is much like a variant of DeepONet but with the advanced attention mechanism.

By introducing the attention mechanism, such models as described above can better model complicated properties of the latent operator, which are beneficial for real-world physical systems. In the future, big models incorporating attention mechanisms will be proposed, just like BERT [4] in natural language processing. However, such models should have a larger number of parameters and will require more training samples. Also, samples obtained from physical systems are often very expensive. How to address this contradiction will be the key to popularizing big models in the field of neural operators.

3.3.5 Graph-based Operator Learning

Graphs have gained much popularity in the community of science and engineering, as an expressive structure for modeling interactions between individuals and discretizing continuous space. In particular, the standard output of the numerical PDE solver (e.g., FEM) is a triangle mesh which is a specific type of graph. Therefore, it is natural to model the grid $\{\mathbf{x}_i\}_{i=1}^N$ (see Section 3.3.4) as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with nodes $\mathbf{x}_i \in \mathcal{V}$, edges $e_{ij} \in \mathcal{E}$, and node features, including the input and output functions $\{v(\mathbf{x}_i)\}_{i=1}^N$ and $\{u(\mathbf{x}_i)\}_{i=1}^N$. Our goal is to learn the latent operator $\mathcal{G}$ in Equation (129) defined on the graph $\mathcal{G}$ in a data-driven manner.

Graph neural operators. Inspired by the format of Green's function (see Equation (122)), Li *et. al.* have introduced a graph kernel network into operator learning [185]. The model can be described as

$$z_0(\mathbf{x}) = \mathbf{P}(\mathbf{x}, v(\mathbf{x}), v_\epsilon(\mathbf{x}), \nabla v_\epsilon(\mathbf{x})) + p, \quad (142)$$

$$z_{t+1}(\mathbf{x}) = \sigma \left(\mathbf{W} z_t(\mathbf{x}) + \int_{\Omega} \kappa_\phi(\mathbf{x}, \mathbf{y}, v(\mathbf{x}), v(\mathbf{y})) z_t(\mathbf{y}) \nu_x(d\mathbf{y}) \right), \quad (143)$$

$$u(\mathbf{x}) = \mathbf{Q} z_T(\mathbf{x}) + q, \quad (144)$$

where $t = 0, \dots, T-1$ denote the indexes of hidden feature embeddings, $\mathbf{P} \in \mathbb{R}^{n \times 2(d+1)}$ (we assume $\mathbf{x} \in \mathbb{R}^d$ is a pure spatial coordinate), $p, z_0, \dots, z_T \in \mathbb{R}^n$, $\mathbf{Q} \in \mathbb{R}^{1 \times n}$, and $q \in \mathbb{R}$. In addition, σ is an activation function, ν_x is a fixed Borel measure for $\forall \mathbf{x} \in \Omega$, and v_ϵ is a Gaussian smoothed version of the input function v . The learnable weights include $\mathbf{W} \in \mathbb{R}^{n \times n}$ and the parameters ϕ of the kernel $\kappa_\phi: \mathbb{R}^{2(d+1)} \rightarrow$

$\mathbb{R}^{n \times n}$. With the help of message-passing architectures, we can approximate the integral in Equation (143) as,

$$z_{t+1}(\mathbf{x}_i) \approx \sigma \left(\mathbf{W} z_t(\mathbf{x}_i) + \frac{1}{|\mathcal{N}(i)|} \sum_{\mathbf{x}_j \in \mathcal{N}(i)} \kappa_\phi(\mathbf{x}_i, \mathbf{x}_j, v(\mathbf{x}_i), v(\mathbf{x}_j)) z_t(\mathbf{x}_j) \right), \quad (145)$$

where $\mathcal{N}(i)$ is the neighborhood of the node $\mathbf{x}_i$ and κ_ϕ is parameterized by a neural network. Here, it is noted that the input $\mathbf{x}_i$ is located on the graph but is not an arbitrary coordinate in the implementation.

Based on the above framework, Li *et al.* later proposed the multipole graph neural operator [186], which decomposes the kernel into multi-level sub-kernels to capture short-to-long-range interactions instead of only neighboring interactions (see Equation (145)) with linear complexity in the node numbers. Compared with the vanilla graph neural operator, such a model can better gather global information and thus has better generalizability. Recently, another study [187] combined the graph neural operator with autoregressive methods to facilitate learning operators defined by time-dependent PDEs. Further, an encoder-decoder mechanism was incorporated into the network architecture to boost the expression ability.

Graph neural operators are good at dealing with inputs of the format of the graph which can allow for unstructured discretization and model the interactions between nodes via edge features (which are very useful for some physical systems like multi-particle systems). However, we also notice that processing dense graphs can be computationally unfavorable for such models.

3.3.6 Open Challenges and Future Work

As previously mentioned, there has been fruitful work along with impressive success in this field of the neural operator. Moreover, some of these achievements have found their stages in the vast land of applications in science and engineering, which will be discussed later in Section 3.5. Still, the neural operator is a young and fast-growing field, where many open challenges remain to be solved. Some of them are noted as follows.

- **Incorporating physical priors.** Introducing physical priors (including fully or partially known governing equations, physical intuition, conservation, and symmetry, etc.) into training can effectively improve the generalizability of the model and reduce training data demands. Nowadays, the most popular method is to employ the framework of physics-informed learning (see Section 3.2.2). However, the speed and accuracy of physics-informed learning are far inferior to traditional numerical methods such as the FEM in solving PDEs [208]. Future work includes theoretically improving the framework of physics-informed learning or pursuing schemes that can effectively combine physical priors with data-driven approaches.
- **Reducing the cost of gathering datasets.** The basic goal of the neural operator is to learn a mapping from the parameter space to the solution space via data-driven methods. To this end, a considerable

number of samples are often required for a dataset. The samples are generated by numerical methods or experiments, both of which are very expensive, especially when the dimension of the parameter θ is high and the problem domain Ω is geometrically complex. This greatly limits the application of neural operators. How to deal with the expense of data generation will become one of the major challenges in the future.

- **Developing large pre-trained models.** At present, the main neural operator methods are all based on small models, and can only solve a specific type of parameterized physical system in one training. In the future, it may become a trend to develop pretrained large models that can be reused by various physical systems (after fine tuning), as in other machine learning fields such as computer vision and natural language processing. In this way, the model only needs to be trained once and can be employed in many other physical problems, reducing the time overhead of neural operator training and data generation.
- **Modeling real-world physical systems.** Many numerical testing experiments for neural operators are based on idealized physical systems which are far from real-world physical systems. Modeling real-world physical systems will bring more challenges, including geometrically complex problems, high-dimensional problems (such as optimal control problems [209]), chaotic problems (where the resulting solution is very sensitive to the initial conditions [210]), long-term prediction (which needs to deal with the accumulated error over time), etc. More research in this area will be the key to applying neural operators to practical problems.

3.4 Theory

In this subsection, we introduce some preliminary explorations of theoretical justification for physics-informed machine learning, especially for PINNs and DeepONet. First, we introduce the expression ability of neural networks, like DeepONet, for approximating operators. Then we present some work about the convergence of PINNs. Furthermore, we introduce some analyses of approximation and generalization errors. Finally, we mark the open challenges and future work in this field.

3.4.1 Expression Ability

In statistical machine learning, a hypothesis is a mapping of features onto labels, and the hypothesis set is the set of hypothesis [211]. Different algorithms will choose different hypothesis sets and hope to find the optimal hypothesis of the set. For example, the hypothesis set of linear regression is all linear mappings. In most machine learning tasks, our goal is to find a proper hypothesis in the hypothesis space through some algorithm. Since the hypothesis set is the subset of the set of all possible mappings and the optimal mapping may be not in the hypothesis, the expression ability of the hypothesis space determines the optimal hypothesis, of which the analysis is significant.

It is well known that multi-layer neural networks are universal approximators, i.e., they can approximate any

measurable function to arbitrary accuracy [212]. There is a similar and more interesting result for approximating operators, i.e., one-layer neural networks can approximate any operator, which is a mapping from a space of functions to another space of functions, to arbitrary accuracy [188]. Based on this result, current work [167] points out that DeepONet [167] with branch net as well as trunk net, can approximate any operator. We repeat this conclusion as follows:

Theorem 1 (Universal Approximation Theorem for DeepONet [167], [188]). *Suppose X is a Banach space, $K_1 \subseteq X, K_2 \subseteq \mathbb{R}^d, V \subseteq C(K_1)$ are compact sets, here $C(K_1)$ represents the set of all continuous functions in K_1 . Let $G : V \rightarrow C(K_2)$ be a nonlinear continuous operator, i.e., for any function $u \in V, G(u) \in C(K_2)$, then for $\forall y \in K_2 \subseteq \mathbb{R}^d, G(u)(y) \in \mathbb{R}$.*

Then for $\forall \epsilon > 0$, there $\exists n, p, m \in \mathbb{N}$, constants $c_i^k, \xi_{ij}^k, \theta_i^k, \zeta_k \in \mathbb{R}, w_k \in \mathbb{R}^d, x_j \in K_1, i = 1, \dots, n, k = 1, \dots, p, j = 1, \dots, m$, satisfying that

$$\left| G(u)(y) - \underbrace{\sum_{k=1}^p \sum_{i=1}^n c_i^k \sigma \left(\sum_{j=1}^m \xi_{ij}^k u(x_j) + \theta_i^k \right)}_{\text{branch}} \underbrace{\sigma(w_k y + \zeta_k)}_{\text{trunk}} \right| \leq \epsilon, \quad (146)$$

for $\forall u \in V$ and $y \in K_2$.

Theorem 1 indicates that the expressive ability of DeepONet is powerful enough to approximate any nonlinear continuous operator, which reveals the potential application of neural networks for learning operators. However, in Theorem 1, we not only prove the existence of such a neural network but also the size of this neural network, which is significant for designing specific networks, and is not fully understood.

Although these works [167], [188] have revealed the powerful expression ability of one-layer neural networks, some follow-up work [213] points that wide, shallow neural networks may need exponentially many neurons to obtain similar expression ability with deep, narrow ones. And we introduce their results as below

Theorem 2 ([213]). *Suppose X is a Banach space, $K_1 \subseteq X, V \subseteq C(K_1)$ are compact sets. Then, for $\forall n, k \in \mathbb{N}, n, k \geq 1, \exists G_k : V \rightarrow C([0, 1]^n)$ as a nonlinear continuous operator satisfying that*

- *There exists a ReLU neural network ϕ mapping from $[0, 1]^n$ to $\mathbb{R}$ with depth $2k^3 + 8$ and width $\Theta(1)$ satisfying that $\phi(y) = G_k(u)(y) \forall u \in V, y \in [0, 1]^n$.*
- *Let $m \geq 1$ be an integer and $\psi : [0, 1]^{m+n} \rightarrow \mathbb{R}$ be a ReLU neural network of which the depth $\leq k$ and the total nodes $\leq 2^k$. Then for any $x_1, \dots, x_m \in K_1, u \in V$, we have*

$$\int_{[0, 1]^d} |G_k(u)(y) - \psi(u(x_1), \dots, u(x_m), y)| dy \geq \frac{1}{64} \quad (147)$$

Theorem 2 conducts a nonlinear continuous operator, which can be approximated by a deep, narrow neural network with depth $2k^3 + 8$ and width $\Theta(1)$. However, there are no wide, shallow neural networks with depth $\leq k$ and total nodes $\leq 2^k$ that can effectively approximate it.

This result illustrates that deeper neural networks might be more efficient for approximating operators, to some degree. Furthermore, [213] provides an upper bound of the width of the deeper neural networks for approximating operators. The results are as follows

Theorem 3 ([213]). *Assume that the activation function σ satisfies some mild assumptions (details are in [213]), then for $\forall \epsilon > 0, \exists F : \mathbb{R}^{m+n} \rightarrow \mathbb{R}$ is a σ -activated neural network with width at most $m + n + 5$ satisfying*

$$|G(u)(y) - F(u(x_1), \dots, u(x_m), y)| < \epsilon, \quad (148)$$

for $\forall u, y$. Moreover, if we can construct a σ -activated neural network with width 3 and depth L to approximate the mapping $(a, b) \rightarrow ab$ to any error, then we can prove that the network F has depth $\mathcal{O}(M + N + L)$, here $M, N, m, \{x_i\}_{i=1}^m$ are the same as the notation of Theorem 5 in [188].

Theorem 3 provides a theoretical guarantee of deep neural networks for approximating operators with the upper bound of the width and the depth of the network.

Also, the expression ability of other architectures is worth studying and there are also some relevant conclusions; for example, [214] provides a universal approximation theorem for FNO [181].

3.4.2 Convergence

In statistical machine learning, whether an algorithm converges and its convergence speed are important indexes to evaluate the algorithm. For example, [215] analyzes the convergence property of stochastic gradient descent. Since the properties of physics equations are relatively complicated, the convergence properties of physics-informed machine learning algorithms have not been well studied.

[216] takes a first step to analyze the convergence of PINNs for solving time-independent PDEs, i.e.,

$$\begin{aligned} \mathcal{F}(u)(x) &= f(x), x \in \Omega \\ \mathcal{B}(u)(x) &= g(x), x \in \partial\Omega, \end{aligned} \quad (149)$$

where $\mathcal{F}$ is the differential operator and $\mathcal{B}$ is the boundary condition. We assume that these PDEs have a unique classical solution u and hope to approximate the solution of the PDEs with a set of training data, including residual data and initial/boundary data. Moreover, we denote the number of training set as the vector $\mathbf{m} = (m_r, m_b)$, where m_r, m_b are the number of training points of residual data and initial/boundary data respectively. We assume that $\{(\mathbf{x}_r^i, f(\mathbf{x}_r^i))\}_{i=1}^{m_r}$ is the set of the residual data and $\{(\mathbf{x}_b^i, f(\mathbf{x}_b^i))\}_{i=1}^{m_b}$ is the set of the initial/boundary data, here $\mathbf{x}_r^i \in \Omega, i = 1, 2, \dots, m_r; \mathbf{x}_b^j \in \partial\Omega, j = 1, 2, \dots, m_b$. Given a set of neural networks $\mathcal{H}_n$ as our hypothesis set, then for any mapping $h \in \mathcal{H}_n$, we can define its empirical PINN loss via $\{(\mathbf{x}_r^i, f(\mathbf{x}_r^i))\}_{i=1}^{m_r}$ and $\{(\mathbf{x}_b^i, f(\mathbf{x}_b^i))\}_{i=1}^{m_b}$ as

$$\begin{aligned} \text{Loss}_m(h; \boldsymbol{\lambda}, \boldsymbol{\lambda}^R) &= (\lambda_r \|\mathcal{F}(h)(\mathbf{x}_r) - f(\mathbf{x}_r)\|^2) \mathbb{I}_\Omega(\mathbf{x}_r) + \lambda_r^R R_r(h) \\ &+ (\lambda_b \|\mathcal{B}(h)(\mathbf{x}_b) - g(\mathbf{x}_b)\|^2) \mathbb{I}_{\partial\Omega}(\mathbf{x}_b) + \lambda_b^R R_b(h), \end{aligned} \quad (150)$$

here $\mathbb{I}$ is the indicated function, R_r, R_b are regularization functions, and $\boldsymbol{\lambda} = (\lambda_r, \lambda_b), \boldsymbol{\lambda}^R = (\lambda_r^R, \lambda_b^R)$ are hyperparameters. Moreover, we can define the expected PINN loss as

$$\text{Loss}(h; \boldsymbol{\lambda}, \boldsymbol{\lambda}^R) = \mathbb{E}[\text{Loss}_m(h; \boldsymbol{\lambda}, \boldsymbol{\lambda}^R)]. \quad (151)$$

Our goal is to minimize the expected PINN loss in the hypothesis set, i.e.,

$$\min_{h \in \mathcal{H}_n} \text{Loss}(h; \lambda, \lambda^R). \quad (152)$$

However, in practice, it is difficult to calculate the expected PINN loss and we always use $\text{Loss}_m(h; \lambda, \lambda^R)$ as an approximator, i.e.,

$$\min_{h \in \mathcal{H}_n} \text{Loss}_m(h; \lambda, \lambda^R). \quad (153)$$

Consequently, it is significant to analyze the difference between it and the true solution.

Based on (150), if we take $\lambda^R = 0$, we can get the standard empirical PINN loss and the standard expected PINN loss as

$$\begin{aligned} \text{Loss}_m^{\text{PINN}}(h; \lambda) &= \frac{\lambda_r}{m_r} \sum_{i=1}^{m_r} \|\mathcal{F}(h)(\mathbf{x}_r^i) - f(\mathbf{x}_r^i)\|^2 \\ &\quad + \frac{\lambda_b}{m_b} \sum_{i=1}^{m_b} \|\mathcal{B}(h)(\mathbf{x}_b^i) - g(\mathbf{x}_b^i)\|^2, \\ \text{Loss}^{\text{PINN}}(h; \lambda) &= \lambda_r \|\mathcal{F}(h) - f\|_{L^2}^2 + \lambda_b \|\mathcal{B}(h) - g\|_{L^2}^2. \end{aligned} \quad (154)$$

First, [216] utilizes the regularized empirical loss for bounding the expected PINN loss. The result is shown below:

Theorem 4 ([216]). *Under some assumptions (details are in [216]), we set m_r and m_b be the number of points sampled from Ω and $\partial\Omega$ respectively.*

If $d \geq 2$, with probability at least $(1 - \sqrt{m_r}(1 - 1/\sqrt{m_r})^{m_r})(1 - \sqrt{m_b}(1 - 1/\sqrt{m_b})^{m_b})$, we can prove that

$$\text{Loss}^{\text{PINN}}(h; \lambda) \leq C_m \text{Loss}_m(h; \lambda, \hat{\lambda}_m^R) + C'(m_r^{-\frac{\alpha}{d}} + m_b^{-\frac{\alpha}{d-1}}), \quad (155)$$

where C_m and C' are constants.

If $d = 1$, with probability at least $1 - \sqrt{m_r}(1 - 1/\sqrt{m_r})^{m_r}$, we can prove that

$$\text{Loss}^{\text{PINN}}(h; \lambda) \leq C_m \text{Loss}_m(h; \lambda, \hat{\lambda}_m^R) + C'm_r^{-\frac{\alpha}{d}}, \quad (156)$$

here C_m and C' are constants.

Furthermore, by setting the regular term $R_r(h) = \mathcal{F}[h]$, $R_b(h) = \mathcal{B}[h]$, we can define the Hölder regularized empirical PINN loss as

$$\begin{aligned} \text{Loss}_m(h; \lambda, \lambda^R) &= \begin{cases} \text{Loss}_m^{\text{PINN}}(h; \lambda) + \lambda_{r,m}^R [\mathcal{F}[h]]_{\alpha,U}^2 + \lambda_{b,m}^R [\mathcal{B}[h]]_{\alpha,\Gamma}^2, & \text{if } d \geq 2 \\ \text{Loss}_m^{\text{PINN}}(h; \lambda) + \lambda_{r,m}^R [\mathcal{F}[h]]_{\alpha,U}^2, & \text{if } d = 1 \end{cases} \end{aligned} \quad (157)$$

Based on Theorem 4, [216] minimizes a high-probability upper bound of the expected PINN loss by minimizing the Hölder regularized loss. Moreover, [216] further shows that the minimum of Hölder regularized empirical loss has a small expected PINN loss and will converge to the ground truth, i.e.,

Theorem 5 ([216]). *Under some assumptions (details are in [216]), we set m_r and m_b as the number of points sampled from Ω and $\partial\Omega$ respectively and set*

$$h_{m_r} = \arg \min_{h \in \mathcal{H}_{m_r}} \text{Loss}_{m_r}(h; \lambda, \lambda^R) \quad (158)$$

Then we have following results

- With probability at least $(1 - \sqrt{m_r}(1 - c_r/\sqrt{m_r})^{m_r})(1 - \sqrt{m_b}(1 - c_b/\sqrt{m_b})^{m_b})$, we can prove that

$$\text{Loss}^{\text{PINN}}(h_{m_r}; \lambda) = \mathcal{O}(m_r^{-\frac{\alpha}{d}}) \quad (159)$$

- With probability 1, we can prove that

$$\lim_{m_r \rightarrow \infty} \mathcal{F}[h_{m_r}] = f, \quad \lim_{m_r \rightarrow \infty} \mathcal{B}[h_{m_r}] = g. \quad (160)$$

Theorem 5 propose a general convergence analyses for PINNs under some conditions. Moreover, [216] provides more detailed conclusions for linear elliptic PDEs and linear parabolic PDEs.

Besides PINNs, there is other work focusing on the convergence of other physics-informed machine learning methods— for example, [217] provides uniform convergence guarantees for the Deep Ritz Method for Nonlinear Problems and [218] provides convergence rates for learning linear operators from noisy data.

3.4.3 Error Estimation

In statistical machine learning, error estimation is significant for analyzing high or low performance of the model and guiding us to design better algorithms. Also, the error estimation of physics-informed machine learning is relatively preliminary and there is only a little research on this topic.

A recent work [219] analyzes the approximation and generalization errors of DeepONet [167]. First, [219] provides a more general form of DeepONet. Set $D \subseteq \mathbb{R}^d$, $U \subseteq \mathbb{R}^n$ are two compact sets. We consider three operators for building DeepONet:

- **Encoder.** Given a set of points $\{x_i\}_{i=1}^m, x_i \in D$, [219] first defines the Encoder $\mathcal{E}$ as

$$\mathcal{E} : C(D) \rightarrow \mathbb{R}^m, \quad \mathcal{E}(u) = (u(x_1), \dots, u(x_m)), \quad (161)$$

- **Approximator.** Under the same set of points $\{x_i\}_{i=1}^m$, [219] defines the approximator $\mathcal{A}$ as a neural network satisfying that

$$\mathcal{A} : \mathbb{R}^m \rightarrow \mathbb{R}^p, \quad \{u_j\}_{j=1}^m \rightarrow \{\mathcal{A}_k\}_{k=1}^p. \quad (162)$$

- **Reconstructor.** To define the Reconstructor $\mathcal{R}$, [219] first uses a neural network τ to denote a trunk net as

$$\tau : \mathbb{R}^n \rightarrow \mathbb{R}^{p+1}, \quad y = (y_1, \dots, y_n) \rightarrow \{\tau_k(y)\}_{k=0}^p \quad (163)$$

Then, based on τ , [219] defines the Reconstructor $\mathcal{R}$ as

$$\begin{aligned} \mathcal{R} &= \mathcal{R}_\tau : \mathbb{R}^p \rightarrow C(U), \\ \mathcal{R}_\tau(\{\mathcal{A}_k\}_{k=1}^p) &= \tau_0 + \sum_{k=1}^p \mathcal{A}_k \tau_k \in C(U), \\ \mathcal{R}_\tau(\{\mathcal{A}_k\}_{k=1}^p)(y) &= \tau_0(y) + \sum_{k=1}^p \mathcal{A}_k \tau_k(y), \forall y \in U. \end{aligned} \quad (164)$$

Thus, [219] can combine them to get the DeepONet $\mathcal{N}$ as

$$\mathcal{N} : C(D) \rightarrow C(U), \quad \mathcal{N}(u) = (\mathcal{R} \circ \mathcal{A} \circ \mathcal{E})(u), \quad (165)$$

where $\beta = \mathcal{A} \circ \mathcal{E}$ is the branch net. For $\forall y \in U$, we have

$$\begin{aligned}\mathcal{N}(u)(y) &= (\mathcal{R} \circ \mathcal{A} \circ \mathcal{E})(u)(y) \\ &= \tau_0(y) + \sum_{k=1}^p \mathcal{A}_k \tau_k(y).\end{aligned}\quad (166)$$

First, [219] analyzes the approximation error of DeepONet. Assume that the underlying operator is $\mathcal{G} : C(D) \rightarrow C(U)$ and there is a fixed probability measure $\mu \in \mathcal{P}(X)$, we can naturally use the $L^2(\mu)$ -norm of $\mathcal{G}$ and our DeepONet $\mathcal{N}$ to measure the approximation error of $\mathcal{N}$ as

$$\hat{\mathcal{E}} = \left(\int_X \int_U |\mathcal{G}(u)(y) - \mathcal{N}(u)(y)|^2 dy d\mu(u) \right)^{1/2}. \quad (167)$$

However, since $\mathcal{N}$ consists of different operators, it is challenging to directly analyze $\hat{\mathcal{E}}$. To handle this problem, the main result of [219] is to decompose $\hat{\mathcal{E}}$ into three parts: encoding error $\hat{\mathcal{E}}_{\mathcal{E}}$, approximation error $\hat{\mathcal{E}}_{\mathcal{A}}$ and reconstruction error $\hat{\mathcal{E}}_{\mathcal{R}}$.

To better analyze the encoder $\mathcal{E}$ and the reconstructor $\mathcal{R}$, we define the decoder $\mathcal{D}$ and the projector $\mathcal{P}$ satisfying:

$$\begin{aligned}\mathcal{E} \circ \mathcal{D} &= \text{Id} : \mathbb{R}^m \rightarrow \mathbb{R}^m, \\ \mathcal{D} \circ \mathcal{E} &\approx \text{Id} : X \rightarrow X, \\ \mathcal{P} \circ \mathcal{R} &= \text{Id} : \mathbb{R}^p \rightarrow \mathbb{R}^p, \\ \mathcal{R} \circ \mathcal{P} &\approx \text{Id} : Y \rightarrow Y.\end{aligned}\quad (168)$$

Then, we can define the encoding error $\hat{\mathcal{E}}_{\mathcal{E}}$, the approximation error $\hat{\mathcal{E}}_{\mathcal{A}}$ and the reconstruction error $\hat{\mathcal{E}}_{\mathcal{R}}$ respectively as below

$$\begin{aligned}\hat{\mathcal{E}}_{\mathcal{E}} &= \left(\int_X \|\mathcal{D} \circ \mathcal{E}(u) - u\|_X^2 d\mu(u) \right)^{1/2}, \\ \hat{\mathcal{E}}_{\mathcal{A}} &= \left(\int_{\mathbb{R}^m} \|\mathcal{A}(u) - \mathcal{P} \circ \mathcal{G} \circ \mathcal{D}(u)\|_{L^2(\mathbb{R}^p)}^2 d(\mathcal{E}_{\#}\mu)(u) \right)^{1/2}, \\ \hat{\mathcal{E}}_{\mathcal{R}} &= \left(\int_{\mathbb{R}^m} \|\mathcal{R} \circ \mathcal{P}(u) - u\|_{L^2(U)}^2 d(\mathcal{G}_{\#}\mu)(u) \right)^{1/2}.\end{aligned}\quad (169)$$

Furthermore, [219] decomposes $\hat{\mathcal{E}}$ as below

Theorem 6 ([219]). *Under some mild assumptions (details in [219]), the error (167) can be bounded by*

$$\hat{\mathcal{E}} \leq \text{Lip}_{\alpha}(\mathcal{G}) \text{Lip}(\mathcal{R} \circ \mathcal{P})(\hat{\mathcal{E}}_{\mathcal{E}})^{\alpha} + \text{Lip}(\mathcal{R})\hat{\mathcal{E}}_{\mathcal{A}} + \hat{\mathcal{E}}_{\mathcal{R}}, \quad (170)$$

here $\mathcal{G}$ is α -Hölder continuous and $\text{Lip}_{\alpha}, \text{Lip}$ is defined for any mapping $\mathcal{F} : \mathcal{X} \rightarrow \mathcal{Y}$ between Banach spaces $\mathcal{X}, \mathcal{Y}$:

$$\begin{aligned}\text{Lip}_{\alpha}(\mathcal{F}) &= \sup_{u, u' \in \mathcal{X}} \frac{\|\mathcal{F}(u) - \mathcal{F}(u')\|_{\mathcal{Y}}}{\|u - u'\|_{\mathcal{X}}^{\alpha}}, \\ \text{Lip}(\mathcal{F}) &= \text{Lip}_1(\mathcal{F}).\end{aligned}\quad (171)$$

Based on Theorem 6, [219] further provides more detailed analyses for bounding $\hat{\mathcal{E}}_{\mathcal{E}}, \hat{\mathcal{E}}_{\mathcal{A}}, \hat{\mathcal{E}}_{\mathcal{R}}$, which are helpful for bounding and analyzing $\hat{\mathcal{E}}$.

Besides approximation error, [219] also provides analysis of the generalization error of DeepONet. Given the underlying operator $\mathcal{G}$, we hope to train the DeepONet $\mathcal{N}$ that minimizes the loss function

$$\hat{\mathcal{L}}(\mathcal{N}) = \int_{L^2(D)} \int_U |\mathcal{G}(u)(y) - \mathcal{N}(u)(y)|^2 dy d\mu(u). \quad (172)$$

However, we can not directly calculate $\hat{\mathcal{L}}$ and we always use empirical loss as a surrogate. To approximate it, we always sample

$$U_1, U_2, \dots, U_N \sim \mu, Y_1, Y_2, \dots, Y_N \sim \text{Unif}(U), \quad (173)$$

and define the empirical loss as

$$\hat{\mathcal{L}}_N(\mathcal{N}) = \frac{|U|}{N} \sum_{j=1}^N |\mathcal{G}(U_j)(Y_j) - \mathcal{N}(U_j)(Y_j)|^2. \quad (174)$$

Assume that $\hat{\mathcal{N}}, \hat{\mathcal{N}}_N$ be the minimizer of the loss (172) and the loss (174) respectively, i.e.

$$\begin{aligned}\hat{\mathcal{N}} &= \arg \min_{\mathcal{N}} \hat{\mathcal{L}}(\mathcal{N}), \\ \hat{\mathcal{N}}_N &= \arg \min_{\mathcal{N}} \hat{\mathcal{L}}_N(\mathcal{N}),\end{aligned}\quad (175)$$

we can define the generalization error as

$$(\hat{\mathcal{E}}_{\text{gen}})^2 = \hat{\mathcal{L}}(\hat{\mathcal{N}}_N) - \hat{\mathcal{L}}(\hat{\mathcal{N}}). \quad (176)$$

Moreover, [219] provides Theorem 7 as below to bound the generalization error.

Theorem 7 ([219]). *Under some assumptions (details are in [219]), we can bound the generalization error as*

$$\mathbb{E} \left[\left| \hat{\mathcal{L}}(\hat{\mathcal{N}}_N) - \hat{\mathcal{L}}(\hat{\mathcal{N}}) \right| \right] \leq \frac{C}{\sqrt{N}} (1 + Cd_{\theta} \log(CB\sqrt{N})^{2\kappa+1/2}), \quad (177)$$

here C, d_{θ}, B, κ are constants.

Besides this work analyzing the error estimation of DeepONet, there is also some work focus on estimating the error of other architectures. For example, [220] estimates the error of PINN for Linear Kolmogorov PDEs, and [214] analyzes the error of FNO.

3.4.4 Open Challenges and Future Work

As previously mentioned, although physics-informed machine learning has received more and more attention, and although some representative algorithms, like PINNs and DeepONet, have shown encouraging performance, their theoretical properties have not been well explored. Since theoretical justification, including expression ability, convergence, and error estimation, is significant for guiding to design better algorithms, these fields are of great value in future research. Besides current progress as mentioned above, we mark some open problems of theoretical justification for physics-informed machine learning as follows,

- **Expression Ability** The expression ability of neural networks for approximating operators has made great progress, but there are still some challenges and open problems. First, although some work [213] has discussed the expression ability of deep, narrow neural networks and wide, shallow networks, for approximating operators, why deep networks have better expression ability has not been well studied. Moreover, how to design more effective architecture to approximate operators with fewer nodes is significant for designing more stable and effective algorithms. Besides DeepONets, the expression ability

of other architectures is also worth more in-depth analysis.

- **Convergence** The convergence of physics-informed machine learning algorithms is significant to evaluate their effectiveness. Unfortunately, the current research in this field is still very preliminary since analyzing the stability of PDEs itself is complicated. How to analyze the convergence of PINNs for different kinds of PDEs will be one of the major challenges in the future and can inspire us to design more efficient architectures and algorithms. Moreover, the convergence of other algorithms like DeepONets needs further exploration
- **Error Estimation** At present, some studies have preliminarily considered the errors of physics-informed machine learning algorithms, like DeepONet and FNO. There are two kinds of error estimation that have attracted more and more attention, i.e., approximation error and generalization error. Carefully analyzing approximation error is beneficial for designing more effective algorithms. Moreover, analyzing generalization error and improving the generalization of algorithms by using physics knowledge are also noteworthy directions for developing more general and stable algorithms. More research in this field will be the key to better understanding and combining physics knowledge and data.

3.5 Application

Physics-informed machine learning is playing a more and more important role in various fields and solving some problems that cannot be accomplished by traditional methods. In this section, we briefly introduce some important applications of PIML in several fields, including fluid dynamics, material science, optimal control, and scientific discovery.

3.5.1 Fluid Dynamics

Fluids are one of the most difficult physical systems due to the high nonlinearity and mathematical complexity of the governing equations. An example is the Navier-Stokes equation, where chaos may occur under certain conditions. Therefore, methods of physical information learning have been introduced in this field to solve many problems that are difficult for traditional methods. Applications mainly include predicting fluid dynamics (non-newtonian fluids [221], high-speed flows [195], multiscale flows [222], multiphase flows [223], and multiscale bubble dynamics [224]) with/without data, simulating turbulence [225], [226], design problems in the context of fluids [227], and reconstructing high-precision flow data (super-resolution) [228], [229].

For instance, [223] proposed U-FNO for solving parametric multiphase flow problems. U-FNO is designed based on the Fourier neural operator (FNO) and incorporates a U-Net structure to improve the representation ability in high-frequency information. Another study, [230], reviewed physics-informed methods in fluid mechanics for seamlessly integrating data and showed the effectiveness of physics-informed neural networks (PINNs) in the inverse problems related to simulating several types of flows.

3.5.2 Material Science

In material science, researchers utilize physics-informed deep learning methods such as PINNs to model the optical, electrical, and mechanical properties of materials (nonhomogeneous materials [231], metamaterials [232], and elastic-viscoplastic materials [233]), as well as specific structure (e.g., surface cracks [234], fractures [235], defects [236], etc.) under the influence of external force or temperature.

For example, [234] identified and characterized the surface-breaking cracks in a given metal plate by estimating the speed of sound inside with a PINN, which combined physical laws with the effective permittivity parameters of finite-size scattering systems comprised of many interacting multi-component nanoparticles and nanostructures. This approach can facilitate the designing new metamaterials with nanostructures. Another paper, [237], focused on extracting elastoplastic properties of materials such as metals and alloys from instrumented indentation data using a multi-fidelity deep learning method.

3.5.3 Other Fields

In addition to the above, physics-informed deep learning methods have important applications in many other fields, including heat transfer [238], [239], [240], waves [241], [242], [243], nuclear physics [244], [245], traffic [246], electricity & magnetism [247], [248], [249], and the following fields.

Medicine. Physics-informed methods are used to model physical processes in the human system (e.g., blood flow [250], drug assimilation [251]), or the dynamics of a disease [252]), and other relevant physical systems such as diagnostic ultrasound [253]. For example, [254] analyzed a number of epidemiological models through the lens of PINNs in the context of the spread of COVID-19. This paper studied the simulated results with realistic data and reported possible control measures. Graph neural networks based methods are used for molecular property prediction [255], [256], [257], [258] and molecular discovery [259], [260], [261], [262], [263], [264].

Geography. A new line of work has attempted to apply PINNs in several topics of geography, including climate [8], [265], geology [266], seismology [267], and pollution [268]. For instance, [269] evaluated groundwater contamination from unconventional oil and gas development; the predictions brought many critical insights into the prevalence of contamination based on historical data.

Industry. Physics-informed deep learning methods have also emerged as powerful tools in the industry. Examples include applying physics-informed methods in solving civil engineering problems [270], processing composites in smart manufacturing [271], and modeling metal additive manufacturing processes [272].

4 INVERSE PROBLEM

In addition to using neural networks as a surrogate model for simulating physical systems, there is another important and challenging task: to optimize or discover unknown parameters of a physical system. This problem is also called inverse problems (e.g. inverse design), and is widely used in many fields such as engineering [273], [274], [275], design [276], [277], fluid dynamics [278], etc. Since

inverse problem involves numerous scenarios and sub-problems [279], [280], [281], [282], we take inverse design, which is crucial in both academic research and industrial application, as a representative example and review methods that incorporate machine learning algorithms, especially neural networks in this section. We first formalize the problem of inverse design and introduce the basic concepts, traditional methods, and challenges in Section 4.1. Considering that the solution of inverse design usually involves multiple steps, such as the simulation of the physical system or process, the evaluation of the performance, and the representation of the configuration, we present methods according to their roles in the task of inverse design. Neural surrogate modeling of the physical system has received widespread attention and related research is introduced in Section 4.2. Methods that focus on other parts of inverse design are introduced in Section 4.3. We further review methods for more general inverse problems beyond inverse design in Section 4.4. Finally, in Section 4.5 we discuss the remaining challenges and future work in this field.

4.1 Problem Formulation

Generally speaking, in an inverse design task, an optimal configuration of a physical system is sought to achieve the desired performance, while some given constraints, usually associated with physical properties, are satisfied. For example, both the shape optimization of the airfoil to minimize drag during flight, and the heater placement in an office to manage the temperature, are typical examples of inverse design. Considering that we have a collected dataset of physical systems with different parameters $\mathcal{D} = \{u(\mathbf{x}_i; \theta_j)\}_{1 \leq i \leq N_1, 1 \leq j \leq N_2}$, the problem can be formalized as

$$\begin{aligned} \min_{\theta \in \Theta} \mathcal{J}(u(\mathbf{x}; \theta), \theta), \\ \text{s.t. } \mathcal{P}(u; \theta)(\mathbf{x}) = 0. \end{aligned} \quad (178)$$

Here, $u(\mathbf{x}; \theta)$ are the state variables and $\mathcal{J}$ is the design objective of the physical system configured by parameters θ , where the physical process $\mathcal{P}(u; \theta)(\mathbf{x}) = 0$ represents a group of PDEs, or even constraints in other forms, e.g., explicit or implicit functions. Since inverse design is common in various complex scenarios, different problems can be formalized as either PDE-constrained optimization or, more generally, constrained optimization, depending on the form of $\mathcal{P}$. Note that, when we optimize θ , the solution of PDEs $u(\mathbf{x}; \theta)$ at given parameters θ is unknown and needs to be solved using a traditional numerical solver or neural network surrogates. Here, the mathematical formulation of the inverse design is consistent with general inverse problems. If the design parameters θ are (part of) the parameters of the physical systems, estimating optimal θ equals identifying system parameters. If θ denotes the control parameters or design parameters, then the formulation can be used for solving PDE Constrained Optimization (PDECO) problems, such as structural optimization or optimal control of PDEs.

Researchers used to adopt numerical methods to solve the inverse design formalized as an optimization problem, especially PDE-constrained optimization. Traditional methods can mainly be categorized as an *all-at-once* approach and a *black-box* approach [283]. *All-at-once* approaches, such

as sequential quadratic programming (SQP) [284], optimize the state variables and parameters simultaneously, treating them independently, which only requires the PDE constraints to be satisfied at the end of optimization. However, when it comes to large-scale problems, *all-at-once* methods become impractical. *Black-box* methods, including first-order methods (e.g., gradient descent) and higher-order methods (e.g., Newton methods), use iterative schemes with repetitive evaluation of gradients $\frac{\partial \mathcal{J}}{\partial \theta}$. The adjoint method is most commonly employed to calculate the gradients. However, it requires costly solutions of the original PDEs and the adjoint PDEs with numerical solvers like FEM in every round of optimization. A rough estimate of the computational complexity of FEM can be $\mathcal{O}(dn^r)$, where n is the dimension of state variables, r is about 3 for a simple solver, $d = 1$ for a linear system and $d > 1$ for a nonlinear system [52]. The high expense means that the optimization demands a large amount of computational resources with poor efficiency. Meanwhile, a large number of parameters to be optimized, which leads to higher degrees of freedom, could bring intractable complexity. Parameters in the form of continuous functions (e.g., the source function in a physical system) that are not finite-dimensional vectors also impose difficulties for the existing methods. For general constrained optimization problems, more challenges arise in some cases, including but not limited to non-differentiable physical process and lack of uniqueness [285].

Based on the introduction above, problems of inverse design, i.e., identification or controlling physical systems, have been fundamental challenges because of their difficulty, low efficiency and multimodality. It is a promising direction that AI can help accelerate or improve existing methods of inverse design by introducing physics-informed machine learning paradigms.

4.2 Neural Surrogate Models

As mentioned above, the simulation of the physical system and the evaluation of the objective function often make use of traditional numerical methods like FEM, and the computational complexity and the demand for computing resources can be huge. As the scale and dimension of the system increase, the cost of numerical methods during optimization may become unacceptable. To accelerate the process, there has been interest in performing the optimization based on surrogate models instead of numerical solvers. Besides using machine learning algorithms such as random forests and Gaussian processes, more and more researchers are leveraging neural networks to model a physical system where an inverse design is conducted. The advantages of neural networks to approximate any measurable function, to handle high-dimensional and nonlinear problems, and to interpolate and extrapolate across the data contribute to its usage in the task of inverse design. Several typical paradigms of neural surrogate models are introduced below.

With PINNs. PINN [30] proposes to model the constraints of PDE system by minimizing physics-informed loss. One superiority of PINN is that it can successfully address inverse problems [232], [278], [286], [287], which are special cases of PDE-constrained optimization. Some research has also looked into solving inverse design or PDE-

constrained optimization with PINNs describing a physics system.

Considering that PINN seamlessly introduces physical constraints to a neural network by incorporating the physics-informed loss, Mowlavi and Nabi [288] extend the original PINN to problems like optimal control. With two neural networks representing the solution u_w and the control parameters θ_v , where $w \in W$ and $v \in V$, the inverse design can be solved with an augmented loss function as

$$\begin{aligned} \mathcal{L} = & \frac{\lambda_r}{N_r} \sum_{i=1}^{N_r} \|\mathcal{F}(u_w; \theta_v)(\mathbf{x}_i)\|^2 + \frac{\lambda_i}{N_i} \sum_{i=1}^{N_i} \|\mathcal{I}(u_w; \theta_v)(\mathbf{x}_i)\|^2 \\ & + \frac{\lambda_b}{N_b} \sum_{i=1}^{N_b} \|\mathcal{B}(u_w; \theta_v)(\mathbf{x}_i)\|^2 + \lambda_{\mathcal{J}} \mathcal{J}(u_w(\mathbf{x}), \theta_v). \end{aligned} \quad (179)$$

Here, this method simply adds the objective function of the inverse design to the standard PINN loss terms with λ . as scalar weights. To tune a series of hyperparameters, they propose a guideline for optimal control, which is categorized into validation (to ensure that the learned solution u_w^* satisfies the PDE constraints) and evaluation (to accurately evaluate the performance of the optimized parameters θ_v^*). This PINN-based method is compared with direct-adjoint-looping (DAL). Optimal control results in four physical systems, including Laplace, Burgers, Kuramoto-Sivashinsky, and Navier-Stokes equations, prove the capability of PINN in inverse design. Although this work mainly focuses on examining the feasibility of the original PINN in tasks of inverse design, the authors mention that the issue of balancing different objectives when training PINNs also exists for this problem.

To address the challenges due to the multiple loss terms, among which the PDE loss and the objective function are often not consistent, Lu *et al.* [141] propose PINN with hard constraints (hPINN) for inverse design, especially topology optimization. Unlike a soft-constraint method that directly minimizes the sum of PDE loss and the objective function, they take the equality and inequality constraints as hard constraints with the penalty method and the augmented Lagrangian method. The optimization objectives of these three methods are listed in order as

$$\mathcal{L} = \mathcal{J} + \mu_{\mathcal{F}} \mathcal{L}_{\mathcal{F}} + \mu_h \mathcal{L}_h, \quad (180)$$

$$\mathcal{L}^k = \mathcal{J} + \mu_{\mathcal{F}}^k \mathcal{L}_{\mathcal{F}} + \mu_h^k \mathbb{I}_{h>0} h^2, \quad (181)$$

$$\begin{aligned} \mathcal{L}^k = & \mathcal{J} + \mu_{\mathcal{F}}^k \mathcal{L}_{\mathcal{F}} + \mu_h^k \mathbb{I}_{h>0 \text{ or } \lambda_h^k > 0} h^2 \\ & + \frac{1}{MN} \sum_{j=1}^M \sum_{i=1}^N \lambda_{i,j}^k \mathcal{F}_i(u; \theta)(\mathbf{x}_j) + \lambda_h^k h, \end{aligned} \quad (182)$$

where μ . are coefficients, λ^k are Lagrangian multipliers, h represents hard constraints, and k denotes the k -th iteration, because the penalty method and the augmented Lagrangian method transform the constrained problem into a sequence of unconstrained problems. Also, they present novel network architecture to strictly enforce the boundary conditions. The method is demonstrated with experiments on holography in optics and topology optimization in fluids.

Another study [289] proposes to adopt a bi-level optimization framework to handle the conflict PDE losses

and objective loss. It adopts the following mathematical formulation:

$$\begin{aligned} \min_{\theta} \quad & \mathcal{J}(w^*, \theta) \\ \text{s.t.} \quad & w^* = \arg \min_w \mathcal{L}_{PINN}(w, \theta). \end{aligned} \quad (183)$$

It uses PINNs to solve PDE with current control variables in the inner loop and updates the control variables using hypergradients in the outer loop. The hypergradients are computed using Broyden's method based on implicit function differentiation [290]. This method naturally links the traditional adjoint method for solving PDECO with prevailing methods based on PINNs, which has a large scope for exploration.

Other work makes use of PINNs to handle inverse design differently. [291] proposes Control PINN for optimal control. The network is designed to generate a triple of the system solution: the control parameters and the adjoint system state, together with spatial and temporal coordinates as input. The first order optimality condition of the Lagrangian is introduced to the standard PINN loss to perform the optimization under the constraints in a one-stage manner. [292] proposes Physics-Informed Neural Nets for Control (PINN), which modifies the network to output solutions based on initial states and control parameters, making it possible to make long-term predictions and suitable for control tasks. The model is combined with model-based predictive control (MPC) as a surrogate solver to perform control applications.

Physics-informed algorithms incorporate physical knowledge (PDEs) into the neural networks as soft constraints in loss terms. One strength is their flexibility and ease of implementation. This allows them to be extended to inverse design, by either optimizing the objective function along with PDE losses or using trained PINNs as surrogate solvers. However, the challenges concerning convergence when training PINNs can be pathological due to the imbalance among multiple loss terms, which may also exist for inverse design based on PINNs. Strategies like adaptive loss re-weighting [54], [55] have the potential to improve the performance of optimization. Leveraging the advantages of traditional numerical methods for PINN methods may also ease the problems.

With Neural Operators. As introduced in Section 3.3, neural operators learn a mapping $G : \Theta \times \Omega \rightarrow \mathbb{R}^m$, which is a mapping from the input parameters/functions to the solution function under the physical constraints and can be queried for state variables at any arbitrary point in the spatio-temporal domain. This replaces the numerical PDE solvers or expensive physics simulators that are repeatedly called on during optimization.

Amortized Finite Element Analysis (AmorFEA) is developed by Xue *et al.* [52]. It is inspired by the idea of amortized optimization in amortized variational inference. Its purpose is to predict PDE solutions with a neural network, based on the Galerkin minimization formulation of PDE. The neural network is trained to minimize the expected potential energy over the parameter space, from which the PDE can be derived. With the trained surrogate model, gradient-based optimization can be performed with only one forward and

backward pass through the network to compute the gradients w.r.t. the control parameters. The authors conducted experiments on source control of a Poisson equation and inverse kinematics of a soft robot. The fact that not all PDE can be derived from potential energy limits the scope of applications in different physics systems.

Wang *et al.* [293] use physics-informed DeepONets (see Section 3.3.2) as a surrogate to achieve fast PDE-constrained optimization via gradient-based methods, even without paired training data. The self-supervised mechanism with physics-informed losses as soft constraints enables effective training with no need to call on numerical solvers. A two-stage framework is proposed to perform the optimization. It first trains the surrogate model for the given physical system and then minimizes the cost function w.r.t. the input functions, which can be described as

$$w^* = \arg \min_{w \in W} \mathcal{L}_{PINO}, \quad (184)$$

$$\min_{\theta} \mathcal{J}(G_{w^*}(\theta)(\mathbf{x}), \theta), \quad (185)$$

where $\mathcal{L}_{PINO}$ is defined in Equation 93. An additional neural network with learnable weight α is used to parameterize the input functions. Experimental results on optimal control of Poisson and heat equation and drag minimization of obstacles in a Stokes-flow demonstrate the capability of physics-informed DeepONets to perform PDE-constrained optimization. Compared with numerical solver and adjoint methods, the computational cost of their proposed method is much lower. However, the ability of physics-informed DeepONets to accurately solve complex PDE systems is the bottleneck in terms of extending this method to more complex scenarios.

A two-stage framework similar to that used by Wang *et al.* is proposed by Hwang *et al.* [294]. The first phase is to learn the solution operator under the PDE constraints with an autoencoder model. The decoder has two branches, one for solution $G_w(\theta) = (G_w^{sol} \circ G_w^{enc})(\theta)$ and one for reconstruction $\tilde{\theta}_w(\theta) = (G_w^{rec} \circ G_w^{enc})(\theta)$. The surrogate model can be trained in either a data-driven or data-free way. The second phase is to minimize the objective function based on the trained neural operator. To avoid the optimal parameters being outside the training domain, the reconstruction branch of the encoder is used to regularize the optimization. The whole framework can be described as

$$\mathcal{L}_{sup} = \frac{1}{N} \sum_{k=1}^N L(G_w(\theta_k)(\mathbf{x}), u_k(\mathbf{x})), \quad (186)$$

$$\mathcal{L}_{res} = \frac{1}{N} \sum_{k=1}^N l_r^i + l_i^k + l_b^k, \quad (187)$$

$$\mathcal{L}_{rec}^w = \frac{1}{N} \sum_{k=1}^N L(\tilde{\theta}_w(\theta_k), \theta_k), \quad (188)$$

$$w^* = \arg \min_{w \in W} \mathcal{L}_{sup/res} + \lambda_1 \mathcal{L}_{rec}^w, \quad (189)$$

$$\min_{\theta} \mathcal{J}(G_{w^*}(\theta)(\mathbf{x}), \theta) + \lambda_2 \mathcal{L}_{rec}^w, \quad (190)$$

where L is L2 relative error and $\lambda_{1/2}$ are hyperparameters. The authors also provide a theoretical analysis of the error estimates in the optimization phase in terms of the error in the training phase. They conducted experiments on source

control of the Poisson equation, boundary control of the stationary Stokes equation, inverse design of a nonlinear wave equation, and force control of Burger's equation.

Sun *et al.* [285] propose a two-stage neural network architecture that resembles an autoencoder to address the task of synthesis. This can be considered as a generalization of PDE-constrained optimization with state variables, referred to as realization in [285], formalized as implicit functions $G_w(\theta)$ of input parameters θ . The framework can be described as

$$w^* = \arg \min_{w \in W} \|G_w(\theta)(\mathbf{x}) - \tilde{G}(\theta)(\mathbf{x})\|, \quad (191)$$

$$\phi^*(\cdot) = \arg \min_{\phi(\cdot)} \mathbb{E}_{g \sim \mathcal{D}_g} \mathcal{J}_g(G_{w^*}(\phi(g))(\mathbf{x}), \phi(g)), \quad (192)$$

where a neural network $\phi(\cdot)$ is trained to map the desired target of design g to the design parameters $\theta = \phi(g)$ on the distribution $\mathcal{D}_g$ of g with $\mathcal{J}_g$ as the corresponding objective function for g . Instead of the iterative scheme used by previous methods that directly optimize according to the given target via gradient descent, this method trains another network to predict the optimal design from the target. The performance is evaluated on the fiber extruder path planning of a 3D printer and the inverse kinematics of a soft robot. The method is proven to have performance close to AmorFEA, while the speed is faster by orders of magnitude. It also addresses the challenges of the non-differentiability of physical processes and the non-uniqueness of the design solution. The limitations of the approach are that the data-driven training demands a substantial number of samples and the objective needs to be quantifiable to be sent into the network.

Methods based on neural operators commonly address the task of inverse design with a two-stage framework. The operator is trained to solve the PDE system and then used as a differentiable surrogate for an approximate solution to perform optimization via gradient descent or neural prediction. The sequential process usually shows its superior efficiency when the objectives of design over the control parameters in the physical system are potentially changeable and diverse because the surrogate can be trained only once for different targets. However, the challenges remain in the training of neural operators for large-scale and complex physical systems in terms of data generation and training stability. Also, gradient-based optimization may have no effect on discrete and non-differentiable parameters.

With Neural Simulators. In comparison with neural operators, neural simulators refer to neural networks that learn a mapping $S : \Theta \rightarrow \mathbb{R}^n$. This mapping is from the input parameters/functions to the values of interest, which evaluate the specific performance or even directly evaluate the objective functions. The effect of these methods is similar to that of neural operators, i.e., to avoid expensive numerical simulations.

For PDE-constrained optimization, Lye *et al.* [295] propose Iterative Surrogate Model Optimization (ISMO), an active learning algorithm that combines a quasi-Newtonian optimizer with a neural network as a surrogate. The basic idea is to adopt a neural network to model the map between the parameters and the observables (e.g., lift and drag coefficients for an air foil) with supervised learning, and to perform the standard optimization based on the

approximate objective predicted by the trained model. To guarantee accurate approximation around the (local) minima of the cost function, ISMO augments the training set regularly with additional points around the estimated local minima, which are decided by the quasi-Newton method. Compared with surrogates using neural operators, ISMO only substitutes a numerical PDE solver with neural surrogate during the optimization, but still needs an expensive numerical solution when generating the training set, which dominates the cost of the algorithm.

Besides dynamic systems described by PDEs, data-driven surrogate models with neural simulators are more common for general physical systems where the physical processes are too complex to be embedded into the model or to be accurately formalized in mathematics, especially in structural optimization [273], [274], [275], rotating machines [276], [296], [297], etc. The architectures and approaches of neural simulators vary according to different scenarios and design objectives. In [274], a CNN and a fully-connected network are trained to respectively classify the type of detachment mechanism and predict interfacial stress distribution with the input of an adhesive pillar shape, and genetic optimization on the pillar shape is further performed. In [275], a neural network is employed for a surrogate gradient given a specific design by learning the iterative history data. In addition, parallel computing is adopted in the learning process to enable online updating and speed up the topology optimization. For rotating machines, CNN is often trained based on cross-sectional images and evolutionary optimization, including a genetic algorithm (GA), is used for optimization. [276], [297] use GA to optimize the IPM motor based on the classification result of CNNs and FEM is used to evaluate the individual with a certain probability. [297] focuses on a multi-objective topology optimization concerning average torque and torque ripple. [296] leverages an autoencoder to reduce dimension and maps the latent representation to the torque with a CNN. In this way, it eases the challenge that evolutionary optimization is not suited to deal with search in high-dimensional space. Neural networks as neural simulators to accelerate inverse design also have wide applications in different fields, such as aerodynamics [277], photonics [298], [299], mechanics [300], etc.

Similar to neural operators, methods based on neural simulators provide a surrogate for system simulation and evaluation, especially for complex and non-differentiable systems, where the inverse design is often formalized as general constrained optimization. The surrogate models cannot be trained in a self-supervised manner like PINN or Physics-informed DeepONet, but must be trained with simulation data in most cases. This means that time-consuming data generation for network training is necessary as well. Also, taking specific values of interest as the output of neural surrogate makes it less flexible in terms of transferring across different optimization tasks. FEM is sometimes used to simulate or evaluate during optimization even with a neural surrogate [276], [297], which means that the efficiency issue of numerical simulation remains.

4.3 Other Methods

Besides training neural networks as surrogate models to describe physical systems or embed physical knowledge, some other research leverages neural networks to play different roles in the process of inverse design. Several types of methods are listed below.

Neural Representation. Parameterizing the parameters to be optimized with neural networks instead of in their original space may take advantage of the expression of neural networks in high dimensions, in order to achieve more highly-detailed and continuous representations. This is especially the case in structural topology optimization, where conventional methods are usually grid-based or mesh-based [273], [301]. This approach can be combined with methods of neural surrogate models, as some research has already used neural networks to parameterize the control signals along with the surrogates [288], [293]. The characteristics of the parameters may introduce inductive bias to further increase the ability of neural representation.

Design Prediction. Neural surrogate models substitute the usage of numerical methods for the simulation of physical systems and the evaluation of optimization goals as forward modeling, i.e., the solutions or outcomes to physical constraints such as PDEs. Reverse methods are also proposed to predict the optimal design from optimization targets. In [285], a network mapping from the desired target to the design parameters is trained along with a neural operator as surrogate models in a self-supervised way. Moreover, using neural networks to perform inverse design directly in a data-driven manner is also common in fields such as photonics [302], airfoil optimization [303] and materials [261].

Data Generation. [304] proposes data-driven topology design using deep generative models. Given initial material distributions, the trained VAE is used to generate novel samples, which are diverse but with inherited features of the training data, and may have superior performance compared to the original ones. By iterating the process, a series of satisfactory solutions can be obtained.

4.4 Beyond Inverse Design

In fact, inverse design is a realistic application of inverse problem, which also includes property identification, parameter inference and scientific discovery, which are critical problems in scientific researches. The formulation is consistent with Eq. (178), while the objective function $\mathcal{J}$ usually describes the difference between the neural solutions and observations.

The original PINN [30] has demonstrated its potential in inverse problem of property identification and parameter inference under the unified framework with physics-informed losses. Then it's widely adopted to study inverse problems in various fields. In material science, physics-informed deep learning methods are used to discover new materials or identify properties of given materials from experimental/simulated data (inverse problems) [135], [237]. In fluid dynamic, physics-informed algorithms are taken to learn physical quantities such as velocity and pressure from flow data (inverse problems) [195], [222], [226], [278]. Specifically, [278] developed a physics-informed neural network

framework to infer physical quantities of interest such as velocity and pressure from the spatial-temporal visualizations of fluids. They combined known physical priors (namely governing equations) with available data to draw inferences that may be useful for physical or biological problems where direct measurement is infeasible.

Scientific discovery is another class of inverse problems, which intends to discover new physical laws or determine unknown parameters of the physical system from data. Physics-informed deep learning methods such as PINNs and neural operators are competitive choices to solve such problems [305], [306], especially for the cases of high dimension [307] or scarce and noisy data [308]. The authors of [309] used two neural networks trained in a physics-informed manner with some noisy observations to discover the governing equation (which was originally partially known) as well as its solution. Besides, other methods are also proposed to address this task. For instance, PDE-Net [310] learns the specific form of PDE from data, while [311] not only represented the unknown governing equation (physical laws) with a graph neural network in a supervised setting, but also extracted analytical expressions from the learned model via symbolic regression.

4.5 Open Challenges and Future Work

Compared to the problems of neural solvers and neural operators, inverse design is a more complex and comprehensive task with multiple technical parts involving representation, simulation and optimization. Therefore, the methods, which adopt machine learning, especially deep learning to address inverse design, are more diverse. Besides methods discussed in previous sections, there are still other potential options to incorporate neural networks to handle inverse design, such as using reinforcement learning [312].

The remaining challenges of inverse design are also various. We summarize some typical open problems as follows.

- **Neural Surrogate Modeling** Some of the open challenges are consistent with those of neural solvers and operators, including the balance of multiple loss terms and training convergence for physics-informed methods, the large amount of data demand for operator and simulator, etc. These were introduced in detail in the previous sections. Addressing these challenges may contribute to the development of machine learning algorithms for inverse design.
- **Large Scale Application** Inverse design are related to abundant practical application scenarios, where the physical systems have a large number of parameters to be optimized. This could lead to other challenges including but not limited to issues of curse of dimensions, extensions to large-scale scenarios and computational complexity of optimization.
- **Other Directions** Since solving the problems of inverse design involves multiple steps, the usage of neural network in different parts can be further exploited. Improving the efficiency of data utilization, incorporating the physical knowledge into the framework of inverse design and fusing with traditional numerical methods are potential directions for future researches.

5 COMPUTER VISION AND GRAPHICS

In this section, we describe how physical knowledge can be incorporated in computer vision and computer graphics.

5.1 Problem Overview

Computer vision involves diverse tasks including the prediction, generation, and analysis of digital images, videos, and even data like point clouds or sensor measurements, while computer graphics aims at rendering vivid scenes where physics law must be satisfied to achieve sense of reality. Both focus on the perception, generation and interaction with real-world environments, which are governed by plentiful physical rules.

However, current data-driven deep learning methods are not sufficient. Deep neural networks trained from natural or synthesized datasets perform based on the underlying patterns of data, and could encounter challenges concerning stability, reliability and security, e.g., out-of-distribution problem (OOD) [10], [313], adversarial examples [314], [315]. Incorporating deep learning models with physical knowledge can further constrain the prediction to follow the laws and guarantee the preciseness. For instance, in motion tracking and pose analysis, physical constraints or simulations [316], [317] can boost the models' physical plausibility and correctness to reduce unrealistic errors.

Meanwhile, the interpretability of deep learning is still an open problem in machine learning [318]. The black-box nature of neural networks limits the trust in their grounded applications, especially in security-sensitive scenarios like auto-driving [319], [320] and medicine [321]. Empowered by physical knowledge, the performance of models can be partially expressed and explained by formalized rules. In consecutive predictions with videos, recurrent predictors based on dynamical equations could restrict the future trajectories and improve the long-term performance [148].

5.2 Physics-informed Computer Vision and Graphics

Attempts to incorporate physical knowledge in deep learning based computer vision and computer graphics are still primary. We will conclude some methods in the following context, according to the parts of machine learning model to introduce physics.

5.2.1 Data

Latest works [322], [323] have demonstrated the capability of deep learning models to form learned priors and experiences from massive data. Directly introducing training data following the physical rules can help models learn the underlying physical structure and distribution, which is categorized as "observational bias" by [12].

There have been some datasets describing physical objects and scenes [324], [325], [326]. In [327], Visual Newtonian Dynamics (VIND) dataset is compiled, including more than 6000 videos aligned with Newtonian scenarios represented using game engines, and more than 4500 still images with their ground truth dynamics. This could help models understand the objects in a static scene in terms of the forces acting upon it and its long term motion as response to those forces. Physics101 dataset [326] contains thousands of video

clips recording objects in simple physical scenarios like collision, to study the automatic learning of object-centric physical properties. Physical Concepts dataset [37] follows the violation-of-expectation (VoE) paradigm in developmental psychology and collects videos with VoE probes involving basic physical concepts like continuity, persistence, solidity. This corresponds to the learning of intuitive physics, also named naïve physics, that is semantically meaningful but hard to be directly incorporated in the learning procedure. There are also datasets [324], [325] gathering objects with various physical properties.

5.2.2 Architecture

The architecture design of neural networks inspired or guided by physics can improve the efficacy and interpretability of models. In fact, the convolution operation in CNN [2] is based on the translation invariance and has achieved great success.

There could be more space to for improvement when more complex physical knowledge comes in. For instance, Equivariant Object detection Network (EON) [264] introduces object-level rotation equivariance to 3D object detection by designing a rotation equivariance suspension design to extract the equivariant seed features. In scenarios more specific to physical process, some works adopt neural networks as tool to express physical quantities and evolutions. In visual prediction for scenarios involving physical phenomena like collision, a typical way is to use an encoder to extract physical properties or states from images and perform future prediction with a dynamics predictor using different strategies, e.g. recurrent network [37], [328], physical engines [329], optical flow [38], Hamiltonian Generative Network [148]. The constraints on latent physical properties also vary. [329] proposed to explicitly estimate the physical object states and [38] preferred learning entangled representations while distinguishing whether the state is physical variable or intrinsic variable through dimension. Besides, Interaction networks [330] use a graphical representation of objects and relationships and model the effects of relations by a neural network. The current state of an object, aggregate effect of relations, and the external effects are recurrently fed into another neural network to predict next state. This corresponds to the evolution of Newtonian mechanics, where we aggregate the forces exerted by other objects or by an external force to determine acceleration, and update the state by the acceleration.

5.2.3 Loss Function

By embedding prior physical knowledge into the loss function of a machine learning model, it can guide the learning procedure and restrict the model to satisfy physics constraints.

HNN [78] uses Hamiltonian equations as the objective function to constrain the neural network to learn the quantity of Hamiltonian in the system, as introduced in Sec.3.2.6. This is similar to PINN [30] and performs as soft constraint. Besides adopting formalized physical equations directly as objectives, another way is to reconstruct the observations following the physical process and take the reconstructed results to supervise the training. Neural radiance field (NeRF) [134] models the volume density

and a view-dependent color for any given position-view pair. With classical volume rendering techniques, we can generate differentiable images following physical law and train the network supervised by ground-truth inputs. This general framework is also utilized in various visual tasks, for either supervision or regularization. In deblurring, Re-blur2Deblur [331] reconstructs the blurry input by physics-based reblurring with an optical flow network and construct a self-supervised loss term as regularization. In computer graphics, PhyIR [332] addresses the inverse rendering of complex material with a physics-based in-network rendering module to physically re-render realistic reflectance, which serves as a constraint by a re-render loss to realistic lighting effects. Physics simulator can also be used for supervision. For motion capture [333], a physics simulator is taken as a supervisor to train a motion distribution prior for further motion sampling to capture physically plausible human motion from a monocular color video. Other forms of physics regularization are also possible, for example in depth estimation [334], image dehazing [335], 3D reconstruction [336].

5.2.4 Inference

Another option is to impose physical constraints at inference stage, to adjust and correct the predictions to reach physical feasibility.

In motion and pose analysis, physics knowledge of kinematics and dynamics, usually integrated into physics simulators, along with intuitive physical common sense are often embedded into the pipeline of inference. [317] uses feature-complete physical simulation as a building block to incorporate more subtle physical effects such as sliding and rolling friction, or surfaces with varying degrees of softness. Physical Inertial Poser (PIP) [316] achieves real-time motion tracking with satisfying accuracy using only six inertial sensors. The tracking is performed by combining a neural kinematics estimator and a physics-aware motion optimizer. The estimated kinematic parameters of motion status are further refined by the optimizer according to constraints of motion equation, friction cone, and the condition of no sliding.

5.3 Open Challenges and Future Work

There have been some primary explorations to combine computer vision with physical knowledge, while open challenges remain and demand future studies into them. Some of them are as follows.

- **Enforcing Symmetry Constraints.** A lot of current papers try only to imitate the form of physical dynamics and ignore the underlying symmetry. For example, if we incorporate the translation invariance into the network design, the hypothesis space can be reduced, and we can train the neural networks more efficiently.
- **Learning Meaningful Representations.** When encoding images into a latent space or representing the effects from one object to another, the physical meanings of these representations are hardly examined. If we can map from image-based dynamics exactly to

equation-based dynamics with clear physical explanations, we will be able to do simulation, dynamics discovery, and extraction of physical variables at the same time.

- **Formulated Representations of Intuitive Physics.** For vision tasks in daily scenarios, many rules of motion, collision, and interaction are described with intuitive physics, rather than rigorous physical equations. However, the unformulated representations of intuitive physics make it limited to utilize the knowledge in the learning framework, which is usually used in the form of constraints.

6 REINFORCEMENT LEARNING

In this section, firstly we introduce the basic setting and the goal of reinforcement learning. Then, we discuss how physical knowledge can be incorporated into reinforcement learning.

6.1 Problem Formulation

Reinforcement learning is about an agent learning to act in an unknown environment to maximize its reward. One classic formulation of reinforcement learning is based on Markov Decision Processes (MDPs). An MDP defines the effects of all actions in all states of the environment, including the state transitions and rewards received by the agent.

Mathematically, an MDP can be defined as a tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$. Here, the state space $\mathcal{S}$ is the set of all possible states in the environment, and the action space $\mathcal{A}$ is the set of all possible actions that the agent can take. The transition function $T : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ defines the state transition from a given state when the agent performs a given action. The reward function $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbf{P}(\mathbb{R})$ defines the distribution of rewards when the agent performs a given action in a given state. γ is a discount factor that balances short-term and long-term goals by discounting future rewards. The goal of the agent is to learn a policy $\pi : \mathcal{S} \rightarrow \mathbf{P}(\mathcal{A})$ to maximize the expected cumulative discounted reward $\sum_t \gamma^t r_t$, where for all t , $r_t \sim R(s_t, a_t)$, $s_{t+1} = T(s_t, a_t)$, $a_t \sim \pi(s_t)$.

Difficulties in reinforcement learning include sequential correlation between interactions and the environment, as well as learning by reward signals instead of learning by some "ground truth". As a result, a reinforcement learning algorithm usually learns by trial and error. In model-based reinforcement learning, we additionally learn a model of the environment to help the learning of the agent. In safe reinforcement learning, we want to avoid disastrous outcomes during training, so we may want to add some constraints to our policy.

Next, we discuss three aspects of incorporating physical knowledge: policy training, model training, and exploration guiding.

6.2 Policy Training

Many reinforcement learning tasks originate from real-world problems, and our understanding of these problems can help us design a friendly task to accelerate the training of the policy.

[337] focuses on reinforcement learning for mesh refinements when learning solutions of PDEs, which is inspired by adaptive mesh refinement (AMR). It uses the change in errors to guide the selection of refinements and achieve high error reductions.

[338] considers a task where we want to adapt to an unknown reference quantum state by measurements. The method naturally provides many copies of the reference state, and it uses knowledge of quantum mechanics to design reward functions. In this way, it can achieve a high average fidelity with only hundreds of iterations.

[339] introduces reinforcement learning to chip placement. It defines a reinforcement learning task to simulate the placement, including hard constraints on density a reward function dependent on the proxy wire length and congestion after the whole placement process. Experiments show that the algorithm can generate superhuman placements in under 6 hours.

6.3 Model Training

In model-based reinforcement learning, when the model is corresponding to parts of the physical world, our knowledge can help us to build a better model.

A recent work [340] incorporates the form of continuous dynamics into the learning of environment models and transfers the reinforcement learning agent in the environment model to the real environment. It demonstrates that physical models can work better in offline model-based reinforcement learning than black-box models, probably because of better extrapolation power. Similarly, [341] also estimates the dynamics based on a basic form of continuous dynamics via least squares. Besides, it plans over the estimated dynamics together with model-predictive control (MPC) to achieve a real-time policy with high performance.

In visual model-based reinforcement learning, the Object-Centric Perception, Prediction, and Planning (OP3) framework [342] extracts representations of a variable number of objects and aggregates estimated effects from different sources to predict the next state. By planning over the model, their algorithm can outperform a state-of-the-art video prediction model at that time.

6.4 Exploration Guiding

In safe reinforcement learning, we need to add constraints to the policy as guidance in exploration. Usually, we need to use physical knowledge to find suitable constraints such that some safety requirements are satisfied.

[343] introduces differential dynamic logic informed by expert knowledge of the environment to create a sandbox for reinforcement learning. Specifically, it uses a logic formula to model safe transitions, and filter out actions that are not safe. If the formula is precise, the agent can get higher rewards without being unsafe during training.

The RL-CBF algorithm [344] uses expert knowledge to construct a Gaussian Process (GP), which models the state-dependent uncertainty in an environment. The safe policy is the solution to a robust optimization problem over a confidence region of the Gaussian process assuming that the uncertainty is adversarial. In this way, it can approximately

guarantee probabilistic safety only with graceful compromises. [345] also uses a GP to describe uncertainty. They use the Hamilton–Jacobi–Isaacs (HJI) variational inequality instead to solve a robust policy and switch back to the robust policy when the agent is going to a dangerous region.

[346] views safe reinforcement learning as a constraint optimization problem, and introduces the generalized control barrier function (GCBF) together with the constraint policy gradient method to enhance the safety of the policy. In this way, the method can converge faster with fewer constraint violations.

6.5 Open Challenges and Future Work

Reinforcement learning has achieved impressive progress with physical knowledge, but there are still some open challenges to be solved in this field, which are shown below.

- **Generic Modeling of Physical Tasks.** Currently, we mainly convert physical tasks into reinforcement learning tasks in a case-by-case way. If there is a general framework, we can easily deal with new physical tasks with proper expert knowledge. To do this, we may start with building a general framework within a subfield, and then try to expand it to a larger domain.
- **Solving High-Dimensional Problems.** A lot of physical problems are inherently high-dimensional, but reinforcement learning problems with high-dimensional state spaces are not sample-efficient, or even not tractable. To alleviate this, we usually resort to state space compressing such as doing reinforcement learning in a latent space constructed by a world model. However, learning a compact, meaningful latent space remains an open challenge. Besides, if the problem itself is already compact enough, we may only approximate the best policy by defining subproblems and combining their solutions.
- **Guaranteeing Safety in Complex, Uncertain Environments.** In safe reinforcement learning, there is still a trade-off between environment complexity and safety guarantee. In a complex, uncertain environment (e.g., there are transition noises estimated by a Gaussian Process over the state space), we can not get a probabilistic safety guarantee before training without strong assumptions. Even when we enable runtime checking, safety may still be compromised, or the checking itself does not scale to high-dimensional state spaces.

7 CONCLUSION

In this review, we have systematically investigated and summarized the field of physics-informed machine learning as seen through the eyes of machine learning researchers. First, we have identified and introduced the general concept of physics-informed machine learning. We suggest that there are several types of physical prior, i.e. PDEs/ODEs/SDEs, symmetry constraints and intuitive physics. They could be embedded into different parts of machine learning models, i.e. data, architecture, loss functions, optimization methods and inference algorithms. Then, we exhaustively presented

existing methods, challenges, and future directions for these problems. Most of existing works focus on using neural networks for solving or identifying systems governed by PDEs/ODEs, i.e. neural simulation and inverse design. We have summarized the progress of these methods in detail.

From a methodological perspective, there are many open challenges for problems of physics-informed machine learning.

- How to design a standardized dataset for problems of different physical priors is an open challenge. Datasets and benchmarks provide a fair environment to compare different algorithms and inspire researchers to discover new ones. In the field of physics-informed machine learning, such datasets are lacking due to the diversity of physical priors. A valuable, realistic dataset or benchmark containing either a single level or multiple levels of physical prior will play important role in boosting the machine learning community.
- Designing better optimization and inference algorithms incorporating or informed by physical priors is a valuable topic. While there are many works exploring model architectures inspired or to represent physical prior, optimization methods and inference algorithms receive less attention. Different from constraining the hypothesis space, novel optimization methods incorporating physical prior might be another choice to train physics-informed machine learning models. Moreover, when models are pre-trained, we might also need better inference algorithms to ensure the output satisfies physical laws.
- Scalable algorithms incorporating data with real-world physical prior or intuitive physics is a basic issue for building intelligent systems capable of real-world interaction. It might be a prevailing trend for developing machine learning models driven by both large data and physical prior with a broad range of applications in computer vision and robotics control.

From the perspective of tasks of physics-informed machine learning, we also suggest several research directions and opportunities.

- For neural simulation, existing optimization techniques and architectures are far from optimal. There is still a gap between PINNs and highly specialized traditional numerical methods like FEM/FVM/FDM/Spectral methods on both speed and accuracy. Promising future research directions include inventing novel and effective optimization targets, learning paradigms and neural architectures.
- Inverse problems are fundamental problems in scientific discovery, computer vision as well as many other engineering domains. Many works show that methods based on PINNs and DeepONets achieve better results on inverse problems compared with traditional methods. However, the complexity and ill-posedness requires learning algorithms that better utilize data and physical knowledge. Moreover, inverse design, as a specific type of inverse problem has many promising application scenarios in struc-

tural/topology optimization, optimal control and molecular/drug discovery.

- For computer vision and reinforcement learning in real world, we need better algorithms for inference stage or training stages that incorporates physical prior. A possible direction is to model the real world environment with physical prior. The emergence of concepts like world models or NERF provides a possible method that we could learn the real world environment from data with the help of physical prior. Such models could then be used for training downstream models interacting with the world.
- Theoretical analysis like convergence and generalization ability for physics-informed machine learning algorithms are still at a beginning. It is still a challenging task due to the difficulty of analyzing the training process of neural networks. We even don't know what is the theoretical benefits of introducing physical prior into machine learning.

We conclude that physics-informed machine learning will be a fundamental and essential topic of AI. There is still much potential for improving current methods of physics-informed machine learning.

REFERENCES

- [1] T. Mitchell, B. Buchanan, G. DeJong, T. Dietterich, P. Rosenbloom, and A. Waibel, "Machine learning," *Annual review of computer science*, vol. 4, no. 1, pp. 417–433, 1990.
- [2] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [5] D. Amodei, S. Ananthanarayanan, R. Anubhai, J. Bai, E. Battenberg, C. Case, J. Casper, B. Catanzaro, Q. Cheng, G. Chen *et al.*, "Deep speech 2: End-to-end speech recognition in english and mandarin," in *International conference on machine learning*. PMLR, 2016, pp. 173–182.
- [6] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, "Mastering the game of go with deep neural networks and tree search," *nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [7] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Zidek, A. Potapenko *et al.*, "Highly accurate protein structure prediction with alphafold," *Nature*, vol. 596, no. 7873, pp. 583–589, 2021.
- [8] J. Pathak, S. Subramanian, P. Harrington, S. Raja, A. Chattopadhyay, M. Mardani, T. Kurth, D. Hall, Z. Li, K. Azizzadenesheli *et al.*, "Fourcastnet: A global data-driven high-resolution weather model using adaptive fourier neural operators," *arXiv preprint arXiv:2202.11214*, 2022.
- [9] L. Zhang, J. Han, H. Wang, R. Car, and E. Weinan, "Deep potential molecular dynamics: a scalable model with the accuracy of quantum mechanics," *Physical review letters*, vol. 120, no. 14, p. 143001, 2018.
- [10] Z. Shen, J. Liu, Y. He, X. Zhang, R. Xu, H. Yu, and P. Cui, "Towards out-of-distribution generalization: A survey," *arXiv preprint arXiv:2108.13624*, 2021.
- [11] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.
- [12] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.
- [13] N. Thuerey, P. Holl, M. Mueller, P. Schnell, F. Trost, and K. Um, "Physics-based deep learning," *arXiv preprint arXiv:2109.05237*, 2021.
- [14] A. Clark, "Whatever next? predictive brains, situated agents, and the future of cognitive science," *Behavioral and brain sciences*, vol. 36, no. 3, pp. 181–204, 2013.
- [15] S. Cuomo, V. S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli, "Scientific machine learning through physics-informed neural networks: Where we are and what's next," *arXiv preprint arXiv:2201.05624*, 2022.
- [16] C. Beck, M. Hutzenthaler, A. Jentzen, and B. Kuckuck, "An overview on deep learning-based approximation methods for partial differential equations," *arXiv preprint arXiv:2012.12348*, 2020.
- [17] S. Cai, Z. Mao, Z. Wang, M. Yin, and G. E. Karniadakis, "Physics-informed neural networks (pinns) for fluid mechanics: A review," *arXiv preprint arXiv:2105.09506*, 2021.
- [18] A. F. Psaros, X. Meng, Z. Zou, L. Guo, and G. E. Karniadakis, "Uncertainty quantification in scientific machine learning: Methods, metrics, and comparisons," *arXiv preprint arXiv:2201.07766*, 2022.
- [19] A. Heinlein, A. Klawonn, M. Lanser, and J. Weber, "Combining machine learning and domain decomposition methods for the solution of partial differential equations—a review," *GAMM-Mitteilungen*, vol. 44, no. 1, p. e202100001, 2021.
- [20] R. Wang and R. Yu, "Physics-guided deep learning for dynamical systems: A survey," *arXiv preprint arXiv:2107.01272*, 2021.
- [21] K. Zubov, Z. McCarthy, Y. Ma, F. Calisto, V. Pagliarino, S. Azeaglio, L. Bottero, E. Luján, V. Sulzer, A. Barambe *et al.*, "Neuralpde: Automating physics-informed neural networks (pinns) with error approximations," *arXiv preprint arXiv:2107.09443*, 2021.
- [22] K. C. Cheung and S. See, "Recent advance in machine learning for partial differential equation," *CCF Transactions on High Performance Computing*, vol. 3, no. 3, pp. 298–310, 2021.
- [23] J. Blechschmidt and O. G. Ernst, "Three ways to solve partial differential equations with neural networks—a review," *GAMM-Mitteilungen*, vol. 44, no. 2, p. e202100006, 2021.
- [24] D. A. Pratama, M. A. Bakar, M. Man, and M. Mashuri, "Anns-based method for solving partial differential equations: A survey," 2021.
- [25] S. Das and S. Tesfamariam, "State-of-the-art review of design of experiments for physics-informed deep learning," *arXiv preprint arXiv:2202.06416*, 2022.
- [26] R. Rai and C. K. Sahu, "Driven by data or derived through physics? a review of hybrid physics guided machine learning techniques with cyber-physical system (cps) focus," *IEEE Access*, vol. 8, pp. 71 050–71 073, 2020.
- [27] C. Meng, S. Seo, D. Cao, S. Griesemer, and Y. Liu, "When physics meets machine learning: A survey of physics-informed machine learning," *arXiv preprint arXiv:2203.16797*, 2022.
- [28] J. Willard, X. Jia, S. Xu, M. Steinbach, and V. Kumar, "Integrating physics-based modeling with machine learning: A survey," *arXiv preprint arXiv:2003.04919*, vol. 1, no. 1, pp. 1–34, 2020.
- [29] M. Frank, D. Drikakis, and V. Charissis, "Machine-learning methods for computational science and engineering," *Computation*, vol. 8, no. 1, p. 15, 2020.
- [30] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational physics*, vol. 378, pp. 686–707, 2019.
- [31] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, "Neural ordinary differential equations," *Advances in neural information processing systems*, vol. 31, 2018.
- [32] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 652–660.
- [33] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [34] M. Dissanayake and N. Phan-Thien, "Neural-network-based approximations for solving partial differential equations," *communications in Numerical Methods in Engineering*, vol. 10, no. 3, pp. 195–201, 1994.

- [35] T. Cohen and M. Welling, "Group equivariant convolutional networks," in *International conference on machine learning*. PMLR, 2016, pp. 2990–2999.
- [36] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković, "Geometric deep learning: Grids, groups, graphs, geodesics, and gauges," *arXiv preprint arXiv:2104.13478*, 2021.
- [37] L. S. Piloto, A. Weinstein, P. Battaglia, and M. Botvinick, "Intuitive physics learning in a deep-learning model inspired by developmental psychology," *Nature human behaviour*, vol. 6, no. 9, pp. 1257–1267, 2022.
- [38] T. Ye, X. Wang, J. Davidson, and A. Gupta, "Interpretable intuitive physics model," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 87–102.
- [39] J. Duan, A. Dasgupta, J. Fischer, and C. Tan, "A survey on machine learning approaches for modelling intuitive physics," *arXiv preprint arXiv:2202.06481*, 2022.
- [40] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. Battaglia, "Learning to simulate complex physics with graph networks," in *International Conference on Machine Learning (ICML)*, 2020, pp. 8459–8468.
- [41] S. A. Faroughi, N. Pawar, C. Fernandes, S. Das, N. K. Kalantari, and S. K. Mahjour, "Physics-guided, physics-informed, and physics-encoded neural networks in scientific computing," *arXiv preprint arXiv:2211.07377*, 2022.
- [42] K. Xu, A. Srivastava, D. Gutfreund, F. Sosa, T. Ullman, J. Tenenbaum, and C. Sutton, "A bayesian-symbolic approach to reasoning and learning in intuitive physics," *Advances in Neural Information Processing Systems*, vol. 34, pp. 2478–2490, 2021.
- [43] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [44] J. Willard, X. Jia, S. Xu, M. Steinbach, and V. Kumar, "Integrating scientific knowledge with machine learning for engineering and environmental systems," *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–37, 2022.
- [45] F. Bjelonic, J. Lee, P. Arm, D. Sako, D. Tateo, J. Peters, and M. Hutter, "Learning-based design and control for quadrupedal robots with parallel-elastic actuators," *IEEE Robotics and Automation Letters*, 2023.
- [46] A. Ramesh and B. Ravindran, "Physics-informed model-based reinforcement learning," *arXiv preprint arXiv:2212.02179*, 2022.
- [47] D. Causon and C. Mingham, *Introductory finite difference methods for PDEs*. Bookboon, 2010.
- [48] J. C. Butcher, "A history of runge-kutta methods," *Applied numerical mathematics*, vol. 20, no. 3, pp. 247–260, 1996.
- [49] R. Eymard, T. Gallouët, and R. Herbin, "Finite volume methods," *Handbook of numerical analysis*, vol. 7, pp. 713–1018, 2000.
- [50] C. A. Felippa, "Introduction to finite element methods," *University of Colorado*, vol. 885, 2004.
- [51] C. Bernardi and Y. Maday, "Spectral methods," *Handbook of numerical analysis*, vol. 5, pp. 209–485, 1997.
- [52] T. Xue, A. Beatson, S. Adriaenssens, and R. Adams, "Amortized finite element analysis for fast pde-constrained optimization," in *International Conference on Machine Learning*. PMLR, 2020, pp. 10 638–10 647.
- [53] H.-J. Yoo, "Deep convolution neural networks in computer vision: a review," *IEIE Transactions on Smart Processing and Computing*, vol. 4, no. 1, pp. 35–43, 2015.
- [54] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," *SIAM Journal on Scientific Computing*, vol. 43, no. 5, pp. A3055–A3081, 2021.
- [55] S. Wang, X. Yu, and P. Perdikaris, "When and why pinns fail to train: A neural tangent kernel perspective," *Journal of Computational Physics*, vol. 449, p. 110768, 2022.
- [56] S. Maddu, D. Sturm, C. L. Müller, and I. F. Sbalzarini, "Inverse dirichlet weighting enables reliable training of physics informed neural networks," *Machine Learning: Science and Technology*, vol. 3, no. 1, p. 015026, 2022.
- [57] J. Sirignano and K. Spiliopoulos, "Dgm: A deep learning algorithm for solving partial differential equations," *Journal of computational physics*, vol. 375, pp. 1339–1364, 2018.
- [58] P.-H. Chiu, J. C. Wong, C. Ooi, M. H. Dao, and Y.-S. Ong, "Can-pinn: A fast physics-informed neural network based on coupled-automatic-numerical differentiation method," *Computer Methods in Applied Mechanics and Engineering*, vol. 395, p. 114909, 2022.
- [59] R. G. Patel, I. Manickam, N. A. Trask, M. A. Wood, M. Lee, I. Tomas, and E. C. Cyr, "Thermodynamically consistent physics-informed neural networks for hyperbolic systems," *Journal of Computational Physics*, vol. 449, p. 110754, 2022.
- [60] E. Kharazmi, Z. Zhang, and G. E. Karniadakis, "Variational physics-informed neural networks for solving partial differential equations," *arXiv preprint arXiv:1912.00873*, 2019.
- [61] —, "hp-vpinns: Variational physics-informed neural networks with domain decomposition," *Computer Methods in Applied Mechanics and Engineering*, vol. 374, p. 113547, 2021.
- [62] R. Khodayi-Mehr and M. Zavlanos, "Varnet: Variational neural networks for the solution of partial differential equations," in *Learning for Dynamics and Control*. PMLR, 2020, pp. 298–307.
- [63] Y. Zang, G. Bao, X. Ye, and H. Zhou, "Weak adversarial networks for high-dimensional partial differential equations," *Journal of Computational Physics*, vol. 411, p. 109409, 2020.
- [64] J. Yu, L. Lu, X. Meng, and G. E. Karniadakis, "Gradient-enhanced physics-informed neural networks for forward and inverse pde problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 393, p. 114823, 2022.
- [65] H. Son, J. W. Jang, W. J. Han, and H. J. Hwang, "Sobolev training for the neural network solutions of pdes," *arXiv preprint arXiv:2101.08932*, 2021.
- [66] A. D. Jagtap, K. Kawaguchi, and G. E. Karniadakis, "Adaptive activation functions accelerate convergence in deep and physics-informed neural networks," *Journal of Computational Physics*, vol. 404, p. 109136, 2020.
- [67] A. D. Jagtap, K. Kawaguchi, and G. E. Karniadakis, "Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks," *Proceedings of the Royal Society A*, vol. 476, no. 2239, p. 20200334, 2020.
- [68] Z. Liu, W. Cai, and Z.-Q. J. Xu, "Multi-scale deep neural network (mscalednn) for solving poisson-boltzmann equation in complex domains," *arXiv preprint arXiv:2007.11207*, 2020.
- [69] S. Wang, H. Wang, and P. Perdikaris, "On the eigenvector bias of fourier feature networks: From regression to solving multi-scale pdes with physics-informed neural networks," *Computer Methods in Applied Mechanics and Engineering*, vol. 384, p. 113938, 2021.
- [70] W. Peng, W. Zhou, J. Zhang, and W. Yao, "Accelerating physics-informed neural network training with prior dictionaries," *arXiv preprint arXiv:2004.08151*, 2020.
- [71] C. Leake and D. Mortari, "Deep theory of functional connections: A new method for estimating the solutions of partial differential equations," *Machine learning and knowledge extraction*, vol. 2, no. 1, pp. 37–55, 2020.
- [72] Y. Wang, J. Sun, W. Li, Z. Lu, and Y. Liu, "Cenn: Conservative energy method based on neural networks with subdomains for solving heterogeneous problems involving complex geometries," *arXiv preprint arXiv:2110.01359*, 2021.
- [73] H. Sheng and C. Yang, "Pfn: A penalty-free neural network method for solving a class of second-order boundary-value problems on complex geometries," *Journal of Computational Physics*, vol. 428, p. 110085, 2021.
- [74] S. Liu, Z. Hao, C. Ying, H. Su, J. Zhu, and Z. Cheng, "A unified hard-constraint framework for solving geometrically complex pdes," *arXiv preprint arXiv:2210.03526*, 2022.
- [75] P. Ren, C. Rao, Y. Liu, J.-X. Wang, and H. Sun, "Phycrnet: Physics-informed convolutional-recurrent network for solving spatiotemporal pdes," *Computer Methods in Applied Mechanics and Engineering*, vol. 389, p. 114399, 2022.
- [76] R. Zhang, Y. Liu, and H. Sun, "Physics-informed multi-lstm networks for metamodelling of nonlinear structures," *Computer Methods in Applied Mechanics and Engineering*, vol. 369, p. 113226, 2020.
- [77] N. Geneva and N. Zabaras, "Modeling the dynamics of pde systems with physics-constrained deep auto-regressive networks," *Journal of Computational Physics*, vol. 403, p. 109056, 2020.
- [78] S. J. Greydanus, M. Dzumba, and J. Yosinski, "Hamiltonian neural networks," 2019.
- [79] M. Lutter, C. Ritter, and J. Peters, "Deep lagrangian networks: Using physics as model prior for deep learning," *arXiv preprint arXiv:1907.04490*, 2019.
- [80] H. Gao, L. Sun, and J.-X. Wang, "Phygeonet: Physics-informed geometry-adaptive convolutional neural networks for solving parameterized steady-state pdes on irregular domain," *Journal of Computational Physics*, vol. 428, p. 110079, 2021.
- [81] X.-Y. Liu, H. Sun, and J.-X. Wang, "Predicting parametric spatiotemporal dynamics by multi-resolution pde structure-preserved deep learning," *arXiv preprint arXiv:2205.03990*, 2022.

- [82] A. D. Jagtap and G. E. Karniadakis, "Extended physics-informed neural networks (xpinns): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations," *Communications in Computational Physics*, vol. 28, no. 5, pp. 2002–2041, 2020.
- [83] A. D. Jagtap, E. Kharazmi, and G. E. Karniadakis, "Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 365, p. 113028, 2020.
- [84] B. Moseley, A. Markham, and T. Nissen-Meyer, "Finite basis physics-informed neural networks (fbpinns): a scalable domain decomposition approach for solving differential equations," *arXiv preprint arXiv:2107.07871*, 2021.
- [85] K. Shukla, A. D. Jagtap, and G. E. Karniadakis, "Parallel physics-informed neural networks via domain decomposition," *Journal of Computational Physics*, vol. 447, p. 110683, 2021.
- [86] S. Desai, M. Mattheakis, H. Joy, P. Protopapas, and S. Roberts, "One-shot transfer learning of physics-informed neural networks," *arXiv preprint arXiv:2110.11286*, 2021.
- [87] S. Chakraborty, "Transfer learning based multi-fidelity physics informed deep neural network," *Journal of Computational Physics*, vol. 426, p. 109942, 2021.
- [88] A. F. Psaros, K. Kawaguchi, and G. E. Karniadakis, "Meta-learning pinn loss functions," *Journal of Computational Physics*, vol. 458, p. 111121, 2022.
- [89] X. Liu, X. Zhang, W. Peng, W. Zhou, and W. Yao, "A novel meta-learning initialization method for physics-informed neural networks," *Neural Computing and Applications*, pp. 1–24, 2022.
- [90] Y. Zhang and Q. Yang, "A survey on multi-task learning," *IEEE Transactions on Knowledge and Data Engineering*, 2021.
- [91] B. Yu *et al.*, "The deep ritz method: a deep learning-based numerical algorithm for solving variational problems," *Communications in Mathematics and Statistics*, vol. 6, no. 1, pp. 1–12, 2018.
- [92] A. D. Jagtap and G. E. Karniadakis, "Extended physics-informed neural networks (xpinns): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations," in *AAAI Spring Symposium: MLPS*, 2021.
- [93] A. Jacot, F. Gabriel, and C. Hongler, "Neural tangent kernel: Convergence and generalization in neural networks," *Advances in neural information processing systems*, vol. 31, 2018.
- [94] R. Leiteritz, P. Buchfink, B. Haasdonk, and D. Pflüger, "Surrogate-data-enriched physics-aware neural networks," *arXiv preprint arXiv:2112.05489*, 2021.
- [95] R. van der Meer, C. W. Oosterlee, and A. Borovykh, "Optimally weighted loss functions for solving pdes with neural networks," *Journal of Computational and Applied Mathematics*, vol. 405, p. 113887, 2022.
- [96] S. Wang, S. Sankaran, and P. Perdikaris, "Respecting causality is all you need for training physics-informed neural networks," *arXiv preprint arXiv:2203.07404*, 2022.
- [97] F. M. Rohrhofer, S. Posch, and B. C. Geiger, "On the pareto front of physics-informed neural networks," *arXiv preprint arXiv:2105.00862*, 2021.
- [98] D. Liu and Y. Wang, "A dual-dimer method for training physics-constrained neural networks with minimax architecture," *Neural Networks*, vol. 136, pp. 112–125, 2021.
- [99] L. McClenny and U. Braga-Neto, "Self-adaptive physics-informed neural networks using a soft attention mechanism," *arXiv preprint arXiv:2009.04544*, 2020.
- [100] I. M. Sobol', "On the distribution of points in a cube and the approximate evaluation of integrals," *Zhurnal Vychislitel'noi Matematiki i Matematicheskoi Fiziki*, vol. 7, no. 4, pp. 784–802, 1967.
- [101] M. D. McKay, R. J. Beckman, and W. J. Conover, "A comparison of three methods for selecting values of input variables in the analysis of output from a computer code," *Technometrics*, vol. 42, no. 1, pp. 55–61, 2000.
- [102] M. Berblinger and C. Schlier, "Monte carlo integration with quasirandom numbers: some experience," *Computer physics communications*, vol. 66, no. 2-3, pp. 157–166, 1991.
- [103] T.-T. Wong, W.-S. Luk, and P.-A. Heng, "Sampling with hammersley and halton points," *Journal of graphics tools*, vol. 2, no. 2, pp. 9–24, 1997.
- [104] H. Faure, "Discrépance de suites associées à un système de numération (en dimension s)," *Acta arithmetica*, vol. 41, no. 4, pp. 337–351, 1982.
- [105] A. Daw, J. Bu, S. Wang, P. Perdikaris, and A. Karpatne, "Re-thinking the importance of sampling in physics-informed neural networks," *arXiv preprint arXiv:2207.02338*, 2022.
- [106] C. Wu, M. Zhu, Q. Tan, Y. Kartha, and L. Lu, "A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks," *Computer Methods in Applied Mechanics and Engineering*, vol. 403, p. 115671, 2023.
- [107] M. A. Nabian, R. J. Gladstone, and H. Meidani, "Efficient training of physics-informed neural networks via importance sampling," *Computer-Aided Civil and Infrastructure Engineering*, vol. 36, no. 8, pp. 962–977, 2021.
- [108] A. Katharopoulos and F. Fleuret, "Biased importance sampling for deep neural network training," *arXiv preprint arXiv:1706.00043*, 2017.
- [109] K. Tang, X. Wan, and C. Yang, "Das: A deep adaptive sampling method for solving partial differential equations," *arXiv preprint arXiv:2112.14038*, 2021.
- [110] L. Dinh, J. Sohl-Dickstein, and S. Bengio, "Density estimation using real nvp," *arXiv preprint arXiv:1605.08803*, 2016.
- [111] Y. Gu, H. Yang, and C. Zhou, "Selectnet: Self-paced learning for high-dimensional partial differential equations," *Journal of Computational Physics*, vol. 441, p. 110444, 2021.
- [112] R. Bellman, "Dynamic programming," *Science*, vol. 153, no. 3731, pp. 34–37, 1966.
- [113] K. W. Morton and D. F. Mayers, *Numerical solution of partial differential equations: an introduction*. Cambridge university press, 2005.
- [114] E. Haghighat, A. C. Bekar, E. Madenci, and R. Juanes, "A nonlocal physics-informed deep learning framework using the peridynamic differential operator," *Computer Methods in Applied Mechanics and Engineering*, vol. 385, p. 114012, 2021.
- [115] Y. Lu, H. Chen, J. Lu, L. Ying, and J. Blanchet, "Machine learning for elliptic pdes: fast rate generalization bound, neural scaling law and minimax optimality," *arXiv preprint arXiv:2110.06897*, 2021.
- [116] J. N. Reddy, *Introduction to the finite element method*. McGraw-Hill Education, 2019.
- [117] K. Li, K. Tang, T. Wu, and Q. Liao, "D3m: A deep domain decomposition method for partial differential equations," *IEEE Access*, vol. 8, pp. 5283–5294, 2019.
- [118] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," *Advances in neural information processing systems*, vol. 27, 2014.
- [119] G. Bao, X. Ye, Y. Zang, and H. Zhou, "Numerical solution of inverse problems by weak adversarial networks," *Inverse Problems*, vol. 36, no. 11, p. 115003, 2020.
- [120] G. Lin, P. Hu, F. Chen, X. Chen, J. Chen, J. Wang, and Z. Shi, "Bi-net: learning to solve partial differential equations with boundary integral networks," *arXiv preprint arXiv:2110.00352*, 2021.
- [121] A. E. Hoerl and R. W. Kennard, "Ridge regression: applications to nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 69–82, 1970.
- [122] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996.
- [123] R. Müller, S. Kornblith, and G. E. Hinton, "When does label smoothing help?" *Advances in neural information processing systems*, vol. 32, 2019.
- [124] G. Hinton, O. Vinyals, J. Dean *et al.*, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, vol. 2, no. 7, 2015.
- [125] T. M. Mitchell, *The need for biases in learning generalizations*. Department of Computer Science, Laboratory for Computer Science Research . . . , 1980.
- [126] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.
- [127] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning internal representations by error propagation," *California Univ San Diego La Jolla Inst for Cognitive Science, Tech. Rep.*, 1985.
- [128] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [129] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

- [130] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Icml*, 2010.
- [131] V. Sitzmann, J. Martel, A. Bergman, D. Lindell, and G. Wetzstein, "Implicit neural representations with periodic activation functions," *Advances in Neural Information Processing Systems*, vol. 33, pp. 7462–7473, 2020.
- [132] J. C. Wong, C. Ooi, A. Gupta, and Y.-S. Ong, "Learning in sinusoidal spaces with physics-informed neural networks," *arXiv preprint arXiv:2109.09338*, 2021.
- [133] M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, and R. Ng, "Fourier features let networks learn high frequency functions in low dimensional domains," *Advances in Neural Information Processing Systems*, vol. 33, pp. 7537–7547, 2020.
- [134] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," in *European conference on computer vision*. Springer, 2020, pp. 405–421.
- [135] E. Haghighat, M. Raissi, A. Moure, H. Gomez, and R. Juanes, "A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics," *Computer Methods in Applied Mechanics and Engineering*, vol. 379, p. 113741, 2021.
- [136] C. Lin, M. Maxey, Z. Li, and G. E. Karniadakis, "A seamless multiscale operator neural network for inferring bubble dynamics," *Journal of Fluid Mechanics*, vol. 929, 2021.
- [137] R. Mojgani, M. Balajewicz, and P. Hassanzadeh, "Lagrangian pinns: A causality-conforming solution to failure modes of physics-informed neural networks," *arXiv preprint arXiv:2205.02902*, 2022.
- [138] C. Rao, H. Sun, and Y. Liu, "Physics-informed deep learning for incompressible laminar flows," *Theoretical and Applied Mechanics Letters*, vol. 10, no. 3, pp. 207–212, 2020.
- [139] I. E. Lagaris, A. Likas, and D. I. Fotiadis, "Artificial neural networks for solving ordinary and partial differential equations," *IEEE transactions on neural networks*, vol. 9, no. 5, pp. 987–1000, 1998.
- [140] K. S. McFall and J. R. Mahan, "Artificial neural network method for solution of boundary value problems with exact satisfaction of arbitrary boundary conditions," *IEEE Transactions on Neural Networks*, vol. 20, no. 8, pp. 1221–1233, 2009.
- [141] L. Lu, R. Pestourie, W. Yao, Z. Wang, F. Verdugo, and S. G. Johnson, "Physics-informed neural networks with hard constraints for inverse design," *SIAM Journal on Scientific Computing*, vol. 43, no. 6, pp. B1105–B1132, 2021.
- [142] E. Schiassi, R. Furfaro, C. Leake, M. De Florio, H. Johnston, and D. Mortari, "Extreme theory of functional connections: A fast physics-informed neural network method for solving ordinary and partial differential equations," *Neurocomputing*, vol. 457, pp. 334–356, 2021.
- [143] L. Sun, H. Gao, S. Pan, and J.-X. Wang, "Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data," *Computer Methods in Applied Mechanics and Engineering*, vol. 361, p. 112732, 2020.
- [144] H. Gao, M. J. Zahr, and J.-X. Wang, "Physics-informed graph neural galerkin networks: A unified framework for solving pde-governed forward and inverse problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 390, p. 114502, 2022.
- [145] J. Berg and K. Nyström, "A unified deep artificial neural network approach to partial differential equations in complex geometries," *Neurocomputing*, vol. 317, pp. 28–41, 2018.
- [146] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.
- [147] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho, "Lagrangian neural networks," *arXiv preprint arXiv:2003.04630*, 2020.
- [148] P. Toth, D. J. Rezende, A. Jaegle, S. Racanière, A. Botev, and I. Higgins, "Hamiltonian generative networks," *arXiv preprint arXiv:1909.13789*, 2019.
- [149] T. Bertalan, F. Dietrich, I. Mezić, and I. G. Kevrekidis, "On learning hamiltonian systems from data," *Chaos: An Interdisciplinary Journal of Nonlinear Science*, vol. 29, no. 12, p. 121107, 2019.
- [150] A. Zhu, P. Jin, and Y. Tang, "Deep hamiltonian networks based on symplectic integrators," *arXiv preprint arXiv:2004.13830*, 2020.
- [151] Y. D. Zhong, B. Dey, and A. Chakraborty, "Symplectic ode-net: Learning hamiltonian dynamics with control," *arXiv preprint arXiv:1909.12077*, 2019.
- [152] A. Sanchez-Gonzalez, V. Bapst, K. Cranmer, and P. Battaglia, "Hamiltonian graph networks with ode integrators," *arXiv preprint arXiv:1909.12790*, 2019.
- [153] P. Jin, Z. Zhang, A. Zhu, Y. Tang, and G. E. Karniadakis, "Symplets: Intrinsic structure-preserving symplectic networks for identifying hamiltonian systems," *Neural Networks*, vol. 132, pp. 166–179, 2020.
- [154] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.
- [155] Y. Chen, D. Huang, D. Zhang, J. Zeng, N. Wang, H. Zhang, and J. Yan, "Theory-guided hard constraint projection (hcp): A knowledge-based data-driven scientific machine learning method," *Journal of Computational Physics*, vol. 445, p. 110624, 2021.
- [156] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *International Conference on Medical image computing and computer-assisted intervention*. Springer, 2015, pp. 234–241.
- [157] J. F. Thompson, F. C. Thames, and C. W. Mastin, "Automatic numerical generation of body-fitted curvilinear coordinate system for field containing any number of arbitrary two-dimensional bodies," *Journal of computational physics*, vol. 15, no. 3, pp. 299–319, 1974.
- [158] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, and P. W. Battaglia, "Learning mesh-based simulation with graph networks," *arXiv preprint arXiv:2010.03409*, 2020.
- [159] X. Han, H. Gao, T. Pfaff, J.-X. Wang, and L.-P. Liu, "Predicting physics in mesh-reduced space with temporal attention," *arXiv preprint arXiv:2201.09113*, 2022.
- [160] L. Jiang, L. Wang, X. Chu, Y. Xiao, and H. Zhang, "Phygnet: Solving spatiotemporal pdes with physics-informed graph neural network," *arXiv preprint arXiv:2208.04319*, 2022.
- [161] W. Löttsch, S. Ohler, and J. S. Otterbach, "Learning the solution operator of boundary value problems using graph neural networks," *arXiv preprint arXiv:2206.14092*, 2022.
- [162] X. Meng, Z. Li, D. Zhang, and G. E. Karniadakis, "Ppinn: Parareal physics-informed neural network for time-dependent pdes," *Computer Methods in Applied Mechanics and Engineering*, vol. 370, p. 113250, 2020.
- [163] Y. Huang, J. Xu, S. Fang, Z. Zhu, L. Jiang, and X. Liang, "Parallel physics-informed neural networks with bidirectional balance," in *2022 the 6th International Conference on Innovation in Artificial Intelligence (ICIAI)*, 2022, pp. 23–30.
- [164] P. Stiller, F. Bethke, M. Böhme, R. Pausch, S. Torge, A. Debus, J. Vorberger, M. Bussmann, and N. Hoffmann, "Large-scale neural solvers for partial differential equations," in *Smoky Mountains Computational Sciences and Engineering Conference*. Springer, 2020, pp. 20–34.
- [165] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," *arXiv preprint arXiv:1701.06538*, 2017.
- [166] J. Han, A. Jentzen, and E. Weinan, "Solving high-dimensional partial differential equations using deep learning," *Proceedings of the National Academy of Sciences*, vol. 115, no. 34, pp. 8505–8510, 2018.
- [167] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via deepnet based on the universal approximation theorem of operators," *Nature Machine Intelligence*, vol. 3, no. 3, pp. 218–229, 2021.
- [168] S. Wang, H. Wang, and P. Perdikaris, "Learning the solution operator of parametric partial differential equations with physics-informed deepnets," *Science advances*, vol. 7, no. 40, p. eabi8605, 2021.
- [169] —, "Improved architectures and training algorithms for deep operator networks," *Journal of Scientific Computing*, vol. 92, no. 2, pp. 1–42, 2022.
- [170] L. Lu, X. Meng, S. Cai, Z. Mao, S. Goswami, Z. Zhang, and G. E. Karniadakis, "A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data," *Computer Methods in Applied Mechanics and Engineering*, vol. 393, p. 114778, 2022.
- [171] P. Jin, S. Meng, and L. Lu, "Mionet: Learning multiple-input operators via tensor product," *arXiv preprint arXiv:2202.06137*, 2022.
- [172] Z. Mao, L. Lu, O. Marxen, T. A. Zaki, and G. E. Karniadakis, "Deepm&mnet for hypersonics: Predicting the coupled flow

- and finite-rate chemistry behind a normal shock using neural-network approximation of operators," *Journal of computational physics*, vol. 447, p. 110698, 2021.
- [173] S. Cai, Z. Wang, L. Lu, T. A. Zaki, and G. E. Karniadakis, "Deepm&net: Inferring the electroconvection multiphysics fields based on operator approximation by neural networks," *Journal of Computational Physics*, vol. 436, p. 110296, 2021.
- [174] G. Lin, C. Moya, and Z. Zhang, "Accelerated replica exchange stochastic gradient langevin diffusion enhanced bayesian deepoanet for solving noisy parametric pdes," *arXiv preprint arXiv:2111.02484*, 2021.
- [175] A. A. Howard, M. Perego, G. E. Karniadakis, and P. Stinis, "Multifidelity deep operator networks," *arXiv preprint arXiv:2204.09157*, 2022.
- [176] J. Zhang, S. Zhang, and G. Lin, "Multiauto-deepoanet: A multi-resolution autoencoder deepoanet for nonlinear dimension reduction, uncertainty quantification and operator learning of forward and inverse stochastic problems," *arXiv preprint arXiv:2204.03193*, 2022.
- [177] L. Zhang, T. Luo, Y. Zhang, Z.-Q. J. Xu, and Z. Ma, "Mod-net: A machine learning approach via model-operator-data network for solving pdes," *arXiv preprint arXiv:2107.03673*, 2021.
- [178] N. Boullé, C. J. Earls, and A. Townsend, "Data-driven discovery of green's functions with human-understandable deep learning," *Scientific reports*, vol. 12, no. 1, pp. 1–9, 2022.
- [179] C. R. Gin, D. E. Shea, S. L. Brunton, and J. N. Kutz, "Deepgreen: Deep learning of green's functions for nonlinear boundary value problems," *Scientific reports*, vol. 11, no. 1, pp. 1–14, 2021.
- [180] B. Khara, A. Balu, A. Joshi, A. Krishnamurthy, S. Sarkar, C. Hegde, and B. Ganapathysubramanian, "Field solutions of parametric pdes," 2021.
- [181] Z. Li, N. B. Kovachki, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar *et al.*, "Fourier neural operator for parametric partial differential equations," in *International Conference on Learning Representations*, 2020.
- [182] S. Cao, "Choose a transformer: Fourier or galerkin," *Advances in Neural Information Processing Systems*, vol. 34, pp. 24 924–24 940, 2021.
- [183] Z. Li, K. Meidani, and A. B. Farimani, "Transformer for partial differential equations' operator learning," *arXiv preprint arXiv:2205.13671*, 2022.
- [184] G. Kissas, J. Seidman, L. F. Guilhoto, V. M. Preciado, G. J. Pappas, and P. Perdikaris, "Learning operators with coupled attention," *arXiv preprint arXiv:2201.01032*, 2022.
- [185] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Neural operator: Graph kernel network for partial differential equations," *arXiv preprint arXiv:2003.03485*, 2020.
- [186] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, A. Stuart, K. Bhattacharya, and A. Anandkumar, "Multipole graph neural operator for parametric partial differential equations," *Advances in Neural Information Processing Systems*, vol. 33, pp. 6755–6766, 2020.
- [187] J. Brandstetter, D. Worrall, and M. Welling, "Message passing neural pde solvers," *arXiv preprint arXiv:2202.03376*, 2022.
- [188] T. Chen and H. Chen, "Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems," *IEEE Transactions on Neural Networks*, vol. 6, no. 4, pp. 911–917, 1995.
- [189] A. Kovacs, L. Exl, A. Kornell, J. Fischbacher, M. Hovorka, M. Gusenbauer, L. Breth, H. Oezelt, M. Yano, N. Sakuma *et al.*, "Conditional physics informed neural networks," *Communications in Nonlinear Science and Numerical Simulation*, vol. 104, p. 106041, 2022.
- [190] S. Wang and P. Perdikaris, "Long-time integration of parametric evolution equations with physics-informed deepoanets," *arXiv preprint arXiv:2106.05384*, 2021.
- [191] K. Kontolati, S. Goswami, M. D. Shields, and G. E. Karniadakis, "On the influence of over-parameterization in manifold based surrogates and deep neural operators," *arXiv preprint arXiv:2203.05071*, 2022.
- [192] X. Huang, Z. Ye, H. Liu, B. Shi, Z. Wang, K. Yang, Y. Li, B. Weng, M. Wang, H. Chu *et al.*, "Meta-auto-decoder for solving parametric partial differential equations," *arXiv preprint arXiv:2111.08823*, 2021.
- [193] P. C. Di Leoni, L. Lu, C. Meneveau, G. Karniadakis, and T. A. Zaki, "Deepoanet prediction of linear instability waves in high-speed boundary layers," *arXiv preprint arXiv:2105.08697*, 2021.
- [194] S. Dong and N. Ni, "A method for representing periodic functions and enforcing exactly periodic boundary conditions with deep neural networks," *Journal of Computational Physics*, vol. 435, p. 110242, 2021.
- [195] Z. Mao, A. D. Jagtap, and G. E. Karniadakis, "Physics-informed neural networks for high-speed flows," *Computer Methods in Applied Mechanics and Engineering*, vol. 360, p. 112789, 2020.
- [196] L. Lu, R. Pestourie, S. G. Johnson, and G. Romano, "Multifidelity deep neural operators for efficient learning of partial differential equations with application to fast inverse design of nanoscale heat transport," *Physical Review Research*, vol. 4, no. 2, p. 023210, 2022.
- [197] K. O'Shea and R. Nash, "An introduction to convolutional neural networks," *arXiv preprint arXiv:1511.08458*, 2015.
- [198] N. Winovich, K. Ramani, and G. Lin, "Convdpde-uc: Convolutional neural networks with quantified uncertainty for heterogeneous elliptic partial differential equations on varied domains," *Journal of Computational Physics*, vol. 394, pp. 263–279, 2019.
- [199] T. J. Grady II, R. Khan, M. Louboutin, Z. Yin, P. A. Witte, R. Chandra, R. J. Hewett, and F. J. Herrmann, "Towards large-scale learned solvers for parametric pdes with model-parallel fourier neural operators," *arXiv preprint arXiv:2204.01205*, 2022.
- [200] L. Chun-Lin, "A tutorial of the wavelet transform," *NTUEE, Taiwan*, vol. 21, p. 22, 2010.
- [201] G. Gupta, X. Xiao, and P. Bogdan, "Multiwavelet-based operator learning for differential equations," *Advances in Neural Information Processing Systems*, vol. 34, pp. 24 048–24 062, 2021.
- [202] T. Tripura and S. Chakraborty, "Wavelet neural operator: a neural operator for parametric partial differential equations," *arXiv preprint arXiv:2205.02191*, 2022.
- [203] Z. Li, D. Z. Huang, B. Liu, and A. Anandkumar, "Fourier neural operator with learned deformations for pdes on general geometries," *arXiv preprint arXiv:2207.05209*, 2022.
- [204] A. Tran, A. Mathews, L. Xie, and C. S. Ong, "Factorized fourier neural operators," *arXiv preprint arXiv:2111.13802*, 2021.
- [205] H. You, Q. Zhang, C. J. Ross, C.-H. Lee, and Y. Yu, "Learning deep implicit fourier neural operators (ifnos) with applications to heterogeneous material modeling," *Computer Methods in Applied Mechanics and Engineering*, vol. 398, p. 115296, 2022.
- [206] M. A. Rahman, Z. E. Ross, and K. Azizzadenesheli, "U-no: U-shaped neural operators," *arXiv preprint arXiv:2204.11127*, 2022.
- [207] Z. Niu, G. Zhong, and H. Yu, "A review on the attention mechanism of deep learning," *Neurocomputing*, vol. 452, pp. 48–62, 2021.
- [208] A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, and M. W. Mahoney, "Characterizing possible failure modes in physics-informed neural networks," *Advances in Neural Information Processing Systems*, vol. 34, pp. 26 548–26 560, 2021.
- [209] R. Luus, "Application of dynamic programming to high-dimensional non-linear optimal control problems," *International Journal of Control*, vol. 52, no. 1, pp. 239–250, 1990.
- [210] G. Williams, *Chaos theory tamed*. CRC Press, 1997.
- [211] M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of machine learning*. MIT press, 2018.
- [212] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural networks*, vol. 2, no. 5, pp. 359–366, 1989.
- [213] A. Yu, C. Becquey, D. Halikias, M. E. Mallory, and A. Townsend, "Arbitrary-depth universal approximation theorems for operator neural networks," *arXiv preprint arXiv:2109.11354*, 2021.
- [214] N. Kovachki, S. Lanthaler, and S. Mishra, "On universal approximation and error bounds for fourier neural operators," *Journal of Machine Learning Research*, vol. 22, pp. Art–No, 2021.
- [215] O. Shamir and T. Zhang, "Stochastic gradient descent for non-smooth optimization: Convergence results and optimal averaging schemes," in *International conference on machine learning*. PMLR, 2013, pp. 71–79.
- [216] Y. Shin, "On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type pdes," *Communications in Computational Physics*, vol. 28, no. 5, pp. 2042–2074, 2020.
- [217] P. Dondl, J. Müller, and M. Zeinhofer, "Uniform convergence guarantees for the deep ritz method for nonlinear problems," *arXiv preprint arXiv:2111.05637*, 2021.
- [218] M. V. de Hoop, N. B. Kovachki, N. H. Nelsen, and A. M. Stuart, "Convergence rates for learning linear operators from noisy data," *arXiv preprint arXiv:2108.12515*, 2021.

- [219] S. Lanthaler, S. Mishra, and G. E. Karniadakis, "Error estimates for deepnets: A deep learning framework in infinite dimensions," *Transactions of Mathematics and Its Applications*, vol. 6, no. 1, p. tnac001, 2022.
- [220] T. De Ryck and S. Mishra, "Error analysis for physics informed neural networks (pinns) approximating kolmogorov pdes," *arXiv preprint arXiv:2106.14473*, 2021.
- [221] M. Mahmoudabadbozchelou, G. E. Karniadakis, and S. Jamali, "nn-pinns: Non-newtonian physics-informed neural networks for complex fluid modeling," *Soft Matter*, vol. 18, no. 1, pp. 172–185, 2022.
- [222] Q. Lou, X. Meng, and G. E. Karniadakis, "Physics-informed neural networks for solving forward and inverse flow problems via the boltzmann-bgk formulation," *Journal of Computational Physics*, vol. 447, p. 110676, 2021.
- [223] G. Wen, Z. Li, K. Azizzadenesheli, A. Anandkumar, and S. M. Benson, "U-fno—an enhanced fourier neural operator-based deep-learning model for multiphase flow," *Advances in Water Resources*, vol. 163, p. 104180, 2022.
- [224] C. Lin, Z. Li, L. Lu, S. Cai, M. Maxey, and G. E. Karniadakis, "Operator learning for predicting multiscale bubble growth dynamics," *The Journal of Chemical Physics*, vol. 154, no. 10, p. 104118, 2021.
- [225] G. Pang, M. D'Elia, M. Parks, and G. E. Karniadakis, "npinns: nonlocal physics-informed neural networks for a parametrized nonlocal universal laplacian operator: algorithms and applications," *Journal of Computational Physics*, vol. 422, p. 109760, 2020.
- [226] X. Jin, S. Cai, H. Li, and G. E. Karniadakis, "Nsfnet (navier-stokes flow nets): Physics-informed neural networks for the incompressible navier-stokes equations," *Journal of Computational Physics*, vol. 426, p. 109951, 2021.
- [227] T. Zhang, B. Dey, P. Kakkar, A. Dasgupta, and A. Chakraborty, "Frequency-compensated pinns for fluid-dynamic design problems," *arXiv preprint arXiv:2011.01456*, 2020.
- [228] H. Gao, L. Sun, and J.-X. Wang, "Super-resolution and denoising of fluid flow using physics-informed convolutional neural networks without high-resolution labels," *Physics of Fluids*, vol. 33, no. 7, p. 073603, 2021.
- [229] M. Li and C. McComb, "Using physics-informed generative adversarial networks to perform super-resolution for multiphase fluid simulations," *Journal of Computing and Information Science in Engineering*, vol. 22, no. 4, p. 044501, 2022.
- [230] S. Cai, Z. Mao, Z. Wang, M. Yin, and G. E. Karniadakis, "Physics-informed neural networks (pinns) for fluid mechanics: A review," *Acta Mechanica Sinica*, pp. 1–12, 2022.
- [231] E. Zhang, M. Yin, and G. E. Karniadakis, "Physics-informed neural networks for nonhomogeneous material identification in elasticity imaging," *arXiv preprint arXiv:2009.04525*, 2020.
- [232] Y. Chen, L. Lu, G. E. Karniadakis, and L. Dal Negro, "Physics-informed neural networks for inverse problems in nano-optics and metamaterials," *Optics express*, vol. 28, no. 8, pp. 11 618–11 633, 2020.
- [233] R. Arora, P. Kakkar, B. Dey, and A. Chakraborty, "Physics-informed neural networks for modeling rate-and temperature-dependent plasticity," *arXiv preprint arXiv:2201.08363*, 2022.
- [234] K. Shukla, P. C. Di Leoni, J. Blackshire, D. Sparkman, and G. E. Karniadakis, "Physics-informed neural network for ultrasound nondestructive quantification of surface breaking cracks," *Journal of Nondestructive Evaluation*, vol. 39, no. 3, pp. 1–20, 2020.
- [235] S. Goswami, C. Anitescu, S. Chakraborty, and T. Rabczuk, "Transfer learning enhanced physics informed neural network for phase-field modeling of fracture," *Theoretical and Applied Fracture Mechanics*, vol. 106, p. 102447, 2020.
- [236] E. Zhang, M. Dao, G. E. Karniadakis, and S. Suresh, "Analyses of internal structures and defects in materials using physics-informed neural networks," *Science advances*, vol. 8, no. 7, p. eabk0644, 2022.
- [237] L. Lu, M. Dao, P. Kumar, U. Ramamurty, G. E. Karniadakis, and S. Suresh, "Extraction of mechanical properties of materials through deep learning from instrumented indentation," *Proceedings of the National Academy of Sciences*, vol. 117, no. 13, pp. 7052–7062, 2020.
- [238] S. Cai, Z. Wang, S. Wang, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks for heat transfer problems," *Journal of Heat Transfer*, vol. 143, no. 6, 2021.
- [239] S. Wang and P. Perdikaris, "Deep learning of free boundary and stefan problems," *Journal of Computational Physics*, vol. 428, p. 109914, 2021.
- [240] A. Bora, W. Dai, J. P. Wilson, and J. C. Boyt, "Neural network method for solving parabolic two-temperature microscale heat conduction in double-layered thin films exposed to ultrashort-pulsed lasers," *International Journal of Heat and Mass Transfer*, vol. 178, p. 121616, 2021.
- [241] P. Clark di Leoni, C. Meneveau, G. Karniadakis, and T. Zaki, "Deep operator neural networks (deeponets) for prediction of instability waves in high-speed boundary layers," in *APS Division of Fluid Dynamics Meeting Abstracts*, 2020, pp. R01–004.
- [242] S. Guan, K.-T. Hsu, and P. V. Chitnis, "Fourier neural operator networks: A fast and general solver for the photoacoustic wave equation," *arXiv preprint arXiv:2108.09374*, 2021.
- [243] A. Bihlo and R. O. Popovych, "Physics-informed neural networks for the shallow-water equations on the sphere," *Journal of Computational Physics*, vol. 456, p. 111024, 2022.
- [244] A. Ivanov and I. Agapov, "Physics-based deep neural networks for beam dynamics in charged particle accelerators," *Physical review accelerators and beams*, vol. 23, no. 7, p. 074601, 2020.
- [245] E. Schiassi, M. De Florio, B. D. Ganapol, P. Picca, and R. Furfaro, "Physics-informed neural networks for the point kinetics equations for nuclear reactor dynamics," *Annals of Nuclear Energy*, vol. 167, p. 108833, 2022.
- [246] M. Barreau, M. Aguiar, J. Liu, and K. H. Johansson, "Physics-informed learning for identification and state reconstruction of traffic density," in *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, 2021, pp. 2653–2658.
- [247] G. S. Misyris, A. Venzke, and S. Chatzivasileiadis, "Physics-informed neural networks for power systems," in *2020 IEEE Power & Energy Society General Meeting (PESGM)*. IEEE, 2020, pp. 1–5.
- [248] R. Li, E. Lee, and T. Luo, "Physics-informed neural networks for solving multiscale mode-resolved phonon boltzmann transport equation," *Materials Today Physics*, vol. 19, p. 100429, 2021.
- [249] Y. Chen and L. Dal Negro, "Physics-informed neural networks for imaging and parameter retrieval of photonic nanostructures from near-field data," *APL Photonics*, vol. 7, no. 1, p. 010802, 2022.
- [250] G. Kissas, Y. Yang, E. Hwuang, W. R. Witschey, J. A. Detre, and P. Perdikaris, "Machine learning in cardiovascular flows modeling: Predicting arterial blood pressure from non-invasive 4d flow mri data using physics-informed neural networks," *Computer Methods in Applied Mechanics and Engineering*, vol. 358, p. 112623, 2020.
- [251] K. Goswami, A. Sharma, M. Pruthi, and R. Gupta, "Study of drug assimilation in human system using physics informed neural networks," *arXiv preprint arXiv:2110.05531*, 2021.
- [252] H. Cavanagh, A. Mosbach, G. Scalliet, R. Lind, and R. G. Endres, "Physics-informed deep learning characterizes morphodynamics of asian soybean rust disease," *Nature communications*, vol. 12, no. 1, pp. 1–9, 2021.
- [253] S. Alkhadhr, X. Liu, and M. Almekkawy, "Modeling of the forward wave propagation using physics-informed neural networks," in *2021 IEEE International Ultrasonics Symposium (IUS)*. IEEE, 2021, pp. 1–4.
- [254] E. Kharazmi, M. Cai, X. Zheng, Z. Zhang, G. Lin, and G. E. Karniadakis, "Identifiability and predictability of integer-and fractional-order epidemiological models using physics-informed neural networks," *Nature Computational Science*, vol. 1, no. 11, pp. 744–753, 2021.
- [255] K. Schütt, P.-J. Kindermans, H. E. Sauceda Felix, S. Chmiela, A. Tkatchenko, and K.-R. Müller, "SchNet: A continuous-filter convolutional neural network for modeling quantum interactions," *Advances in neural information processing systems*, vol. 30, 2017.
- [256] V. G. Satorras, E. Hoogeboom, and M. Welling, "E (n) equivariant graph neural networks," in *International conference on machine learning*. PMLR, 2021, pp. 9323–9332.
- [257] Z. Hao, C. Lu, Z. Huang, H. Wang, Z. Hu, Q. Liu, E. Chen, and C. Lee, "Asgn: An active semi-supervised graph neural network for molecular property prediction," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 731–752.
- [258] J. Zhu, Y. Xia, L. Wu, S. Xie, T. Qin, W. Zhou, H. Li, and T.-Y. Liu, "Unified 2d and 3d pre-training of molecular representations," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 2626–2636.

- [259] W. Jin, R. Barzilay, and T. Jaakkola, "Junction tree variational autoencoder for molecular graph generation," in *International conference on machine learning*. PMLR, 2018, pp. 2323–2332.
- [260] J. You, B. Liu, Z. Ying, V. Pande, and J. Leskovec, "Graph convolutional policy network for goal-directed molecular graph generation," *Advances in neural information processing systems*, vol. 31, 2018.
- [261] D. W. Abueidda, S. Koric, and N. A. Sobh, "Topology optimization of 2d structures with nonlinearities using deep learning," *Computers & Structures*, vol. 237, p. 106283, 2020.
- [262] N. Gebauer, M. Gastegger, and K. Schütt, "Symmetry-adapted generation of 3d point sets for the targeted discovery of molecules," *Advances in neural information processing systems*, vol. 32, 2019.
- [263] E. Hoogeboom, V. G. Satorras, C. Vignac, and M. Welling, "Equivariant diffusion for molecule generation in 3d," in *International Conference on Machine Learning*. PMLR, 2022, pp. 8867–8887.
- [264] F. Bao, M. Zhao, Z. Hao, P. Li, C. Li, and J. Zhu, "Equivariant energy-guided sde for inverse molecular design," *arXiv preprint arXiv:2209.15408*, 2022.
- [265] K. Kashinath, M. Mustafa, A. Albert, J. Wu, C. Jiang, S. Esmailzadeh, K. Azizadenesheli, R. Wang, A. Chattopadhyay, A. Singh *et al.*, "Physics-informed machine learning: case studies for weather and climate modelling," *Philosophical Transactions of the Royal Society A*, vol. 379, no. 2194, p. 20200093, 2021.
- [266] Q. Zheng, L. Zeng, and G. E. Karniadakis, "Physics-informed semantic inpainting: Application to geostatistical modeling," *Journal of Computational Physics*, vol. 419, p. 109676, 2020.
- [267] S. Karimpouli and P. Tahmasebi, "Physics informed machine learning: Seismic wave equation," *Geoscience Frontiers*, vol. 11, no. 6, pp. 1993–2001, 2020.
- [268] A. H. Bukhari, M. A. Z. Raja, M. Shoaib, and A. K. Kiani, "Fractional order lorenz based physics informed sarfima-narx model to monitor and mitigate megacities air pollution," *Chaos, Solitons & Fractals*, vol. 161, p. 112375, 2022.
- [269] M. A. Soriano, H. G. Siegel, N. P. Johnson, K. M. Gutchess, B. Xiong, Y. Li, C. J. Clark, D. L. Plata, N. C. Deziel, and J. E. Saiers, "Assessment of groundwater well vulnerability to contamination through physics-informed machine learning," *Environmental Research Letters*, vol. 16, no. 8, p. 084013, 2021.
- [270] S. R. Vadyala, S. N. Betgeri, J. C. Matthews, and E. Matthews, "A review of physics-based machine learning in civil engineering," *Results in Engineering*, p. 100316, 2021.
- [271] M. Ramezankhani, A. Nazemi, A. Narayan, H. Voggenreiter, M. Harandi, R. Seethaler, and A. S. Milani, "A data-driven multi-fidelity physics-informed learning framework for smart manufacturing: A composites processing case study," *arXiv preprint arXiv:2202.06139*, 2022.
- [272] Q. Zhu, Z. Liu, and J. Yan, "Machine learning for metal additive manufacturing: predicting temperature and melt pool fluid dynamics using physics-informed neural networks," *Computational Mechanics*, vol. 67, no. 2, pp. 619–635, 2021.
- [273] S. Hoyer, J. Sohl-Dickstein, and S. Greydanus, "Neural reparameterization improves structural optimization," *arXiv preprint arXiv:1909.04240*, 2019.
- [274] Y. Kim, C. Yang, Y. Kim, G. X. Gu, and S. Ryu, "Designing an adhesive pillar shape with deep learning-based optimization," *ACS applied materials & interfaces*, vol. 12, no. 21, pp. 24 458–24 465, 2020.
- [275] S. Bi, J. Zhang, and G. Zhang, "Scalable deep-learning-accelerated topology optimization for additively manufactured materials," *arXiv preprint arXiv:2011.14177*, 2020.
- [276] H. Sasaki and H. Igarashi, "Topology optimization accelerated by deep learning," *IEEE Transactions on Magnetics*, vol. 55, no. 6, pp. 1–5, 2019.
- [277] G. Sun and S. Wang, "A review of the artificial neural network surrogate modeling in aerodynamic design," *Proceedings of the Institution of Mechanical Engineers, Part G: Journal of Aerospace Engineering*, vol. 233, no. 16, pp. 5863–5872, 2019.
- [278] M. Raissi, A. Yazdani, and G. E. Karniadakis, "Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations," *Science*, vol. 367, no. 6481, pp. 1026–1030, 2020.
- [279] L. Piloizzi, F. A. Farrelly, G. Marcucci, and C. Conti, "Machine learning inverse problem for topological photonics," *Communications Physics*, vol. 1, no. 1, pp. 1–7, 2018.
- [280] M. Shahnas, D. Yuen, and R. Pysklywec, "Inverse problems in geodynamics using machine learning algorithms," *Journal of Geophysical Research: Solid Earth*, vol. 123, no. 1, pp. 296–310, 2018.
- [281] V. I. Gorbachenko, T. V. Lazovskaya, D. A. Tarkhov, A. N. Vasilyev, and M. V. Zhukov, "Neural network technique in some inverse problems of mathematical physics," in *International Symposium on Neural Networks*. Springer, 2016, pp. 310–316.
- [282] K. Xu and E. Darve, "The neural network approach to inverse problems in differential equations," *arXiv preprint arXiv:1901.07758*, 2019.
- [283] R. Herzog and K. Kunisch, "Algorithms for pde-constrained optimization," *GAMM-Mitteilungen*, vol. 33, no. 2, pp. 163–176, 2010.
- [284] P. T. Boggs and J. W. Tolle, "Sequential quadratic programming," *Acta numerica*, vol. 4, pp. 1–51, 1995.
- [285] X. Sun, T. Xue, S. Rusinkiewicz, and R. P. Adams, "Amortized synthesis of constrained configurations using a differentiable surrogate," *Advances in Neural Information Processing Systems*, vol. 34, pp. 18 891–18 906, 2021.
- [286] A. M. Tartakovsky, C. O. Marrero, P. Perdikaris, G. D. Tartakovsky, and D. Barajas-Solano, "Physics-informed deep neural networks for learning parameters and constitutive relationships in subsurface flow problems," *Water Resources Research*, vol. 56, no. 5, p. e2019WR026731, 2020.
- [287] A. Yazdani, L. Lu, M. Raissi, and G. E. Karniadakis, "Systems biology informed deep learning for inferring parameters and hidden dynamics," *PLoS computational biology*, vol. 16, no. 11, p. e1007575, 2020.
- [288] S. Mowlavi and S. Nabi, "Optimal control of pdes using physics-informed neural networks," *arXiv preprint arXiv:2111.09880*, 2021.
- [289] Z. Hao, C. Ying, H. Su, J. Zhu, J. Song, and Z. Cheng, "Bi-level physics-informed neural networks for pde constrained optimization using broyden's hypergradients," *arXiv preprint arXiv:2209.07075*, 2022.
- [290] R. Liu, J. Gao, J. Zhang, D. Meng, and Z. Lin, "Investigating bi-level optimization for learning and vision from a unified perspective: A survey and beyond," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
- [291] J. Barry-Straume, A. Sarshar, A. A. Popov, and A. Sandu, "Physics-informed neural networks for pde-constrained optimization and control," *arXiv preprint arXiv:2205.03377*, 2022.
- [292] E. A. Antonelo, E. Camponogara, L. O. Seman, E. R. de Souza, J. P. Jordanou, and J. F. Hubner, "Physics-informed neural nets for control of dynamical systems," *arXiv preprint arXiv:2104.02556*, 2021.
- [293] S. Wang, M. A. Bhouri, and P. Perdikaris, "Fast pde-constrained optimization via self-supervised operator learning," *arXiv preprint arXiv:2110.13297*, 2021.
- [294] R. Hwang, J. Y. Lee, J. Y. Shin, and H. J. Hwang, "Solving pde-constrained control problems using operator learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 4, 2022, pp. 4504–4512.
- [295] K. O. Lye, S. Mishra, D. Ray, and P. Chandrashekar, "Iterative surrogate model optimization (ismo): an active learning algorithm for pde constrained optimization with deep neural networks," *Computer Methods in Applied Mechanics and Engineering*, vol. 374, p. 113575, 2021.
- [296] M. Tucci, S. Barmada, L. Sani, D. Thomopoulos, and N. Fontana, "Deep neural networks based surrogate model for topology optimization of electromagnetic devices," in *2019 International Applied Computational Electromagnetics Society Symposium (ACES)*. IEEE, 2019, pp. 1–2.
- [297] S. Doi, H. Sasaki, and H. Igarashi, "Multi-objective topology optimization of rotating machines using deep learning," *IEEE transactions on magnetics*, vol. 55, no. 6, pp. 1–5, 2019.
- [298] R. Pestourie, Y. Mroueh, T. V. Nguyen, P. Das, and S. G. Johnson, "Active learning of deep surrogates for pdes: application to metasurface design," *npj Computational Materials*, vol. 6, no. 1, pp. 1–7, 2020.
- [299] J. Peurifoy, Y. Shen, L. Jing, Y. Yang, F. Cano-Renteria, B. G. DeLacy, J. D. Joannopoulos, M. Tegmark, and M. Soljačić, "Nanophotonic particle simulation and inverse design using artificial neural networks," *Science advances*, vol. 4, no. 6, p. eaar4206, 2018.
- [300] M. C. Messner, "Convolutional neural network surrogate models for the mechanical properties of periodic structures," *Journal of Mechanical Design*, vol. 142, no. 2, p. 024503, 2020.

- [301] J. Zehnder, Y. Li, S. Coros, and B. Thomaszewski, "Ntopo: Mesh-free topology optimization using implicit neural representations," *Advances in Neural Information Processing Systems*, vol. 34, pp. 10 368–10 381, 2021.
- [302] D. Liu, Y. Tan, E. Khoram, and Z. Yu, "Training deep neural networks for the inverse design of nanophotonic structures," *Acs Photonics*, vol. 5, no. 4, pp. 1365–1369, 2018.
- [303] V. Sekar, M. Zhang, C. Shu, and B. C. Khoo, "Inverse design of airfoil using a deep convolutional neural network," *Aiaa Journal*, vol. 57, no. 3, pp. 993–1003, 2019.
- [304] S. Yamasaki, K. Yaji, and K. Fujita, "Data-driven topology design using a deep generative model," *Structural and Multidisciplinary Optimization*, vol. 64, no. 3, pp. 1401–1420, 2021.
- [305] L. Bar and N. Sochen, "Unsupervised deep learning algorithm for pde-based forward and inverse problems," *arXiv preprint arXiv:1904.05417*, 2019.
- [306] N. Takeishi and A. Kalousis, "Physics-integrated variational autoencoders for robust and interpretable generative modeling," *arXiv preprint arXiv:2102.13156*, 2021.
- [307] D. MacKinlay, D. Pagendam, P. M. Kuhnert, T. Cui, D. Robertson, and S. Janardhanan, "Model inversion for spatio-temporal processes using the fourier neural operator."
- [308] Z. Chen, Y. Liu, and H. Sun, "Physics-informed learning of governing equations from scarce data," *Nature communications*, vol. 12, no. 1, pp. 1–13, 2021.
- [309] M. Raissi, "Deep hidden physics models: Deep learning of nonlinear partial differential equations," *The Journal of Machine Learning Research*, vol. 19, no. 1, pp. 932–955, 2018.
- [310] Z. Long, Y. Lu, X. Ma, and B. Dong, "Pde-net: Learning pdes from data," in *International Conference on Machine Learning*. PMLR, 2018, pp. 3208–3216.
- [311] M. Cranmer, A. Sanchez Gonzalez, P. Battaglia, R. Xu, K. Cranmer, D. Spergel, and S. Ho, "Discovering symbolic models from deep learning with inductive biases," *Advances in Neural Information Processing Systems*, vol. 33, pp. 17 429–17 442, 2020.
- [312] S. Jang, S. Yoo, and N. Kang, "Generative design by reinforcement learning: enhancing the diversity of topology optimization designs," *Computer-Aided Design*, vol. 146, p. 103225, 2022.
- [313] N. Ye, K. Li, H. Bai, R. Yu, L. Hong, F. Zhou, Z. Li, and J. Zhu, "Ood-bench: Quantifying and understanding two dimensions of out-of-distribution generalization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 7947–7958.
- [314] Y. Dong, Q.-A. Fu, X. Yang, W. Xiang, T. Pang, H. Su, J. Zhu, J. Tang, Y. Chen, X. Mao *et al.*, "Adversarial attacks on ml defense models competition," *arXiv preprint arXiv:2110.08042*, 2021.
- [315] Y. Chen, X. Mao, Y. He, H. Xue, C. Li, Y. Dong, Q.-A. Fu, X. Yang, W. Xiang, T. Pang *et al.*, "Unrestricted adversarial attacks on imagenet competition," *arXiv preprint arXiv:2110.09903*, 2021.
- [316] X. Yi, Y. Zhou, M. Habermann, S. Shimada, V. Golyanik, C. Theobalt, and F. Xu, "Physical inertial poser (pip): Physics-aware real-time human motion tracking from sparse inertial sensors," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 13 167–13 178.
- [317] E. Gärtner, M. Andriluka, H. Xu, and C. Sminchisescu, "Trajectory optimization for physics-based reconstruction of 3d human pose from monocular video," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 13 106–13 115.
- [318] Q.-s. Zhang and S.-C. Zhu, "Visual interpretability for deep learning: a survey," *Frontiers of Information Technology & Electronic Engineering*, vol. 19, no. 1, pp. 27–39, 2018.
- [319] Y. Zhang, Z. Zhu, H. Su, J. Zhu, S. Zheng, Y. He, and H. Xue, "To make yourself invisible with adversarial semantic contours," *Available at SSRN 4129907*, 2022.
- [320] C. Michaelis, B. Mitzkus, R. Geirhos, E. Rusak, O. Bringmann, A. S. Ecker, M. Bethge, and W. Brendel, "Benchmarking robustness in object detection: Autonomous driving when winter is coming," *arXiv preprint arXiv:1907.07484*, 2019.
- [321] S. G. Finlayson, J. D. Bowers, J. Ito, J. L. Zittrain, A. L. Beam, and I. S. Kohane, "Adversarial attacks on medical machine learning," *Science*, vol. 363, no. 6433, pp. 1287–1289, 2019.
- [322] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [323] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 16 000–16 009.
- [324] R. Gao, Y.-Y. Chang, S. Mall, L. Fei-Fei, and J. Wu, "Objectfolder: A dataset of objects with implicit visual, auditory, and tactile representations," *arXiv preprint arXiv:2109.07991*, 2021.
- [325] R. Gao, Z. Si, Y.-Y. Chang, S. Clarke, J. Bohg, L. Fei-Fei, W. Yuan, and J. Wu, "Objectfolder 2.0: A multisensory object dataset for sim2real transfer," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10 598–10 608.
- [326] J. Wu, J. J. Lim, H. Zhang, J. B. Tenenbaum, and W. T. Freeman, "Physics 101: Learning physical object properties from unlabeled videos," in *BMVC*, vol. 2, no. 6, 2016, p. 7.
- [327] R. Mottaghi, H. Bagherinezhad, M. Rastegari, and A. Farhadi, "Newtonian scene understanding: Unfolding the dynamics of objects in static images," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 3521–3529.
- [328] N. Watters, D. Zoran, T. Weber, P. Battaglia, R. Pascanu, and A. Tacchetti, "Visual interaction networks: Learning a physics simulator from video," *Advances in neural information processing systems*, vol. 30, 2017.
- [329] J. Wu, E. Lu, P. Kohli, B. Freeman, and J. Tenenbaum, "Learning to see physics via visual de-animation," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [330] P. Battaglia, R. Pascanu, M. Lai, D. Jimenez Rezende *et al.*, "Interaction networks for learning about objects, relations and physics," *Advances in neural information processing systems*, vol. 29, 2016.
- [331] H. Chen, J. Gu, O. Gallo, M.-Y. Liu, A. Veeraraghavan, and J. Kautz, "Reblur2deblur: Deblurring videos via self-supervised learning," in *2018 IEEE International Conference on Computational Photography (ICCP)*. IEEE, 2018, pp. 1–9.
- [332] Z. Li, L. Wang, X. Huang, C. Pan, and J. Yang, "Phyir: Physics-based inverse rendering for panoramic indoor images," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 713–12 723.
- [333] B. Huang, L. Pan, Y. Yang, J. Ju, and Y. Wang, "Neural mocon: Neural motion control for physically plausible human motion capture," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 6417–6426.
- [334] X. Fei, A. Wong, and S. Soatto, "Geo-supervised visual depth prediction," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1661–1668, 2019.
- [335] P. Morales, T. Klinghoffer, and S. Jae Lee, "Feature forwarding for efficient single image dehazing," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2019, pp. 0–0.
- [336] T. Chen, P. Wang, Z. Fan, and Z. Wang, "Aug-nerf: Training stronger neural radiance fields with triple-level physically-grounded augmentations," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 15 191–15 202.
- [337] J. Yang, T. Dzanic, B. Petersen, J. Kudo, K. Mittal, V. Tomov, J.-S. Camier, T. Zhao, H. Zha, T. Kolev *et al.*, "Reinforcement learning for adaptive mesh refinement," *arXiv preprint arXiv:2103.01342*, 2021.
- [338] F. Albarrán-Arriagada, J. C. Retamal, E. Solano, and L. Lamata, "Measurement-based adaptation protocol with quantum reinforcement learning," *Physical Review A*, vol. 98, no. 4, p. 042315, 2018.
- [339] A. Mirhoseini, A. Goldie, M. Yazgan, J. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, S. Bae *et al.*, "Chip placement with deep reinforcement learning," *arXiv preprint arXiv:2004.10746*, 2020.
- [340] M. Lutter, J. Silberbauer, J. Watson, and J. Peters, "Differentiable physics models for real-world offline model-based reinforcement learning," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 4163–4170.
- [341] C. Xie, S. Patil, T. Moldovan, S. Levine, and P. Abbeel, "Model-based reinforcement learning with parametrized physical models and optimism-driven exploration," in *2016 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2016, pp. 504–511.
- [342] R. Veerapaneni, J. D. Co-Reyes, M. Chang, M. Janner, C. Finn, J. Wu, J. Tenenbaum, and S. Levine, "Entity abstraction in vi-

- sual model-based reinforcement learning," in *Conference on Robot Learning*. PMLR, 2020, pp. 1439–1456.
- [343] N. Fulton and A. Platzer, "Safe reinforcement learning via formal methods: Toward safe control through proof and learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
 - [344] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, "End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 3387–3395.
 - [345] J. F. Fisac, A. K. Akametalu, M. N. Zeilinger, S. Kaynama, J. Gillula, and C. J. Tomlin, "A general safety framework for learning-based control in uncertain robotic systems," *IEEE Transactions on Automatic Control*, vol. 64, no. 7, pp. 2737–2752, 2018.
 - [346] H. Ma, J. Chen, S. Eben, Z. Lin, Y. Guan, Y. Ren, and S. Zheng, "Model-based constrained reinforcement learning using generalized control barrier function," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 4552–4559.